GIT IN SKY

Agent-Ops

Методология ИИ-агентной эксплуатации и технической поддержки инфраструктуры

White paper

Факты, выводы и рекомендации не являются разрешением на изменение инфраструктуры.

Публичный нормативный релиз-кандидат v0.5.2 | исходная ревизия aom-04-r9 | приоритет английской версии

Git in Sky, 2026

Содержание

О документе . . . 5

Целевая аудитория . . . 5

Как читать этот кандидат . . . 5

Базовые термины и роли . . . 5

Соглашение о терминологии . . . 6

1. Краткое резюме . . . 6

2. Проблемы, которые решает Agent-Ops . . . 8

2.1. Описание инфраструктуры кодом (IaC) не содержит всей эксплуатационной реальности . . . 8

2.2. Наблюдаемость (Observability) даёт сигналы, но не формирует операционный вывод . . . 8

2.3. Языковая модель (LLM) без контракта смешивает факт и предположение . . . 8

2.4. Ручная эксплуатация плохо масштабируется и передаётся . . . 8

2.5. Компрометация агента создаёт риск вредных действий . . . 9

3. Определение и границы методологии . . . 9

3.1. Чем Agent-Ops не является . . . 10

3.2. Agent-Ops и употребления термина AgentOps . . . 10

3.3. Связанные работы и границы доказательств . . . 10

4. Принципы Agent-Ops . . . 11

5. Три стороны управления работой: данные, правила и независимая проверка . . . 13

5.1. Зрелость процесса M0-M6 . . . 15

5.2. Серьёзность результата проверки S0-S5 . . . 16

5.3. Техническая возможность C0-C5 . . . 16

5.4. Операционное воздействие I0-I5 . . . 17

5.5. Автономность R0-R5 . . . 17

5.6. Возможности независимого контролёра T1-T5 . . . 18

5.7. Применение шкал на одном случае . . . 18

6. Намерение (Intent) . . . 19

6.1. Гипотеза о результате (Outcome Hypothesis) и план проверки (validation plan) . . . 20

7. Доказательства (Evidence) . . . 20

7.1. Базовый каталог проверок . . . 21

7.2. Профили сбора (profiles) и стоимость сбора . . . 22

7.3. Диалекты платформ (platform dialects) и поставщики данных (providers) . . . 22
7.4. Готовность наблюдаемости (Observability Readiness) . . . 22
7.5. Проверка, проба и метрика . . . 23
7.6. Число сочетаний меток (cardinality) и актуальность данных (freshness) . . . 23
7.7. Устойчивые идентификаторы наблюдаемости (stable observability identities) . . . 24

8. Диагностика (Diagnosis) . . . 24
8.1. Конвейер диагностики (diagnostics pipeline) . . . 25

9. План (Plan) . . . 25
9.1. Откат (rollback) не всегда существует . . . 26

10. Решение об утверждении (Approval) как типизированная проверяемая запись . . . 26

11. Контролируемое изменение (Controlled Change) . . . 26
11.1. Совместимость с унаследованными настройками (legacy compatibility) . . . 27
11.2. Бюджет попыток контролируемого изменения (Controlled Change) и автоматический размыкатель . . . 28

12. Проверка результата (Verification) . . . 28
12.1. Проверка достигнутого результата (Outcome Verification) . . . 29
12.2. Проверка после изменения (post-change Verification) . . . 29
12.3. Завершение случая не равно успешному достижению результата . . . 29

13. Извлечение уроков (Learning) . . . 30
13.1. Цикл уточнения задачи (Discovery loop) . . . 30
13.2. Возврат инцидента к уточнению (Incident-to-Discovery loop) . . . 30

14. Единый пакет доказательств (Unified Evidence Bundle), трасса и система артефактов . . . 30
14.1. Обязательная трасса процесса . . . 32
14.2. Версионируемый жизненный цикл (versioned lifecycle) . . . 32
14.3. Контракт завершения работы (completion contract) . . . 32
14.4. Форматы артефактов и контракты . . . 32

15. Политики: заявленное состояние (declared state), наблюдаемое состояние (observed state) и действующая политика (effective policy) . . . 33

16. Проектная эксплуатационная обвязка (Project Operations Harness) . . . 33
16.1. Иерархия доверия к источникам (trust hierarchy) . . . 35
16.2. Машиночитаемые разрешения (machine-readable permissions) . . . 35
16.3. Проверка обвязки (Harness validation) . . . 35
16.4. Точки расширения (extension points) . . . 35

16.5. Инвентарь агентных артефактов (agent artifact inventory) . . . 35
16.6. Контрольные ворота допуска интеграции . . . 37

17. Полномочия и решения человека (Human Authority and Approvals) . . . 37

17.1. Карта решений с участием человека (HITL Decision Map) . . . 38
17.2. Границы человеческого решения . . . 38

18. Сквозной слой управляющих правил (Governance Mesh), автономность и операционное воздействие . . . 38

18.1. Квитанции допуска и исхода (admission and outcome receipts) . . . 40
18.2. Технические возможности C0-C5 и граница исполнения . . . 41

19. Плоскость независимой проверки и доверия: независимый контролёр (Guardian) . . . 41

19.1. Независимый контролёр/оценщик (Independent Guardian/evaluator) . . . 42
19.2. Программная проверка (library gate) или ИИ-агент . . . 42
19.3. Наблюдаемость среды исполнения агента и внешний контроллер состояния . . . 43

20. Безопасность, доверие и жизненный цикл контекста агента . . . 43

20.1. Безопасность . . . 43
20.2. Модель доверия . . . 45
20.3. Жизненный цикл контекста агента . . . 46
20.4. Продолжение работы человеком при потере ответа (информативный пример) . . . 48

21. Проверка качества, воспроизводимость и экономика применения ИИ (AI economics) . . . 48

21.1. Проверка качества . . . 49
21.2. Воспроизводимость . . . 51
21.3. Экономика применения ИИ (AI economics) . . . 52
21.4. Полные затраты на внедрение (информативное пояснение) . . . 53

22. Как принципы реализуются в трёх плоскостях . . . 53

23. Практический сценарий: быстрое заполнение диска . . . 54

23.1. Небольшая команда и владелец сервиса (информативный пример) . . . 56

24. Критерии успеха применения методологии . . . 57

25. Внедрение по этапам . . . 58

26. Ожидаемый эффект и позиционирование . . . 59

27. Двуязычная нормативная публикация . . . 59

28. Статус этой редакции . . . 60

Приложение A. 47 базовых проверок . . . 60

Приложение C. Материалы, положенные в основу . . . 62

Приложение D. Нормативный контракт результата . . . 63

Приложение E. Семантика статусов результата проверки . . . 63

Приложение F. Реестр шкал и пространств имён . . . 64

Приложение G. Ограниченный профиль C4 применения после одобрения человеком 1.0.0 . . . 65

G.1. Статус, идентичность и охват . . . 65

G.2. Словарь гарантий и правила заявлений . . . 65

G.3. Каталог допущений . . . 66

G.4. Доверенная вычислительная база . . . 68

G.5. Общие причины отказа и независимость контролёра . . . 70

G.6. Матрица заявлений . . . 70

G.7. Исполняемые формальные модели состояний и переходов и каталог отношений . . . 72

G.8. Публичный срез схем и валидации отношений . . . 75

Приложение H. Профиль сбора доказательств Evidence Acquisition 1.0.0 . . . 75

H.1. Статус, идентичность, охват и нецели . . . 75

H.2. Управляемая цепочка сбора . . . 75

H.3. Независимые оси качества и правило заявления . . . 76

H.4. Доверенная вычислительная база . . . 77

H.5. Семантика отказов и отрицательных доказательств . . . 77

H.6. Интерфейс с ограниченным профилем C4 . . . 78

H.7. Границы общих причин отказа и контрпримеры . . . 78

H.8. Граница кандидата . . . 79

H.9. Интегрированное соответствие и протокол оценки до запуска среды исполнения . . . 79

При расхождениях между русской и английской версиями одной исходной ревизии приоритет имеет английская версия.

О документе

Этот аналитический документ (white paper) формулирует публичную версию методологии Agent-Ops. Она разработана Git in Sky на основе практики аудита, эксплуатации и технической поддержки серверной, облачной и Kubernetes-инфраструктуры, а также обзора опубликованных подходов и продуктов для применения ИИ-агентов в эксплуатации инфраструктуры. Документ описывает основные правила методологии, целевую архитектуру и связи между требованиями, архитектурными решениями и реализацией. Эти связи позволяют проверить, почему принято решение, какие данные и компоненты с ним связаны и где заканчивается ответственность каждого компонента.

Целевая аудитория

Руководители инфраструктуры и технической поддержки, технические директора и директора по информационным технологиям (CTO/CIO), архитекторы, инженеры по автоматизации разработки и эксплуатации (DevOps), инженеры по надёжности сервисов (SRE), владельцы услуг управляемой эксплуатации (managed services) и команды, внедряющие ИИ-агентов в эксплуатацию боевых сред.

Как читать этот кандидат

Эта редакция — проект нормативного описания, а не выпущенный стандарт и не утверждение, что все целевые компоненты уже существуют. Требование описывает обязательное свойство соответствующего процесса; рекомендация указывает предпочтительный подход; пример поясняет правило, но не создаёт нового требования; явно помеченное целевое или будущее состояние описывает направление развития.

Слова **ДОЛЖЕН** и **ОБЯЗАН** обозначают обязательное требование в рамках этого кандидата. **СЛЕДУЕТ** и **НЕ СЛЕДУЕТ** обозначают рекомендацию, отступление от которой требует зафиксированного обоснования. **МОЖЕТ** обозначает допустимый вариант. Эти слова задают силу формулировки, но не предоставляют полномочий на изменение системы.

Базовые термины и роли

Термин	Понятное объяснение в этом документе
Контракт	Согласованные требования к входным данным, результату и ограничениям компонента; здесь это не договор с заказчиком.
Артефакт	Зафиксированный результат работы: доказательство, отчёт, план, запись решения, файл настроек или журнал действий.
Прослеживаемость, или трассируемость	Возможность по результату восстановить, на каких данных он основан, кто принял решение и какое действие было выполнено.
Детерминированный компонент	Программа, обрабатывающая входные данные по заданным правилам. Это способ работы, а не обещание отсутствия ошибок.
Политика	Формализованные правила: что допустимо, кто вправе разрешить действие и при каких условиях.
Проектная эксплуатационная обвязка (Project Operations Harness)	Версионируемый комплект сведений о конкретном проекте: сервисы, владельцы, ограничения, инструкции и разрешённые инструменты.
Типизированная запись	Запись с заранее заданными полями и правилами проверки их значений.
Полномочия и авторитетность источника	Полномочия на исполнение разрешают названному субъекту ограниченное действие. Авторитетность источника показывает, для какого факта этот источник признан. Одно не создаёт другого.

Термин	Понятное объяснение в этом документе
Независимая проверка	Проверка отдельной контрольной ролью или компонентом, которые не утверждают собственную работу. Независимость определяется ролями и полномочиями, а не числом ИИ-агентов.
Извлечение уроков (Learning)	Подготовка предложений по улучшению правил, проверок и инструкций; оно не означает самообучения языковой модели.
Базовая модель (backbone model)	Языковая модель, которую агент фактически использует на конкретном шаге.
Исполняемая формальная модель	Детерминированная спецификация состояний и переходов, которую проверяющая программа обходит для поиска нарушений инвариантов. Это не языковая модель: она не принимает эксплуатационные решения и не доказывает правильность работающей реализации.

Большая языковая модель (LLM) создаёт или анализирует текст. ИИ-агент использует модель, контекст и инструменты в границах одной роли. В Agent-Ops планировщик готовит предложение, уполномоченный человек принимает решение, исполнитель через контрольные ворота (Gated Executor) применяет только одобренное изменение, а независимый контролёр (Guardian) проверяет обязательные артефакты и переходы. Независимая проверка не равна одобрению человека: первая проверяет соблюдение правил и качество доказательств, второе даёт ответственное полномочие. Прямой путь от ответа агента к изменению целевой системы запрещён.

Методология, изначально построенная вокруг ИИ (AI-native), учитывает его безопасное участие в ограниченных ролях анализа и планирования. Соответствие методологии не требует ИИ в каждой роли: детерминированный сервис может выполнять роль, если сохраняет те же контракты и границы полномочий.

Соглашение о терминологии

Имена компонентов, полей схем, статусов и машиночитаемых идентификаторов приводятся точно так, как они заданы в артефактах реализации. Подробный управляемый словарь приведён в самостоятельном [глоссарии Agent-Ops](../normative/glossary.ru.md). Правила перевода и приоритета языков заданы в §27.

История редакций вынесена в отдельный документ «Agent-Ops. История редакций white paper», публикуемый вместе с этим аналитическим документом на agent-ops.ru.

1. Краткое резюме

Эта редакция задаёт требования и указывает ожидаемые преимущества, подлежащие проверке. Она не содержит контролируемой оценки исполнения и не устанавливает сквозную безопасность реализации. Проверка JSON Schema устанавливает соответствие данных закодированным ограничениям; сама по себе она не обеспечивает связи между записями или временные инварианты.

Современная инфраструктура описывается кодом, наблюдается через метрики и логи и изменяется через конвейер непрерывной сборки и поставки (continuous integration / continuous delivery, CI/CD). Однако эксплуатационное знание по-прежнему часто остаётся в головах инженеров, чатах, разрозненных пошаговых инструкциях по типовым эксплуатационным действиям (runbooks) и неформальных исключениях. ИИ-агент получает либо слишком мало контекста, либо слишком широкие полномочия. Оба варианта опасны.

Agent-Ops предлагает сначала создать проверяемый контур намерений, фактов, политик и операционных артефактов, а затем назначать ИИ-агентам строго определённые роли на этапах анализа данных, построения гипотез и планирования изменений:

- детерминированная среда исполнения (runtime) собирает проверяемые доказательства (evidence) и оценивает их по формализованным правилам и порогам политики (policy);

- агент объясняет ситуацию, сопоставляет сигналы, формирует проверяемые гипотезы и готовит предложение;
- человек (инженер) сохраняет право принятия решения и полномочия (authority);
- применение утверждённого изменения выполняет только отдельный исполнитель через контрольные ворота.

При уровне зрелости технологий, принятом в этой редакции, ИИ-агентов НЕ СЛЕДУЕТ использовать внутри исполнителя через контрольные ворота (Gated Executor). Текущая граница методологии требует, чтобы этот исполнитель был детерминированной программой — эксплуатационной инструкцией (runbook), скриптом (script) или формализованным сценарием (playbook), — которую запускает инженер. Ограниченное самовосстановление отдельно обозначено в §18.2 как будущая техническая возможность, выключенная по умолчанию.

Полный управляемый жизненный цикл любого изменения ИТ-инфраструктуры имеет вид:

Намерение -> Доказательства -> Диагностика -> План -> Утверждение -> Контролируемое изменение -> Проверка -> Извлечение уроков

(Intent -> Evidence -> Diagnosis -> Plan -> Approval -> Controlled Change -> Verification -> Learning)



Восемь шагов Agent-Ops: ИИ анализирует и предлагает, человек принимает решение, а одобренное изменение применяет отдельный исполнитель.

Текстовое пояснение. Намерение задаёт цель и критерии успеха. Доказательства дают актуальные факты. Диагностика формирует проверяемые гипотезы, а план предлагает действия, но не исполняет их. До контролируемого изменения уполномоченный человек фиксирует одобрение. Проверка сравнивает результат с исходными критериями, а извлечение уроков готовит улучшения. Недостающие доказательства возвращают случай к сбору; пути в обход одобрения нет.

Двенадцать принципов из раздела 4 сведены здесь к пяти сквозным опорам:

1. Намерение и проверенный результат: действие предваряют явная цель, охват и критерии успеха. Завершение случая определяется разделом 12.3: обязательная работа и доказательства завершены, а владелец принимает конечное решение по случаю. Записанный результат различает успех, неуспех и неопределённость относительно точного намерения; одно закрытие не доказывает успех. Предложения по извлечённым урокам рассматриваются по разделу 13. 2. Доказательства и явно зафиксированная

неопределённость (Evidence and explicit uncertainty): факты, заявленное состояние и интерпретации не смешиваются; каждый вывод ссылается на источник, отметку времени, степень свежести данных и полномочие источника, а отсутствие доказательств остаётся `unknown` (явным статусом «неизвестно») и не превращается в предположение. 3. Детерминированное исполнение в заданных границах (Deterministic bounded execution): сбор данных и исполнение по возможности выполняются ограниченными детерминированными инструментами; утверждённое изменение может применить только отдельный исполнитель через контрольные ворота, который журналирует действия и обеспечивает максимально возможный откат или компенсирующее действие. 4. Управление и ответственность за полномочия (Governance and accountable authority): политики и версионируемая проектная эксплуатационная обвязка ограничивают каждый переход; человек сохраняет право утверждения и принятия решения, а полномочия не передаются неявно. 5. Независимая проверка и прослеживаемость (Independent assurance and traceability): критичные результаты проверяют независимые контрольные ворота (gate) или независимый контролёр, а каждый переход жизненного цикла, решение и содержательная реплика инженера или агента фиксируются в добавляемой без перезаписи трассе; отсутствие обязательной записи блокирует продвижение.

2. Проблемы, которые решает Agent-Ops

2.1. Описание инфраструктуры кодом (IaC) не содержит всей эксплуатационной реальности

Описание инфраструктуры кодом (Infrastructure as Code) задаёт ресурсы и конфигурацию, но не определяет полностью обязательные сигналы, владельцев, допустимые отклонения, разрешённые окна изменений (change windows), правила утверждения (approval rules), диагностические маршруты и доказательства восстановления.

2.2. Наблюдаемость (Observability) даёт сигналы, но не формирует операционный вывод

Prometheus, Loki, Tempo и OpenSearch хранят или индексируют телеметрию (telemetry) — метрики, логи и трассировки, поступающие из работающих систем; Grafana визуализирует и запрашивает её; OpenTelemetry инструментирует, собирает, обрабатывает и экспортирует сигналы. Чтобы превратить сигнал в управляемое решение, нужны применимость, ссылка на конкретное правило (policy reference), степень серьёзности (severity), актуальность данных (freshness), ответственный владелец (owner), рекомендация без права её применить (recommendation) и ограничение полномочий.

2.3. Языковая модель (LLM) без контракта смешивает факт и предположение

Языковая модель не должна самостоятельно выбирать авторитетный источник, разрешённую команду или право изменения боевой среды (production). Без ссылок на доказательства (evidence references) и явных границ полномочий (permission boundaries) агент способен создать убедительное, но неподтверждённое объяснение.

2.4. Ручная эксплуатация плохо масштабируется и передаётся

Команды управляемых услуг и платформенные команды обслуживают разные технологии и уровни критичности. Накопление эксплуатационных знаний о проекте происходит у членов команды в соответствии с процессами, принятыми в этой команде. Из-за этого введение нового человека в команду, не говоря о её замене, требует дополнительных затрат на погружение. Введение в команду аналитического ИИ-агента затруднено из-за необходимости передать ему ту часть информации о проекте, которая не формализована, хранится в памяти, передаётся бессистемно и содержит белые пятна.

2.5. Компрометация агента создаёт риск вредных действий

ИИ-агент может быть скомпрометирован через внедрение вредоносных инструкций в запрос или контекст, подмену модели, навыка или инструмента, кражу учётных данных либо атаку на цепочку поставки. После этого даже формально допустимые инструменты и ранее выданные полномочия могут быть использованы для несанкционированного изменения инфраструктуры, раскрытия данных, закрепления в среде или искажения доказательств.

3. Определение и границы методологии

Agent-Ops — методология организации эксплуатации, в которой:

- детерминированные компоненты собирают и нормализуют доказательства (evidence);
- политики и целевое состояние (expected state) находятся вне кода этих компонентов;
- ИИ-агенты получают ограниченный по составу и объёму контекст (bounded context);
- неопределённость сохраняется как явный статус `unknown` («неизвестно»);
- изменение проходит через намерение, план, решение об утверждении, проверку результата и извлечение уроков.

Контур	Назначение
Намерение (Intent)	Цель, охват и границы, ограничения, критерии успеха и признанный источник постановки.
Доказательства (Evidence)	Инвентаризация, проверки, расхождение фактического состояния с заявленным (drift), телеметрия и подтверждающие записи.
Политики (Policy)	Пороги, роли, автономность, масштаб операционного воздействия, правила утверждения и обращения с данными.
Проектная обвязка (Harness)	Ограничения проекта, владельцы, цели качества сервиса (SLO), эксплуатационные инструкции и контракты инструментов.
Диагностика (Diagnosis)	Хронология событий, гипотезы, противоречия и отсутствующие доказательства.
План устранения (Remediation)	Предложение по изменению, оценка риска, план проверки результата и план отката либо компенсации. На этом этапе никаких действий с целевой системой не производится.
Исполнение (Execution)	Отдельный контур за контрольными воротами с учётными данными, ограниченными по области и сроку, и журналом аудита.
Извлечение уроков (Learning)	Результат, журнал решений, разбор инцидента после закрытия и обновление эксплуатационных правил, хранимых как код (Operations-as-Code).

Случай Agent-Ops — ограниченная работа по эксплуатационному запросу, аудиту, инциденту или предложению об изменении, организованная вокруг явного намерения, доказательств, решений и трассы. Путь из восьми шагов описывает работу, требующую изменения целевой системы. Вместо этого случай может дать проверенное заключение либо принятое владельцем конечное решение без изменения, как описано в разделе 12.3. Отказ изменить систему сам по себе не означает, что исходная цель восстановления сервиса достигнута.

Случай выполняется в рамках ответственности организации за сервис и продукт. Владение сервисом, бизнес-приоритеты, решения о риске и коммуникация с пользователями остаются за ответственными функциями организации; применимая политика и карта решений определяют, кто вправе решать. Доступ к инструменту, авторство намерения и полномочия на исполнение — разные вещи. Выводы и проверенные результаты могут поступать в рецензируемые обновления знаний, работу над надёжностью и задачи разработки по разделу 13. Agent-Ops не требует определённой внешней методологии жизненного цикла продукта.

3.1. Чем Agent-Ops не является

Agent-Ops не заменяет наблюдаемость (observability), управление ИТ-услугами (ITSM), GitLab или описание инфраструктуры кодом (IaC); не является универсальным агентом удалённого доступа с правами администратора; не обещает автоматически устанавливать единственную корневую причину; не запускает автоматическое самовосстановление (self-healing) по умолчанию.

3.2. Agent-Ops и употребления термина AgentOps

AgentOps используется в нескольких значениях, включая эксплуатацию систем ИИ-агентов и применение агентов для эксплуатационных задач. AIOpsLab (v1, 2025) использует этот термин для агентов, работающих на протяжении жизненного цикла инцидента. В этом документе Agent-Ops обозначает конкретную методологию управления и исполнения для эксплуатации и технической поддержки инфраструктуры. Её отличие — предлагаемый контракт и область применения, а не исключительное право на термин.

3.3. Связанные работы и границы доказательств

Это справочное сравнение указывает предшествующие и смежные подходы. Оно не устанавливает приоритет изобретения, доказанное превосходство или соответствие другой системе. Ссылки указывают редакции, использованные для сравнения; текущая ветка main репозитория не подтверждает, какая реализация использовалась для экспериментальных результатов статьи.

Агенты для эксплуатации. Agent-Ops продолжает традицию явных целей управления и обратной связи в автономных вычислениях, описанную в работе Kephart (ICSE 2005). AIOpsLab (v1, 2025) предлагает среду оценки эксплуатационных агентов и предшествующее употребление термина AgentOps. STRATUS (v2, 2026) — смежная многоагентная система обеспечения надёжности. Её транзакционная цель отсутствия регрессии и соблюдение полномочий относятся к разным свойствам; сравнение зависит от согласованных предпосылок о компонентах, выполняющих запись, наблюдении и отмене действий. Жизненный цикл и организация трёх плоскостей в этом документе — адаптация к эксплуатации инфраструктуры, а не заявление об изобретении контуров управления с обратной связью.

Контроль исполнения и полномочия. Минимальные привилегии, безопасный отказ по умолчанию и проверка каждого доступа — устоявшиеся принципы из работы Saltzer and Schroeder (1975). AgentSpec (ICSE 2026, camera-ready) описывает контроль исполнения с предикатами, которые могут учитывать состояние исполнения. Progent (v3, 2026) рассматривает управление привилегиями агентов. Macaroons (IEEE S&P 2014) и Zanzibar (USENIX ATC 2019) дают более ранние подходы к сужению полномочий и согласованной авторизации соответственно. Agent-Ops не заявляет проверки во время исполнения или ограниченные полномочия как новые механизмы; содержательное сравнение включает правила с учётом состояния и доверенные сервисы, от которых они зависят.

Идентичность и среда исполнения агентов. Архитектура нулевого доверия (Zero Trust architecture, NIST SP 800-207, 2020) даёт предшествующие принципы явной проверки, минимальных привилегий и допущения компрометации. Teleport в «From Zero Trust to Agent Trust» (2026) расширяет их для агентов: идентичность, аттестуемая относительно предоставившего полномочия субъекта (grantor), отсутствие постоянных привилегий, доверенная эфемерная среда исполнения, ограничение совокупной автономии группы агентов и допущение дрейфа цели. Agent-Ops опирается на те же предпосылки об идентичности и учётных данных (Приложение G.3, G.4), но размещает границу иначе: доверенная среда ограничивает, к чему может обратиться рассуждающий компонент, тогда как профиль C4 ограничивает множество исполнимых действий и связывает каждое с утверждённым пакетом. Сравнение двух подходов зависит от предпосылки о том, кто инициирует действие; эта редакция не заявляет превосходства и не содержит совместной оценки.

Недоверенные данные и сохраняемый контекст. CaMeL (v2, 2025) и Fides (v2, 2025) рассматривают технически обеспечиваемые границы потоков информации; SPA (v1, 2026) — агентов, сохраняющих состояние между запросами. Сохранение меток доверия не представляется изобретением Agent-Ops. Требования к контексту в §20 описывают предполагаемое операционное поведение; одни текстовые метки и разграничение частей запроса к модели не доказывают, что реализация его обеспечивает.

Происхождение артефактов и контракты действий. in-toto (USENIX Security 2019) — предшествующий подход к проверяемым связям артефактов. PCAA (v1, 2026) и CAVA (v1, 2026) — близкие основания для сравнения управления действиями, канонической идентичности действия, привязки одобрения и квитанций. PACE (v1, 2026) исследует исполнение с аттестацией политики в децентрализованных финансах; предпосылки о целевой системе и результаты этой работы не переносятся автоматически на эксплуатацию инфраструктуры. Компенсация имеет устоявшийся предшествующий подход в Sagas (SIGMOD 1987). Объединение этих механизмов само по себе не доказывает нового свойства безопасности.

Оценка. AIOpsLab и ITBench (ICML 2025) предоставляют среды для оценки эксплуатационных задач. AgentDojo (v3, 2024) даёт основание для сравнения при разделении полезности для законной задачи и успеха атакующего. Прохождение внутренних контрольных ворот, наблюдение разрешённого эффекта и восстановление сервиса — разные измерения. Эти работы помогают проектировать оценку; данная редакция не содержит сравнительных результатов исполнения Agent-Ops, а различие в предмете статьи не доказывает отсутствия возможности у её системы.

Методологические и инженерные источники. AI-Disrupt PDLC v2.0 содержит методологические предшественники шкал R/T, пакета доказательств завершения, сквозного слоя управляющих правил и подходов к управлению контекстом; адаптации указаны в Приложении C. Отчёт Anthropic об агентах для торговли содержит предшествующие приёмы реализации за пределами эксплуатации инфраструктуры. Эти источники отделяются от рецензируемых экспериментальных или формальных доказательств. Цитируемые редакции CaMeL, Fides, Progent, SPA, PCAA, CAVA и PACE — препринты; ссылки признают описанные подходы, но не означают независимой проверки их результатов. Вклад этой редакции состоит в спецификации и организации операционной методологии. Научная новизна и улучшения безопасности, полезности или стоимости пока не установлены.

4. Принципы Agent-Ops

Принцип	Практический смысл
1. Намерение прежде действия (Intent before action)	Действие рассматривается только в контексте явной цели, охвата и критериев успеха.
2. Доказательство прежде вывода (Evidence before inference)	Факт и интерпретация имеют разные схемы и основания доверия.
3. Детерминированное ядро и исполнитель (Deterministic core and Executor)	Сбор, разбор вывода команд, применение правил, запись артефактов, а после утверждения — чтение и изменение целевой системы по возможности выполняются детерминированными инструментами. Исполнитель через контрольные ворота принимает только утверждённый пакет, обеспечивает максимально возможный откат или компенсацию и журналирует каждое действие и результат.
4. Неизвестно ≠ в порядке (Unknown is not OK)	Каждый непроверенный факт явно записывается как `unknown`. Агент не вправе превращать выдуманный или предположенный факт в опору для вывода, рекомендации, решения или действия.
5. Заявленное состояние — тоже доказательство (Declared state is evidence)	Описание инфраструктуры кодом выражает намерение, а работающая система показывает наблюдаемую реальность.
6. Управление внутри процесса (Governance in process)	Политика применяется до планирования и исполнения, а не постфактум.
7. Сбор в границах (Bounded collection)	Предельное время и объём вывода, сокрытие чувствительных данных и запрет неявной сети обязательны. Сетевой доступ разрешён только при явно заданных источнике, цели, назначении, полномочии, сроке, правилах обработки данных и записи доказательства.

Принцип	Практический смысл
8. Обвязка под конкретный проект (Project-specific harness)	Контекст и разрешения версионируются для каждого проекта.
9. Ответственность человека (Human accountability)	Одобрение изменения и право принимать решение не делегируются неявно.
10. Независимая оценка (Independent evaluation)	Критичный результат проверяют независимые контрольные ворота или независимый контролёр.
11. Проверь и извлеки урок (Verify and learn)	Проверенный результат порождает предложения по изменению политик, проверок, инструкций и перечня задач.
12. Полная трасса процесса (Complete process trace)	Каждый переход жизненного цикла, решение и содержательная реплика инженера или агента фиксируются в добавляемой без перезаписи трассе. Отсутствие обязательной записи означает `unknown` и блокирует продвижение.



Двенадцать принципов Agent-Ops: от намерения и доказательств до ответственности человека, независимой проверки, извлечения уроков и полной трассы процесса.

Текстовая альтернатива: двенадцать карточек показывают, что намерение предшествует действию, доказательство — выводу, сбор и исполнение опираются на детерминированные компоненты, неизвестное не считается нормой, управление встроено в процесс, сбор ограничен, обвязка привязана к проекту, решение остаётся за человеком, критичный результат проверяется независимо, а каждый переход записывается и используется для извлечения уроков.

Два положения требуют отдельного пояснения.

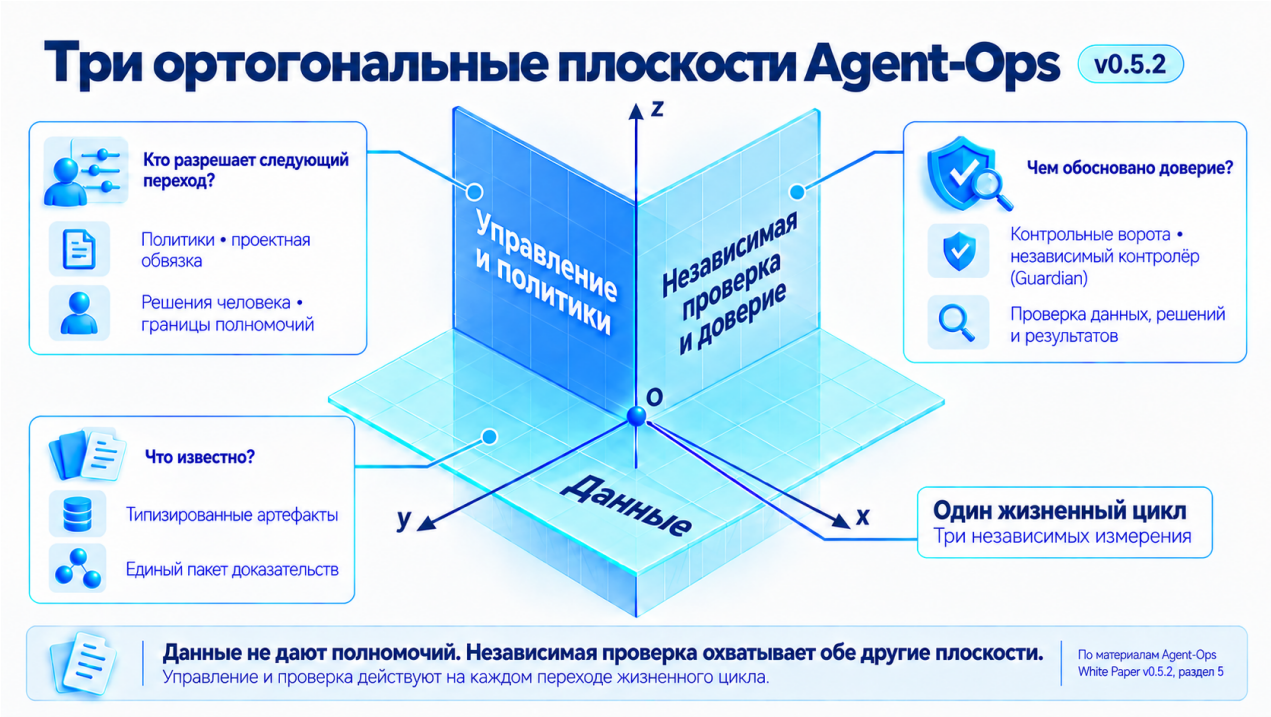
Правило «Неизвестно ≠ в порядке» (Unknown is not OK) относится к уровню знания и не требует угадывать ответ. Если факт невозможно проверить детерминированными средствами, в артефакте фиксируется `unknown`, а агент рассматривает это значение как пробел, который надо либо устранить, либо передать ответственному лицу для решения. Агент может анализировать последствия неопределённости, но не вправе подставлять воображаемый факт вместо доказательства.

Запрет неявной сети (No implicit network) не является полным запретом сетевого сбора. Запрещены необъявленные сетевые обращения, которые могут возникать при разборе данных, разрешении ссылок, загрузке зависимостей, определении сетевых имён или из-за скрытого поведения инструмента по умолчанию. Такие обращения делают прогон невоспроизводимым, могут привести к раскрытию данных

или учётных данных, быть направлены не в ту среду и превращают доступность удалённого ресурса в незаписанную зависимость. Явный, разрешённый политикой и ограниченный сетевой сбор допустим, если он формирует доказательство происхождения и полноты.

5. Три стороны управления работой: данные, правила и независимая проверка

Agent-Ops рассматривает один и тот же жизненный цикл в трёх независимых измерениях. Далее они называются тремя ортогональными плоскостями: ни одна из них не заменяет две другие.



Три плоскости Agent-Ops: данные, управление и политики, независимая проверка и доверие.

Текстовая альтернатива: плоскость данных отвечает, что известно и каким артефактом это представлено; плоскость управления и политик определяет, кто и при каких условиях разрешает переход; плоскость независимой проверки и доверия устанавливает, чем подтверждены данные, решения, переходы и результаты. Данные не предоставляют полномочий, а проверяющая сторона не принимает решение вместо владельца.

Плоскость данных отвечает на вопрос «какие сведения известны и какой типизированный артефакт возникает дальше?». В ней последовательно развиваются намерение, доказательства, диагностика, план, утверждение, контролируемое изменение, проверка результата и извлечение уроков. Единый пакет доказательств (Unified Evidence Bundle) связывает версии этих артефактов, не смешивая факт, вывод, рекомендацию, решение и запись исполнения.

Плоскость управления и политик отвечает на вопрос «кто и при каких условиях вправе разрешить следующий переход?». Политики, проектная эксплуатационная обвязка, решения человека (Human Approvals) и сквозной слой управляющих правил (Governance Mesh) ограничивают переходы, полномочия, воздействие и автономию. Наличие данных или рекомендации само по себе не предоставляет полномочий на исполнение (execution authority).

Плоскость независимой проверки и доверия отвечает на вопрос «чем доказано, что данные, решение, переход и результат заслуживают доверия?». Детерминированные контрольные ворота, независимый контролёр, проверки безопасности, воспроизводимости и полноты, а также оценки поведения (evals) проверяют обе другие плоскости, не наследуя полномочий автора или исполнителя.

Агент диагностики и анализа корневых причин (Diagnosis/RCA Agent), планировщик устранения (Remediation Planner), независимый контролёр и исполнитель через контрольные ворота могут читать артефакты нескольких плоскостей, но не наследуют права друг друга. Исполнитель не получает полномочия из ответа языковой модели и принимает только утверждённый, типизированный и актуальный пакет изменения (change package).

Типизированные агенты-специалисты — необязательная топология реализации, а не требование методологии. Соответствующий методологии процесс может быть реализован одним агентом, несколькими агентами-специалистами или детерминированными сервисами. Нормативны типизированные входы и выходы, изоляция полномочий и проверка независимого контролёра; добавление агентов не предоставляет полномочий и не ослабляет ни одни контрольные ворота.

При передаче между несколькими агентами состояние, отсутствующее в переданном типизированном пакете, не сохраняется. Поэтому каждый узел пути, в котором участвует модель, ДОЛЖЕН отдельно проходить оценку и аттестацию применительно к фактически исполняющей модели и допущенному контексту; аттестация конечных точек не подтверждает промежуточные узлы, включая их устойчивость к инъекции. Детерминированные сборщики и контрольные ворота, а также границы минимальных полномочий остаются самостоятельными контролями, поэтому подтверждение надёжности цепочки нельзя свести к одной оценке модели.

Правила управления и независимые проверки применяются к каждому переходу жизненного цикла и особенно к границам преобразования данных.

Шкала описывает определённое свойство процесса, результата, компонента или действия. Число имеет смысл только вместе с буквой шкалы и объектом оценки: например, серьёзность находки S3 и воздействие действия I3 отвечают на разные вопросы. Уровни нельзя складывать в общий балл или выводить один из другого.

Шкала	**Что оценивается и для чего**	**Плоскости методологии**
Зрелость процесса (Process Maturity) M0-M6	Организация эксплуатации в заданном охвате: какие практики уже действуют и чем подтверждены.	Все три плоскости
Серьёзность результата проверки (Severity) S0-S5	Эксплуатационная значимость находки или неопределённости: насколько существенен риск, который требует рассмотрения.	Данные и независимая проверка
Техническая возможность (Capability) C0-C5	Способность конкретного компонента наблюдать, объяснять, планировать, готовить или выполнять изменение.	Управление и независимая проверка
Операционное воздействие (Operational Impact) I0-I5	Последствия конкретного предлагаемого или выполненного действия: что затронуто, насколько широко и обратимо.	Данные и управление
Автономность (Autonomy) R0-R5	Допустимая инициатива агента для конкретной задачи и среды: где он может продолжать работу и где требуется решение человека.	Управление
Возможности независимого контролёра (Guardian maturity) T1-T5	Способность независимого контроля выявлять, проверять, перенаправлять и блокировать нарушения, а также организовывать их устранение.	Независимая проверка и доверие

Происхождение шкал. Лестница R0-R5 заимствована из таблицы «Риск-адаптивные полномочия» в §4.3 AI-Disrupt PDLC 2.0 — полного руководства Сбера, июнь 2026 года. Автор методологии — Кирилл Меньшов. Обозначения и последовательность T1-T5 взяты из §4.5 того же руководства и адаптированы к границе Agent-Ops: независимый контролёр не получает полномочий на изменение цели, а T5 организует исправление через отдельного уполномоченного исполнителя. Шкалы M, S, C и I ниже заданы самой методологией Agent-Ops; M0-M6 не является переименованием организационной шкалы L0-L5 из AI-Disrupt PDLC.



Шесть независимых шкал Agent-Ops: зрелость процесса M, серьёзность результата S, техническая возможность C, воздействие I, автономность R и независимая проверка T. Пример применения шкал при обработке быстрого заполнения диска: R1/C2/I3.

Текстовая альтернатива: шесть шкал отвечают на разные вопросы и не образуют единый рейтинг. В примере обработки быстрого заполнения диска `R1` ограничивает агента подготовкой предложения, `C2` описывает способность планировать, а `I3` — воздействие предлагаемого изменения внутри одного боевого сервиса. Эти значения не определяют автоматически зрелость процесса M, серьёзность результата S или возможности контролёра T.

5.1. Зрелость процесса M0-M6

Шкала M показывает, насколько устойчиво организована эксплуатация в оцениваемом проекте или сервисе. Уровень подтверждается действующими практиками и их артефактами; установка инструмента или наличие отдельного агента сами по себе не доказывают зрелость. Это характеристика процесса, а не разрешение на действие.

Уровень	Значение	Как проявляется в эксплуатации
M0	Неуправляемый процесс / неизвестное состояние (Unmanaged / Unknown)	Работа не оформлена как управляемый процесс либо его состояние не установлено. Нет подтверждённой опорной картины, на которую можно положиться.
M1	Разовый аудит (Initial Audit)	Получен первый проверяемый снимок состояния и список находок. Регулярное обновление этой картины ещё не является устойчивой практикой.
M2	Регулярная диагностика (Regular Diagnostics)	Проверки повторяются, результаты сопоставляются во времени; отклонения и пробелы обнаруживаются систематически.
M3	Выстроенная наблюдаемость (Observability)	Диагностика связана с актуальными метриками, логами, трассировками и оповещениями; проверяется достаточность этих сигналов для эксплуатации.
M4	Управляемая эксплуатация (Managed Operations)	Работа проходит через явные намерения, доказательства, планы, решения ответственных лиц, проверку результата и журнал; действуют политики и владельцы сервисов.

Уровень	Значение	Как проявляется в эксплуатации
M5	Полуавтоматическое восстановление под утверждением человека (Approval-controlled Semi-automated Recovery)	Подготовка и исполнение ограниченных изменений автоматизированы, но каждое реальное изменение требует решения человека и запуска им детерминированного исполнителя; результат проверяется.
M6	Ограниченное самовосстановление и поддержка надёжности с участием ИИ (Bounded Self-Healing / AI-assisted SRE)	Будущий режим для заранее ограниченных и одобренных сценариев: восстановление выполняется в заданных политикой границах с проверкой, остановкой и отзывом допуска. Эта редакция не вводит его как действующий режим.

5.2. Серьёзность результата проверки S0-S5

Шкала S выражает эксплуатационную значимость результата проверки, включая риск из-за неопределённости. Политика конкретной проверки связывает наблюдения, критичность сервиса и возможные последствия с уровнем S; универсального процента свободного места или единого срока реакции для всех сервисов здесь нет.

Уровень	Значение	Что означает для оценки результата
S0	Информационная (Informational)	Результат даёт сведения о состоянии; сам по себе не указывает на эксплуатационный риск, требующий устранения.
S1	Низкая (Low)	Риск мал и ограничен; находка имеет небольшую эксплуатационную значимость в данном контексте.
S2	Умеренная (Moderate)	Риск заметен и требует рассмотрения; возможные последствия существеннее локальной малозначимой находки.
S3	Значимая (Material)	Результат указывает на существенный эксплуатационный риск, способный заметно ухудшить работу сервиса.
S4	Высокая (High)	Есть высокий риск отказа, потери доступности или нарушения безопасности.
S5	Критическая (Critical)	Критический риск, аварийное состояние или риск потери данных; последствия относятся к наиболее серьёзному классу.

S не заменяет статус `ok`, `warning`, `critical`, `unknown` или другой статус из приложения E. Например, серьёзная неопределённость может требовать внимания, но высокая S не превращает непроверенный факт в подтверждённое отклонение и не разрешает исправление.

5.3. Техническая возможность C0-C5

Шкала C описывает функцию, которую способен выполнять конкретный компонент. Техническая возможность и полномочие на её использование проверяются отдельно. Для C0-C3 сохраняется граница без изменения целевой системы; правила исполнения раскрыты в §18.2.

Уровень	Значение	Результат и граница
C0	Наблюдать (Observe)	Собрать факты и доказательства без изменения цели.
C1	Объяснять (Explain)	Объяснить возможные причины, сформулировать вопросы и указать пробелы в доказательствах.
C2	Планировать (Plan)	Оценить воздействие и подготовить план; изменение ещё не выполняется.
C3	Готовить изменение (Prepare Change)	Подготовить запрос на слияние, план пробного прогона, проверки и отката либо компенсации. Наличие пакета не запускает действий с целью.

Уровень	Значение	Результат и граница
C4	Применять после одобрения человеком (Human-approved Apply)	Отдельный детерминированный исполнитель применяет утверждённое изменение после явного решения и запуска человеком через контрольные ворота.
C5	Ограниченное самовосстановление (Bounded Self-Healing)	Будущая возможность для сценариев малого воздействия с заранее одобренными классом действий, охватом, предусловиями, правилами остановки и отзыва. В этой редакции выключена по умолчанию.

5.4. Операционное воздействие I0-I5

Шкала I относится к действию и его последствиям. Оцениваются критичность, охват затрагиваемых ресурсов, обратимость и класс данных. До изменения оценивается прогнозируемое результирующее состояние, после — фактические последствия; порядок проверки и учёта пересекающихся изменений задан в §11.2. Короткая команда не означает малого воздействия.

Уровень	Значение	Граница воздействия
I0	Без воздействия на боевую среду (Read-only / Offline)	Только чтение или работа без подключения к сети; побочных последствий для боевой среды нет.
I1	Изолированное обратимое изменение вне боевой среды (Isolated Reversible Non-production)	Последствия ограничены изолированным контуром вне боевой среды, а изменение обратимо.
I2	Локальное обратимое изменение низкой критичности (Localized Reversible Low-criticality)	Затрагивается ограниченный компонент низкой критичности; границы последствий и возможность возврата установлены.
I3	Изменение внутри одного боевого сервиса (Production Service-local)	Воздействие ограничено одним боевым сервисом; требуются утверждение человека, проверка результата и план отката.
I4	Межсервисное, критичное или чувствительное изменение (Multi-service / Critical / Sensitive)	Затрагиваются несколько сервисов, критичные функции либо чувствительные данные; применяются разделение обязанностей и выкатка на малую долю.
I5	Системное или необратимое воздействие (Systemic / Irreversible)	Возможны системные последствия, необратимость, потеря данных или регуляторное воздействие; требуется решение высшей уполномоченной инстанции либо действует прямой запрет.

I описывает последствия, а не выдаёт допуск. Неизвестное воздействие не считается I0: по §18 оно приводит к запрету либо эскалации. Одинаковая находка S может иметь несколько планов устранения с разными I.

5.5. Автономность R0-R5

Шкала R задаёт допустимую инициативу агента в конкретной среде и задаче. Ниже сохранён смысл таблицы §4.3 AI-Disrupt PDLC 2.0; правила применения в Agent-Ops приведены в §18. Уровни, связанные с ветками кода и слиянием, относятся к подготовке изменений в репозитории и не дают права самостоятельно менять боевую систему.

Уровень	Значение	Что допускается в указанных границах
R0	Только чтение (Read-only)	Получение сведений; типичная граница — боевая среда без подтверждённых оценок поведения.
R1	Только предложения (Proposals Only)	Агент готовит предложение, решение принимает человек; так ограничивается работа на критичных маршрутах боевой среды.

Уровень	Значение	Что допускается в указанных границах
R2	Работа в отдельной ветке с разбором запроса на слияние (Feature Branch with Review)	Изменения в ветке конкретной задачи проходят разбор перед слиянием.
R3	Автоматическое слияние после успешных оценок (Auto-merge after Evals)	Для стандартных проверенных шаблонов допускается автоматическое слияние только при успешных оценках поведения.
R4	Многосессионная работа с контрольными точками (Multi-session with Checkpoints)	Длительный зрелый и проверенный сценарий продолжается через несколько сессий с сохранением состояния и ограничений.
R5	Полная автономия в изолированной песочнице (Sandbox Autonomy)	Эксперимент выполняется автономно в изолированной среде; эти права не распространяются за её пределы.

У одного агента могут различаться R для боевой среды, ветки кода и песочницы. Уровень определяется политикой с учётом среды, воздействия, доказательств и оценок; агент не повышает его сам. R5 не означает больше прав в боевой среде, чем R1. В этой редакции каждое реальное изменение целевой системы по-прежнему запускает человек через детерминированного исполнителя.

5.6. Возможности независимого контролёра T1-T5

Шкала T описывает, как независимый контроль реагирует на результат или действие другого компонента. Это адаптация лестницы Guardian Agents из §4.5 AI-Disrupt PDLC 2.0 к разделению проверки и исполнения в Agent-Ops. Уровень T не отменяет обязательных контрольных ворот и не позволяет контролёру одобрять собственную работу.

Уровень	Значение	Функция независимого контроля
T1	Разбор (Review)	Рассмотреть выходные артефакты и подготовить отчёт с замечаниями для ответственного лица.
T2	Проверка (Verification)	Проверить соответствие требованиям, политике и доказательствам и зафиксировать результат проверки.
T3	Перенаправление (Redirect)	Направить рискованное или неоднозначное действие на решение человека либо вернуть работу за необходимым уточнением.
T4	Блокировка (Block)	Остановить недопустимое продвижение по правилам, не ожидая, что автор результата сам откажется от действия.
T5	Организация исправления (Remediate)	Передать нарушение в отдельно уполномоченный путь исправления с исполнителем, политикой и единицей работы для отката; затем проверить исправленный результат. Сам контролёр цель не изменяет.

Единица работы (unit of work, UOW) — ограниченная задача реализации со своим охватом и проверкой. Обозначения T описывают возможности контроля, а не степень доверия к ответу модели. Наличие T1 или T2 не освобождает процесс от обязательного блокирования небезопасного перехода: его обеспечивают независимый контролёр и детерминированные контрольные ворота по §19. Внешняя идея автономного исправления на T5 не является действующим полномочием контролёра Agent-Ops.

5.7. Применение шкал на одном случае

При обработке быстрого заполнения диска из §23 шкалы используются совместно, но относятся к разным объектам. Запись R1/C2/I3 означает: агент может только предложить решение, умеет подготовить план, а предлагаемое изменение затронет один боевой сервис. Она не описывает все свойства случая и не является допуском к исполнению.

Шкала	Как применяется в случае с диском
M	Оценивается организация эксплуатации сервиса: есть ли только разовый аудит, регулярная диагностика, наблюдаемость или полный управляемый цикл. Процент свободного места уровень M не определяет.
S	Серьёзность результата определяется по политике проверки с учётом скорости заполнения, критичности сервиса и риска для данных. Исходные 8% не являются универсальным основанием назначить S4 или S5.
C	C2 характеризует компонент, который оценивает варианты и составляет план. Для последующего применения нужен отдельный исполнитель C4 с требуемым решением человека.
I	I3 относится к предлагаемому расширению тома и исправлению правила срока хранения внутри одного боевого сервиса. Сбор фактов без побочных последствий может иметь I0; это другое действие.
R	R1 запрещает агенту применять своё предложение. Одобрение конкретного плана человеком не превращает этого агента автоматически в исполнителя.
T	Учитываются реально действующие функции контроля: разбор, проверка, перенаправление, блокировка или организация исправления. Один успешный отчёт не доказывает наличие всех уровней.

M, S и T устанавливаются по собственным основаниям; из сочетания R1/C2/I3 их значения не выводятся. При записи оценки указываются объект, основание, применимая политика и актуальные доказательства. Порядок допуска определяется §18, а единый реестр значений и ссылки на определения собраны в приложении F.

> *Машиночитаемый контекст — это данные, а не команда и не утверждение.*
>
> *(Machine-readable context is data, not a command and not an approval.)*
>
> *Допустимый предел воздействия задаётся политикой; прогнозируемое и фактическое воздействие фиксируются как данные и доказательства.*

6. Намерение (Intent)

Намерение начинает плоскость данных: человеческая цель становится типизированным, проверяемым и версионируемым контрактом, но ещё не командой и не полномочием на изменение.

Карточка типа	Контракт
Тип	`OperationalIntent`
Вход	Заявка, инцидент, цель владельца и исходные ограничения.
Выход	Операционное намерение (Operational Intent) с указанием версии и ссылкой на обязательную итоговую проверку результата (Outcome Check).
Ключевые поля	`ref`, `owner_ref`, `objective`, `expected_outcome`, `baseline`, `measurable_targets`, `scope`, `constraints`, `prohibited_actions`, `falsification_criteria`, `required_outcome_check`, `decision_map_ref`.
Схема	[`foundation_operational_intent.schema.json`](../../schemas/foundation_operational_intent.schema.json).
Статус контракта	Нормативная схема JSON существует.

Операционное намерение переводит человеческую цель в проверяемый контракт до начала анализа или планирования. Оно не является командой и не предоставляет полномочий на исполнение изменений.

Поле	Семантика
`intent_id` / `version`	Стабильная идентичность и версия постановки.

Поле	Семантика
`goal` / `expected_outcome` (цель и ожидаемый результат)	Желаемый результат без навязывания способа реализации.
`scope` (охват)	Проект, среда, сервисы и исключения.
`constraints` (ограничения)	Запрещённые действия, временное окно, правила обращения с данными и максимальный допустимый масштаб последствий (blast-radius ceiling).
`success` / `stop criteria` (критерии успеха и остановки)	Проверяемое завершение и условия остановки.
`authority`	Кто сформулировал и кто может изменить намерение.
`policy` / `evidence refs`	Применимые правила и исходные доказательства.
`validity` (актуальность)	Свежесть постановки и срок её актуальности.

6.1. Гипотеза о результате (Outcome Hypothesis) и план проверки (validation plan)

До реализации фиксируются исходное состояние (baseline), измеримое целевое значение (target), временное окно, способ измерения и решение о доработке либо полном перепроектировании (adapt-versus-redesign). План проверки определяет обязательные оценки поведения, контрольные ворота, элементы пакета доказательств и итоговую проверку результата.

Намерение допускает переход к доказательствам только тогда, когда цель, охват, признанный статус источника постановки, критерии успеха и остановки достаточно определены. Неустранённая неоднозначность остаётся `unknown` и возвращает постановку к процессу уточнения задачи (Discovery).

7. Доказательства (Evidence)

Доказательства фиксируют наблюдаемую и заявленную реальность до интерпретации. Сбор не должен незаметно превращаться ни в диагноз, ни в разрешение на действие.

Независимо версионируемый профиль `agent-ops.evidence-acquisition@1.0.0` задаёт границу сбора доказательств в [Приложении H](#приложение-h-профиль-сбора-доказательств-evidence-acquisition-100). Он разделяет требования, попытки сбора, артефакты, утверждения и пакеты, устанавливает независимые оси качества доказательств и сохраняет видимость сбоев сбора. Его ограниченное семейство записей использует новые идентичности схем и Evidence Bundle v2 (пакет доказательств версии 2), не меняя смысл Evidence Bundle v1 (пакета доказательств версии 1).

Карточка типа	Контракт
Тип	`EvidenceRecord / EvidenceBundle`
Вход	Операционное намерение, авторитетные источники, профиль сбора, диалект платформы и политика.
Выход	Типизированные результаты проверок, снимки состояния и ссылки на пакет доказательств.
Ключевые поля	`ref`, идентичность цели, источник, отметка времени, свежесть, полномочие источника, полнота, усечение, статус, степень серьёзности и свёртка.
Схема	[`check_result.schema.json`](../../schemas/check_result.schema.json), [`foundation_evidence_bundle.schema.json`](../../schemas/foundation_evidence_bundle.schema.json), [`declared_inventory.schema.json`](../../schemas/declared_inventory.schema.json), [`foundation_object_ref.schema.json`](../../schemas/foundation_object_ref.schema.json).
Статус контракта	Нормативные схемы существуют; пакет доказательств развивается на протяжении всего цикла.

Детерминированный механизм сбора доказательств (Deterministic Evidence Engine) — базовый слой Agent-Ops. Его задача не «починить сервер», а получить проверяемые факты, нормализовать их,

оценить относительно политики и сформировать машиночитаемые результаты, пригодные для автоматической обработки без участия человека.

Свойство	Контракт
Идентичность (Identity)	`check_id`, идентичность проекта, среды и сервиса, версии инструмента и схемы данных: [`check_result.schema.json`](../../schemas/check_result.schema.json), [`foundation_object_ref.schema.json`](../../schemas/foundation_object_ref.schema.json).
Статус (Status)	`ok`, `info`, `warning`, `critical`, `unknown`, `not_applicable`, `error` (в порядке; информация; предупреждение; критично; неизвестно; неприменимо; техническая ошибка): [`check_result.schema.json`](../../schemas/check_result.schema.json).
Доказательство (Evidence)	Источник, временная отметка, актуальность, криптографическая свёртка содержимого, полнота, а также факт и причина усечения данных: [`check_result.schema.json`](../../schemas/check_result.schema.json), [`foundation_evidence_bundle.schema.json`](../../schemas/foundation_evidence_bundle.schema.json).
Безопасность (Safety)	Сбор только на чтение, ограниченный вывод, сокрытие чувствительных данных, запрет неявных сетевых обращений и границы исполнения: [`check_card.schema.json`](../../schemas/check_card.schema.json), [`run_envelope.schema.json`](../../schemas/run_envelope.schema.json).
Воспроизводимость (Reproducibility)	Канонический порядок элементов, детерминированные модули записи, явно указанный профиль сбора и политика: [`run_request.schema.json`](../../schemas/run_request.schema.json), [`run_plan.schema.json`](../../schemas/run_plan.schema.json), [`foundation_evidence_completion_profile.schema.json`](../../schemas/foundation_evidence_completion_profile.schema.json).
Авторитетность источника и полномочия (Authority)	Заявленные, наблюдаемые и выведенные данные не смешиваются; признанный статус источника и полномочия на исполнение записываются отдельно: [`foundation_semantic_object.schema.json`](../../schemas/foundation_semantic_object.schema.json), [`foundation_governance_result.schema.json`](../../schemas/foundation_governance_result.schema.json).

проба -> ограниченное доказательство -> разбор -> применение правил -> результат проверки -> запись артефактов

(probe -> bounded evidence -> parser -> policy evaluation -> check result -> artifact writers)

7.1. Базовый каталог проверок

Проверка является самостоятельным модулем с контрактом [`check_card.schema.json`](../../schemas/check_card.schema.json) и результатом по [`check_result.schema.json`](../../schemas/check_result.schema.json). Каталог из 47 проверок служит исходной опорной точкой, а не пределом: конкретная реализация может содержать 100 и более проверок. Новая проверка получает стабильный `check_id`, роль хоста или сервиса, технологию, риск, схему, пример результата, детерминированную реализацию, доказательство того, что проверка сама протестирована, и статус зрелости. Добавление модуля расширяет версионируемый каталог и не меняет смысл существующих `check_id`.

7.2. Профили сбора (profiles) и стоимость сбора

Профиль аудита является самостоятельным версионируемым модулем композиции: он выбирает проверки, их параметры, пределы времени и объёма, но не меняет смысл проверок и не предоставляет полномочия на изменение. `audit-fast`, `audit-regular`, `audit-deep` и `continuous-check` — примеры, а не закрытый перечень; конкретная реализация может определять 10 и более профилей. Новый идентификатор профиля явно добавляется в применимую версию контракта [`check_card.schema.json`](../../schemas/check_card.schema.json), выбирается через `profile_ref` в [`run_request.schema.json`](../../schemas/run_request.schema.json), а разрешённый набор вызовов фиксируется в [`run_plan.schema.json`](../../schemas/run_plan.schema.json). `audit-fast` содержит дешёвые проверки только на чтение; `audit-regular` выполняет основной набор; `audit-deep` включает тяжёлую диагностику; `continuous-check` выдаёт сигналы с малым числом различных сочетаний меток (low-cardinality signals), чтобы не создавать чрезмерную нагрузку на хранилище метрик; профили планирования устранения не предоставляют права изменения.

7.3. Диалекты платформ (platform dialects) и поставщики данных (providers)

Диалект платформы является самостоятельным модулем-адаптером по контракту [`dialect.schema.json`](../../schemas/dialect.schema.json): он сопоставляет неизменный логический `check_id` с поставщиками данных и разборщиками конкретной платформы. Перечень диалектов открыт и версионируется; реализация может содержать 20 и более диалектов. Добавление диалекта не создаёт новый смысл проверки и не требует изменения других диалектов. Доказанная неприменимость даёт `not_applicable`; сбой ожидаемого поставщика данных или маршрута сбора данных даёт `error`; неразрешённая применимость либо недостаточный объём доказательств дают `unknown`. Ни одно из этих состояний не эмулирует успешность.

7.4. Готовность наблюдаемости (Observability Readiness)

Agent-Ops не заменяет наблюдаемость. Он проверяет, достаточно ли телеметрии для эксплуатации и анализа инцидентов и связаны ли сигналы с ответственными владельцами сервисов, целями качества сервиса, степенью серьёзности и эксплуатационными инструкциями.

Зона	Что валидируется
Метрики хоста и приложения (Host / application metrics)	Компонент выдачи метрик (exporter), видимость цели для системы опроса метрик (scrape visibility), основные сигналы здоровья сервиса (golden signals), метки метрик и актуальность данных.
Логи (Logs)	Охват, срок хранения, сокрытие чувствительных данных и ссылки на готовые диагностические запросы.
Распределённые трассировки (Traces)	Покрытие критичных маршрутов запросов и атрибуты, связывающие трассировку с логами и метриками.
Оповещения (Alerts)	Правила определения степени серьёзности, маршрутизация адресату, связь с SLO, владелец и ссылки на эксплуатационные инструкции.
Сигналы самого аудита (Audit signals)	Небольшое число сочетаний меток, отметка актуальности, полнота и отсутствие секретов.

Готовность имеет собственную шкалу статусов с приоритетом в порядке `blocked` -> `unknown` -> `gaps` -> `complete`: невозможность проверить сильнее незнания, незнание сильнее обнаруженных пробелов, и ни одно из состояний не сводится к успеху. Обнаруженный состав (observed inventory), заявленное намерение (declared intent) и готовность остаются разными областями авторитетности и полноты: владелец и критичность являются заявленными фактами и никогда не выводятся из обнаруженных компонентов, а локальный след облачного программного сервиса (SaaS) не создаёт полномочий над его плоскостью управления (control plane). Оценка выполняется без сетевых обращений, по артефактам принятого прогона, и не заявляет работоспособность среды исполнения в текущий момент.

7.5. Проверка, проба и метрика

Проба показывает сигнал во времени. Проверка интерпретирует доказательства относительно политики и формирует статус, степень серьёзности и рекомендацию. Метрика может быть входом проверки, но не заменяет результат проверки.

7.6. Число сочетаний меток (cardinality) и актуальность данных (freshness)

Кардинальность метрики (cardinality) — это число различных сочетаний её меток. Метки (labels) — признаки, по которым различаются измерения, например сервис и среда. Для одной метрики каждое сочетание задаёт отдельный временной ряд (time series), то есть последовательность значений во времени. Новое измерение с теми же метками продолжает существующий ряд; новое сочетание меток создаёт другой ряд.



Кардинальность метрик и актуальность данных: стабильные метки сервиса и среды образуют шесть временных рядов с 600 измерениями, а уникальный идентификатор запуска в метках образует 600 рядов; отдельная шкала показывает устаревание доказательств после окончания срока использования.

Текстовая альтернатива: два сервиса и три среды при стабильных метках создают `2 x 3 = 6` временных рядов. Сто запусков добавляют `6 x 100 = 600` измерений в те же шесть рядов. Если поместить уникальный `run_id` в метки метрики, получится `2 x 3 x 100 = 600` отдельных рядов. Кардинальность считает сочетания меток, а актуальность определяет, не превысил ли возраст доказательства допустимый срок использования.

Пример для одной метрики	Временные ряды	Измерения после 100 запусков
Метки содержат только два сервиса и три среды	`2 x 3 = 6`	`6 x 100 = 600` значений в тех же шести рядах
Метки также содержат уникальный идентификатор запуска	`2 x 3 x 100 = 600`	600 рядов, каждый создан для одного запуска

Поэтому уникальные идентификаторы запусков и временные отметки (timestamps) хранятся в артефактах JSON, а не в метках метрик. Числа поясняют различие и не задают универсального предела кардинальности.

Актуальность данных (freshness) — отдельное свойство: оно показывает, как давно получено доказательство и не истёк ли допустимый срок его использования (TTL). Набор может содержать всего шесть рядов, но уже быть слишком старым для решения. Устаревшее доказательство не обосновывает рискованное изменение без повторного сбора.

Для профиля Evidence Acquisition (сбора доказательств) актуальность оценивает получатель в момент использования относительно требования к доказательству, идентичностей цели и источника, интервала наблюдения, неопределённости часов и каждой версии, критичной для решения. TTL является одним из входов, а не универсальным доказательством пригодности. Изменение версии или поколения цели, непокрытый интервал либо неограниченная неопределённость часов сохраняют зависимое утверждение в состоянии `unknown`, пока текущее доказательство не выполнит требование.

Для ограниченного профиля C4 актуальность относится к конкретному решению, а не определяется только сроком использования (TTL). Время наблюдения, время получения, время использования, заявленная неопределённость часов, точный UID цели и каждая версия ресурса, критичная для решения, являются разными входными данными. Неистёкший TTL не доказывает актуальность после изменения критичной для решения версии. Допущения и границы заявлений заданы в [Приложении G](#приложение-g-ограниченный-профиль-c4-применения-после-одобрения-человеком-100);
исполняемые отношения задают проверку актуальности Evidence (доказательства) в момент использования получателем (`EVD-R09`), распространение инвалидации после изменения критичной версии (`EVD-R14`) и передачу результата проверки актуальности точному решению C4 (`CEI-R02`).

7.7. Устойчивые идентификаторы наблюдаемости (stable observability identities)

`service_id`, `environment_id`, `slo_id`, `alert_id`, `runbook_id`, `telemetry_requirement_id` и `owner_id` должны оставаться стабильными, чтобы пакет материалов по инциденту (Incident Bundle) ссылался на доказательства без изменения базовых схем.

Переход к диагностике разрешён только при явной оценке полноты и свежести. `unknown`, `error`, `not_applicable` и усечение сохраняют собственную семантику и не эмулируют успешный сбор.

8. Диагностика (Diagnosis)

Диагностика преобразует доказательства в проверяемые гипотезы, не смешивая наблюдение и вывод.

Карточка типа	Контракт
Тип	`DiagnosisRecord`
Вход	Намерение, пакет доказательств, события инцидента и применимые диагностические правила.
Выход	Ранжированные гипотезы, противоречия, пробелы и проверки на опровержение.
Ключевые поля	`diagnosis_ref`, `intent_ref`, `evidence_refs`, `observations`, `hypotheses`, `supporting_evidence`, `contradicting_evidence`, `missing_evidence`, `falsification_checks`, `confidence`.
Схема	[`diagnostic_report.schema.json`](../../schemas/diagnostic_report.schema.json) и [`foundation_discovery_assessment.schema.json`](../../schemas/foundation_discovery_assessment.schema.json) покрывают части типа.
Статус контракта	Единая нормативная схема JSON для `DiagnosisRecord` пока отсутствует; документ не выдаёт частичное покрытие за реализованный контракт.

Рабочий процесс обработки инцидента отвечает на вопрос о том, что менялось, какие сигналы наблюдались и какие гипотезы объясняют симптомы. Он использует единый пакет доказательств, но не смешивает факт и вывод.

Поле гипотезы	Назначение
`statement` (формулировка)	Проверяемая формулировка.

Поле гипотезы	Назначение
`confidence` (уверенность)	`low` / `medium` / `high` (низкая, средняя, высокая) по явным критериям.
`supporting_evidence`	Факты, подтверждающие гипотезу.
`contradicting_evidence`	Факты, которые ей противоречат.
`missing_evidence`	Каких данных не хватает.
`falsification_checks` (проверки на опровержение)	Ограниченные действия для опровержения — попытка не подтвердить гипотезу, а именно её опровергнуть.
`likely_root_cause` (вероятная корневая причина)	Флаг, не заменяющий проверку человеком.

> *Корректный анализ корневых причин (RCA) — ранжированные гипотезы с подтверждающими и опровергающими доказательствами, а не уверенный текст без доказательств.*

8.1. Конвейер диагностики (diagnostics pipeline)

Первый уровень диагностики (Tier 1) использует дешёвые счётчики псевдофайловых систем ядра Linux (`procfs`/`sysfs`) и сведения о состоянии среды исполнения; второй уровень (Tier 2) использует инструменты, подключаемые только по явному согласию; третий уровень (Tier 3) использует встроенный механизм eBPF только при отдельном разрешении и подтверждённой поддержке. Сбор остаётся ограниченным и выполняется в две фазы: снятие опорного состояния (baseline) и выборочные измерения (sampling).

Диагностика переходит к плану только при прослеживаемой связи каждой существенной гипотезы с подтверждающими и опровергающими доказательствами. Недостаток данных порождает новый запрос доказательств, а не уверенный вывод.

9. План (Plan)

План (Plan) описывает предлагаемый способ достижения намерения. На этом этапе никаких действий с целевой системой не выполняется.

Карточка типа	Контракт
Тип	`ChangePlan`
Вход	Намерение, диагностика, пакет доказательств, действующая политика и ограничения проектной обвязки.
Выход	План действий, воздействия, проверки, отката или компенсации и требуемых утверждений.
Ключевые поля	`plan_ref`, `intent_ref`, `diagnosis_ref`, `evidence_refs`, `proposed_actions`, `impact`, `preconditions`, `blast_radius`, `dry_run`, `required_approvals`, `verification_plan`, `rollback_or_compensation_plan`.
Схема	[`recommendations.schema.json`](../../schemas/recommendations.schema.json) является экспериментальным артефактом только для планирования (plan-only).
Статус контракта	Единая нормативная схема JSON для `ChangePlan` пока отсутствует.

План устранения (Remediation) — исключительно этап планирования. Планировщик устранения готовит предложение, но не выполняет никаких действий с целевой системой и не применяет предложение. Предложение связано с намерением, инцидентом и пакетом доказательств и содержит предлагаемые действия, предельно допустимый уровень автономии, уровень воздействия, предусловия, ожидаемый эффект, область возможного воздействия, план пробного прогона без реального изменения или выкатки на малую долю (dry-run/canary plan), требуемые утверждения, план проверки результата и план отката либо компенсирующего действия. Применение возможно только позднее через отдельно

уполномоченного исполнителя за контрольными воротами.

9.1. Откат (rollback) не всегда существует

Необратимые действия требуют резервной копии или предварительной проверки, маркировки «обратимо не полностью» (not fully reversible) и плана компенсирующих действий (compensation plan). Отсутствие отката не скрывается общим обещанием обратимости.

План может быть передан на утверждение только вместе с оценкой воздействия, проверяемыми предусловиями, явным планом проверки результата и честным описанием обратимости.

10. Решение об утверждении (Approval) как типизированная проверяемая запись

В плоскости данных решение об утверждении (Approval) — это не просто нажатие кнопки или фраза в переписке, а отдельная адресуемая и проверяемая запись. Организационные правила и полномочия человека раскрываются позднее в §17.

Карточка типа	Контракт
Тип	`HumanDecision`
Вход	Намерение, `ChangePlan`, требуемые доказательства и оценки, область полномочий и точка перехода.
Выход	Решение `approve` или `deny` с областью действия, сроком и подтверждением издателя.
Ключевые поля	`ref`, `intent_ref`, `transition_id`, `principal_ref`, `issuer_attestation_ref`, `actor_role`, `authority_scope`, `blocked_action`, `decision`, `issued_at`, `not_before`, `expires_at`, `revocation_ref`.
Схема	[`foundation_human_decision.schema.json`](../../schemas/foundation_human_decision.schema.json) и [`foundation_decision_map.schema.json`](../../schemas/foundation_decision_map.schema.json).
Статус контракта	Нормативные схемы JSON существуют.

Утверждение не изменяет систему и не расширяет охват плана. Оно действительно только для указанного перехода, точной цели, неизменившихся входов и срока действия. Отсутствующая, просроченная, отозванная или относящаяся к другой цели запись означает отсутствие разрешения.

Ограниченный контракт C4 имеет стабильный идентификатор профиля `agent-ops.c4-human-approved-apply@1.0.0`. В этом профиле человек одобряет один канонический пакет изменения, а затем человек отдельно запускает его детерминированного исполнителя. Само утверждение не является допуском backend (серверной стороны исполнения), не доказывает фактическое воздействие и не разрешает вариант пакета. Приложение G фиксирует охват и допущения; исполняемые отношения задают связывание точного пакета (`C4-R01`), актуальность утверждения в момент допуска (`C4-R02`), актуальный аутентифицированный запуск человеком (`C4-R03`) и авторизованный допуск в объявленной атомарной точке (`C4-R06`).

11. Контролируемое изменение (Controlled Change)

Контролируемое изменение — единственный шаг жизненного цикла, на котором отдельно уполномоченный исполнитель через контрольные ворота может изменить целевую систему. Он принимает только утверждённый типизированный пакет изменения, повторно проверяет точную цель, политику, свежесть и охват и журналирует каждое действие и результат.

Исходный эталонный охват `agent-ops.c4-human-approved-apply@1.0.0` ограничен заранее объявленными типизированными операциями над изолированной небоевой тестовой целью. Произвольное исполнение shell-команд, постоянные полномочия C5, необратимые изменения в production (боевой среде) и произвольные сторонние API без заявленной семантики допуска,

идентичности, повтора, наблюдения и fencing (отсечения устаревшего писателя) находятся вне профиля. Проверка перед обычным вызовом невзаимодействующего удалённого backend (серверной стороны исполнения) НЕ ДОЛЖНА описываться как атомарная авторизация удалённого эффекта.

Карточка типа	**Контракт**
Тип	`ChangePackage` / `ExecutionRecord`
Вход	Неизменившиеся намерение, `ChangePlan`, `HumanDecision`, результат применения правил (Governance Result), ограниченные учётные данные и доказательства состояния до изменения.
Выход	Запись фактически выполненных либо отклонённых действий, конечное состояние и ссылки на сбор состояния после изменения.
Ключевые поля	`case_ref`, `attempt_id`, `attempt_budget`, `attempt_index`, `idempotency_key`, `retry_disposition`, `failure_class`, `cooldown_until`, `breaker_state`, `stop_reason`, `escalation_ref`, `action_ref`, `parent_action_ref`, `target_ref`, `evidence_bundle_ref`, `admitted_identifier_set_ref`, `resolver_ref`, `conflict_domain`, `serialization_lease_ref`, `resulting_state_ref`, `in_flight_effect_refs`, `evaluated_impact`, `package_digest`, `approval_ref`, `policy_ref`, `executor_ref`, `started_at`, `actions`, `results`, `terminal_state`, `rollback_or_compensation_ref`.
Схема	[`c4_change_package.schema.json`](../../schemas/c4_change_package.schema.json), [`c4_execution_record.schema.json`](../../schemas/c4_execution_record.schema.json) и [`c4_authority_receipt.schema.json`](../../schemas/c4_authority_receipt.schema.json). Существующий [`run_plan.schema.json`](../../schemas/run_plan.schema.json) относится к сбору и не подменяет пакет изменения.
Статус контракта	Нормативные схемы покрывают ограниченный срез записей C4. Они не реализуют шлюз, не аутентифицируют происхождение и не доказывают безопасность работающей среды.

Чтение и запись целевой системы по возможности выполняются детерминированными инструментами без участия агента или человека в пути исполнения. Исполнитель не переинтерпретирует план, не повышает автономию и не продолжает при несовпадении свёртки, охвата, полномочий или предусловий. Необратимое действие требует заранее утверждённого плана компенсации и не маскируется словом «откат».

Идентификатор, используемый для выбора цели, ресурса или объекта при утверждении, отображении или изменении, ДОЛЖЕН быть допущенным идентификатором (admitted identifier): выданным либо авторитетно подтверждённым системой без модели, допущенным детерминированным механизмом разрешения (resolver) в актуальный пакет доказательств, связанный с точной целью, и свежим на момент использования. Выход агента, пользователя или делегата не может расширять допущенное множество. Исполнитель не создаёт, не дополняет и не исправляет идентификатор; неразрешимая ссылка приводит к `deny` и новому запросу доказательств, а не к поиску приблизительного совпадения. Интерфейс принятия решения показывает только допущенные идентификаторы и обогащённые сервером записи, производные от них.

11.1. Совместимость с унаследованными настройками (legacy compatibility)

Унаследованные поля `auto_remediation.allowed` и `mode=confirm`, а также профиль `remediate-apply` считаются устаревшими входами, оставленными лишь для совместимости (deprecated compatibility inputs). Они не являются утверждением или полномочиями на исполнение и не могут вызывать изменение состояния системы из среды исполнения аудита.

11.2. Бюджет попыток контролируемого изменения (Controlled Change) и автоматический размыкатель

Для каждого случая политика задаёт единый бюджет всех попыток контролируемого изменения, относящихся к одним и тем же намерению и операционному результату. Новая редакция диагностики или плана не обнуляет этот бюджет. Неуспешная или имеющая исход `unknown` проверка результата расходует попытку. Компенсация является отдельно записываемым эффектом и не доказывает, что операция с потерянным ответом не была применена или что её безопасно повторить. Отношение безопасного повтора (`C4-R08`) разрешает повтор только на основании объявленного контракта операции и ключа backend (серверной стороны исполнения), полного применимого Evidence (доказательства) состояния `not_applied` либо отдельно авторизованной сверки; заявление о безопасности повтора работающей среды всё ещё требует принятых машиночитаемых обязательств валидации и доказательств реализации.

Когда бюджет исчерпан, возник запрещённый класс ошибки или не истёк обязательный период ожидания, автоматический размыкатель переходит в открытое состояние. Он блокирует выдачу и использование полномочий и учётных данных исполнителя для затронутого случая и класса действий и формирует эскалацию; планировщик и исполнитель не могут его закрыть или сбросить. Повторный допуск требует определённых политикой доказательств и, когда это требуется, нового решения человека (`HumanDecision`). Каждое решение о повторе, расходование попытки, переход состояния размыкателя, период ожидания, причина остановки и эскалация дописываются в обязательную трассу процесса.

Попытки, охваты изменения которых пересекаются в определённом политикой домене конфликтующих изменений (mutation conflict domain), сериализуются до завершения проверки результата или окончательного освобождения резервирования по политике. Конкурирующие случаи допускаются детерминированной политикой, а не порядком поступления. Перед каждым допуском воздействие и радиус поражения заново оцениваются для результирующего состояния, прогнозируемого с учётом действия, уже зафиксированного состояния и относящихся к нему незавершённых воздействий. Если это состояние превысит допустимый диапазон `I`, попытка запрещается или эскалируется. Такой результат расходует попытку либо открывает автоматический размыкатель лишь тогда, когда применимая политика объявляет соответствующий запрет, запрещённый класс ошибки или условие попытки; диапазон воздействия и бюджет попыток не взаимозаменяемы.

Политика МОЖЕТ задавать совокупные ограничения на класс действий в целевом домене вне зависимости от числа случаев и субъектов: предельное число допусков за окно, предельный суммарный уровень I результирующего состояния и предельный объём изменяемых ресурсов. Прогнозируемое результирующее состояние учитывает все допущенные и незавершённые воздействия домена, а не только текущего случая. Превышение совокупного ограничения даёт `deny` либо эскалацию; агенты, действующие в одном домене, не считаются согласованными оттого, что каждый по отдельности прошёл допуск.

Ограниченные схемы и валидатор межзаписных отношений обеспечивают монотонность бюджета попыток, непрерывность резервирования и отсечения, связывание результирующего состояния и предусловия безопасного повтора. Они не задают самостоятельный универсальный контракт автоматического размыкателя и не доказывают, что backend (серверная сторона исполнения) реализует эти меры.

12. Проверка результата (Verification)

Проверка результата сопоставляет доказательства состояния после изменения с критериями намерения и создаёт результат (Outcome). Успешное исполнение команды само по себе не доказывает достижение результата.

Карточка типа	Контракт
Тип	`OutcomeCheck` / `Outcome`
Вход	Намерение, `ChangePlan`, `HumanDecision`, `ExecutionRecord` и доказательства состояния после изменения.
Выход	Явный исход `succeeded`, `failed` или `unknown`, доказательства по каждому критерию и остаточные риски.
Ключевые поля	`outcome_ref`, `intent_ref`, `execution_ref`, `snapshot_ref`, `decision_ref`, `criteria`, `evidence_refs`, `evaluation_time`, `evaluation_window`, `residual_risks`, `status`.
Схема	[`foundation_outcome_check.schema.json`](../../schemas/foundation_outcome_check.schema.json) задаёт вход проверки; [`c4_outcome.schema.json`](../../schemas/c4_outcome.schema.json) записывает ограниченный результат C4; [`foundation_validation_result.schema.json`](../../schemas/foundation_validation_result.schema.json) задаёт общий результат валидации.
Статус контракта	Схема `Outcome` для C4 сохраняет независимость фактического эффекта, результата относительно Intent (намерения) и полноты доказательств. Она не является универсальным контрактом результата жизненного цикла.

12.1. Проверка достигнутого результата (Outcome Verification)

Результат фиксирует, достигнута ли цель, какими доказательствами состояния после изменения (after-state evidence) это доказано, какие критерии не выполнены и какие остаточные риски (residual risks) остались. Неполные обязательные доказательства не позволяют закрыть результат.

12.2. Проверка после изменения (post-change Verification)

После изменения выполняются адресные проверки затронутого компонента (targeted checks), аудит и проверка соблюдения целей качества сервиса (SLO validation). Результат связывается с заявкой, запросом на слияние или коммитом и свёрткой доказательств; неуспех становится новым доказательством.

Неполная проверка не превращается в успех: отсутствие обязательного доказательства даёт результату (Outcome) значение `unknown`, а полноте доказательств или оценки — значение `incomplete`. `Incomplete` не является четвёртым значением результата. Результат связан с точными намерением, исполнением, политикой и свёртками доказательств.

12.3. Завершение случая не равно успешному достижению результата

Полный путь из восьми шагов описывает случай, в котором требуется изменение инфраструктуры. Аудит, запрос на диагностику или обращение поддержки может завершиться без контролируемого изменения, если намерение требует только проверенного заключения, доказательства показывают, что изменение не нужно, либо уполномоченный человек отклоняет предложенный план. В трассе фиксируется причина, по которой шаг не выполнялся; пропущенное изменение не изображается вымышленной записью исполнения.

Что оценивается	Значения и смысл
Состояние случая	Открыт, пока не завершены обязательная работа и сбор доказательств; закрывается только с принятым владельцем конечным решением по случаю.
Результат (Outcome)	`succeeded`, `failed` или `unknown` относительно точного намерения. Закрытый случай с `failed` не является успешным результатом.
Полнота доказательств или оценки	`complete` либо `incomplete`; неполнота не является четвёртым значением результата и не допускает успеха.
Результат отдельной проверки	`ok`, `info`, `warning`, `critical`, `unknown`, `not_applicable` или `error` по приложению E.

13. Извлечение уроков (Learning)

Извлечение уроков использует уже проверенный результат, но не изменяет историю случая. Новые правила, проверки, инструкции и задачи становятся новыми версионируемыми артефактами со своими владельцами и процедурами утверждения.

Карточка типа	Контракт
Тип	`LearningRecord`
Вход	Результат, пакет доказательств, диагностика, план, решения, трасса исполнения и остаточные риски.
Выход	Проверяемые предложения об изменении политик, проверок, эксплуатационных инструкций, наблюдаемости и перечня задач.
Ключевые поля	`learning_ref`, `outcome_ref`, `evidence_refs`, `lessons`, `proposed_policy_changes`, `proposed_check_changes`, `runbook_changes`, `observability_gaps`, `backlog_refs`, `owner_ref`.
Схема	Самостоятельная нормативная схема отсутствует.
Статус контракта	Требуется отдельная схема JSON для `LearningRecord`; запись не предоставляет полномочий на применение предложений.

13.1. Цикл уточнения задачи (Discovery loop)

Если неоднозначность блокирует текущий случай или требуется немедленное уточнение, работа возвращается к процессу уточнения задачи. Несрочное улучшение, которое не требуется для решения текущего случая, МОЖЕТ вместо этого попасть в перечень отложенных работ с владельцем, обоснованием и явно записанным пробелом в доказательствах. Ни один маршрут не представляется как успешная автоматизация.

13.2. Возврат инцидента к уточнению (Incident-to-Discovery loop)

Нерешённый инцидент формирует пробелы и задачи в перечне отложенных работ для телеметрии, политик, проверок или эксплуатационных инструкций. Постфактум история не переписывается: новый результат добавляется как доказательство.

Неуспешная проверка результата становится новым доказательством и может открыть новое намерение. Исправленная диагностика или план для тех же цели, охвата и ожидаемого операционного результата остаются в исходном случае и используют общий бюджет попыток. Существенно новые цель, охват или ожидаемый результат требуют явно разрешённого нового намерения; новый идентификатор нельзя использовать для обнуления исчерпанного бюджета. Цикл продолжается новой версией, а не переписыванием исходного диагностического заключения или решения.

14. Единый пакет доказательств (Unified Evidence Bundle), трасса и система артефактов

Раздел объединяет сквозной контракт плоскости данных: все восемь шагов остаются разными типами, но связываются одной идентичностью случая, версиями и добавляемой без перезаписи трассой.

Карточка типа	Контракт
Тип	`LifecycleEvent`
Вход	Состояние до перехода, входные ссылки, решение управления и результат независимой проверки.
Выход	Добавляемая без перезаписи запись попытки перехода и новая версия пакета доказательств.

Карточка типа	Контракт
Ключевые поля	`case_id`, `transition_id`, `from_step`, `to_step`, `attempted_at`, `actor_ref`, `input_refs`, `policy_decision_ref`, `human_decision_ref`, `assurance_ref`, `result`, `output_refs`, `next_allowed_transitions`, `event_digest`.
Схема	[`foundation_evidence_bundle.schema.json`](../../schemas/foundation_evidence_bundle.schema.json) задаёт пакет доказательств; [`c4_lifecycle_event.schema.json`](../../schemas/c4_lifecycle_event.schema.json) и [`c4_checkpoint.schema.json`](../../schemas/c4_checkpoint.schema.json) задают ограниченные записи трассы и контрольной точки C4.
Статус контракта	Схемы C4 покрывают именованный срез профиля, а не универсальный контракт жизненного цикла Agent-Ops.

Единый пакет доказательств — неизменяемый после записи (immutable) пакет, адресуемый по содержимому (content-addressed), для задачи, инцидента или предложения об изменении. Он объединяет ссылки на факты и решения, не копируя без необходимости чувствительные исходные необработанные данные (raw data).

Существующий контракт `foundation_evidence_bundle` v1 сохраняет локальный смысл в этой редакции. Он может содержать ссылки, созданные по профилю Evidence Acquisition (сбора доказательств), но его свёртка, запечатывание или соответствие схеме сами по себе не устанавливают полномочия источника, действительность сбора, охват, актуальность, независимость, поддержку утверждения или полномочия на исполнение. Если после принятия формальных спецификаций отношений потребуется усиленный пакет, он получит новую несовместимую идентичность схемы или новую основную версию и явную заметку о миграции.

Группа	Содержимое
Идентичность (Identity)	Идентичности проекта, среды, сервиса, намерения и инцидента.
Конфигурация (Configuration)	Проектная обвязка, схема и политика, а также версии и свёртки инструментов и агентных артефактов; базовые модели узлов, участвовавших в работе.
Доказательства (Evidence)	Наблюдаемые факты, ссылки на телеметрию, полнота, усечение и актуальность данных.
След рассуждения (Reasoning trace)	Наблюдения, гипотезы, подтверждающие и опровергающие доказательства и пробелы.
След решений (Decision trace)	Рекомендации, решения человека, одобрения изменений и сроки их действия.
Результат (Outcome)	Доказательства проверки результата, остаточный риск и ссылки на извлечённые уроки.

- Канонический порядок элементов, детерминированная свёртка и предельный размер пакета.
- Проверка целостности без обращения к внешним системам (offline verification) и ссылки, содержащиеся в самом репозитории либо явно разрешённые.
- Защита от повторного предъявления ранее собранных данных как свежих (replay), устаревших доказательств и подмены материалов одного проекта материалами другого (cross-project substitution).
- Факты, выводы, рекомендации, решения и записи об исполнении имеют разные типы.
- Пакет не закрывается, если обязательные доказательства неполны или получены из недоверенного источника.

14.1. Обязательная трасса процесса

Цикл «Намерение -> Доказательства -> Диагностика -> План -> Утверждение -> Контролируемое изменение -> Проверка -> Извлечение уроков» имеет добавляемую без перезаписи трассу процесса. Запись создаётся при входе и выходе каждого шага, при каждой попытке перехода, при каждом решении (`allow`, `deny`, `modify`, `escalate`, отложить, утвердить или отклонить) и для каждой содержательной реплики инженера или агента, которая предоставляет доказательство, меняет охват, разрешает неоднозначность, предоставляет полномочие, отказывает в нём или влияет на следующий шаг.

Каждая запись содержит устойчивые идентификаторы случая и перехода, отметку времени, автора и его роль, дословную реплику либо адресуемую по содержимому ссылку, ссылки на входное состояние и доказательства, решение и обоснование, основание полномочия, выходной статус и свёртку, а также следующий разрешённый переход. Чувствительное содержимое может быть вымарано по объявленной политике, но сам факт события, автор, время, факт вымарывания и свёртка остаются видимыми. Резюме может сопровождать исходную запись, но не заменяет её. Отсутствующая, переставленная, изменяемая или не совпадающая с точной целью обязательная запись делает переход `unknown` либо неполным и блокирует продвижение.

14.2. Версионируемый жизненный цикл (versioned lifecycle)

Пакет развивается через версионируемые снимки состояния (versioned snapshots) со статусом `open` или `sealed` (открыт или запечатан), полями `parent_digest` (свёртка предыдущей версии), `supersedes` (указание, какую версию данная заменяет) и событиями аудита, которые можно только дописывать, но не изменять и не удалять (append-only audit events). Исходные свёртки не переписываются; результат добавляется новой версией, а запечатывание запрещено без обязательных доказательств.

14.3. Контракт завершения работы (completion contract)

Контракт завершения Agent-Ops содержит восемь обязательных элементов: изменения либо эксплуатационные артефакты и доказательства их фактического состояния; результаты оценок способности, отсутствия регрессии, сохранения ограничений в длинной сессии и корректной эскалации; отчёт о расходе токенов; добавляемую без перезаписи трассу; подтверждения прохождения контрольных ворот; структурированное обоснование для R2+; вклад в объём результатов относительно исходного уровня; указатель на итоговую проверку результата (Outcome Check).

14.4. Форматы артефактов и контракты

JSON и построчный формат JSON (JSON Lines, JSONL), содержащий по одному объекту JSON на строку и удобный для дописывания, используются для машиночитаемых доказательств; YAML — для политик, проектной обвязки и планов; Markdown — для инженерного объяснения; выдача метрик в формате, который считывает система мониторинга (metrics exposition), — только для сигналов с небольшим числом сочетаний меток. Каждый модуль записи артефакта (writer) определяется контрактом схемы и не переинтерпретирует семантику другого слоя.

Артефакт	Роль
`operational_intent.yaml`	Цель, охват, ограничения, полномочия и критерии успеха.
`check_results.jsonl`	Нормативные результаты проверок.
`declared_inventory.json`	Нормализованное намерение из кода инфраструктуры и конфигурации.
`effective_policy.json`	Применённый итог слияния уровней политик и его свёртка.
`agent_context.json`	Ограниченный пакет контекста для роли.
`evidence_bundle.json`	Единая идентичность, доказательства, трасса решений и ссылки на результаты.
`hypotheses.json`	Гипотезы о корневых причинах, уверенность, противоречия и пробелы.
`remediation_plan.yaml`	Проверяемое предложение, риск, утверждение и проверка результата.
`decision_log.jsonl`	Добавляемые без перезаписи решения, утверждения, сроки действия и ссылки.

Артефакт	Роль
`outcome.json`	Проверка состояния после изменения и остаточные риски.

15. Политики: заявленное состояние (declared state), наблюдаемое состояние (observed state) и действующая политика (effective policy)

Карточка типа	Контракт
Тип	`DeclaredInventory` / `DriftReport` / `GovernancePolicy` / `GovernanceResult`
Вход	Версионируемые источники заявленного состояния, наблюдаемые доказательства, слои политики, авторитетность источников и оцениваемый переход либо действие.
Выход	Классифицированный отчёт о расхождениях и результат применения политики, фиксирующий применённые правила, ограничения, полномочия, автономию, воздействие и исход `allow`, `deny`, `modify` либо `escalate`.
Ключевые поля	`target`, `declared_source`, `observed_source`, `findings`, `layers`, `registry_sha256`, `input_sha256`, `policy_sha256`, `state`, `autonomy`, `impact`, `authority`, `outcome`, `modifications`.
Схема	[`declared_inventory.schema.json`](../../schemas/declared_inventory.schema.json), [`drift.schema.json`](../../schemas/drift.schema.json), [`foundation_governance_policy.schema.json`](../../schemas/foundation_governance_policy.schema.json), [`foundation_governance_result.schema.json`](../../schemas/foundation_governance_result.schema.json).
Статус контракта	Нормативные схемы покрывают заявленный инвентарь, расхождения, слои политики и результат её применения. Единая универсальная схема `ObservedState` и отдельная материализованная схема `EffectivePolicy` пока отсутствуют.

Заявленное состояние выражает намерение из кода инфраструктуры, конфигурации и документации. Наблюдаемое состояние содержит факты среды исполнения и телеметрию. Действующая политика — то, что реально применяется после наложения всех уровней настроек, — образуется из слияния значений по умолчанию, настроек среды и роли, переопределений на уровне отдельного хоста и авторитетных источников проекта.

Расхождение состояния (drift) — классифицированное отклонение, а не простая построчная разница текстов (diff). Ситуации «заявлено, но отсутствует» (missing declared), «обнаружено, но не заявлено» (extra observed), «значения не совпадают» (mismatch), «источник устарел» (stale source), «осознанно принятое отклонение» (accepted variance) и «не управляется, но допускается» (unmanaged-but-tolerated) имеют разные последствия. Противоречие между источниками не исчезает из-за приоритета и фиксируется как доказательство.

Различение заявленного и наблюдаемого распространяется на метаданные агентных артефактов. Заявленные в манифесте навыка или инструмента разрешения, охват сетевых обращений и класс риска относятся к заявленному состоянию и сами по себе ничего не подтверждают. Поведение артефакта, наблюдаемое в изолированном прогоне, относится к наблюдаемому состоянию. Расхождение между ними классифицируется той же дисциплиной, что и расхождение в конфигурации инфраструктуры: заниженные заявленные разрешения (understated permissions) и подменённый класс риска (spoofed risk tier) являются находками, а не предупреждениями в интерфейсе.

> *Оценка возникает только при совместном рассмотрении намерения, наблюдаемых фактов и политики.*

16. Проектная эксплуатационная обвязка (Project Operations Harness)

Карточка типа	Контракт
Тип	`HarnessManifest` / документы компонентов / `HarnessResult`
Вход	Явно ограниченный корень проекта, манифест версионируемых локальных ссылок на компоненты и метаданные авторитетности источника.
Выход	Нормализованный результат проверки с находками, метаданными полноты и усечения, ограниченным учётом использованных ресурсов, свёрткой источника и `execution_authority: none`.
Ключевые поля	`project_ref`, `environment_refs`, `policy_refs`, `agent_refs`, `tool_contract_refs`, `knowledge_refs`, `runbook_refs`, `observability_refs`, `outcome`, `authority`, `completeness`, `findings`, `usage`, `truncation`, `execution_authority`.
Схема	[`harness_manifest.schema.json`](../../schemas/harness_manifest.schema.json), [`harness_project.schema.json`](../../schemas/harness_project.schema.json), [`harness_environment.schema.json`](../../schemas/harness_environment.schema.json), [`harness_policy.schema.json`](../../schemas/harness_policy.schema.json), [`harness_agents.schema.json`](../../schemas/harness_agents.schema.json), [`harness_tools.schema.json`](../../schemas/harness_tools.schema.json), [`harness_knowledge.schema.json`](../../schemas/harness_knowledge.schema.json), [`harness_runbook.schema.json`](../../schemas/harness_runbook.schema.json), [`harness_observability_ref.schema.json`](../../schemas/harness_observability_ref.schema.json), [`harness_source_authority.schema.json`](../../schemas/harness_source_authority.schema.json), [`harness_result.schema.json`](../../schemas/harness_result.schema.json).
Статус контракта	Нормативные схемы манифеста проектной обвязки, его компонентов, авторитетности источника и результата существуют. Для инвентаря агентных артефактов из §16.5 отдельной нормативной схемы пока нет.

Проектная эксплуатационная обвязка (Harness) — проверяемый по схемам и хранимый как код комплект правил и сведений о проекте. Он задаёт признанные источники данных, критичность, владельцев, среды, классификацию чувствительности данных, разрешённые окна изменений, цели качества сервиса (SLO), эксплуатационные инструкции, полномочия агентов и контракты входов и выходов инструментов.

Раздел проектной обвязки	Назначение
Проект и среды (`project / environments`)	Идентичность, критичность, владельцы и переопределения для конкретной среды.
Политики (`policies`)	Исходные уровни, серьёзность, расхождения, доступ, окна изменений и правила явного запрета.
Наблюдаемость (`observability`)	Цели качества сервиса (SLO), базовые сигналы здоровья, оповещения, панели визуализации и ссылки на логи и трассировки.
Агенты и инструменты (`agents / tools`)	Роли, запросы, разрешённые и запрещённые действия, обязательные условия и контракты входных и выходных данных.
Агентные артефакты (`agent artifacts`)	Инвентарь навыков, инструментов и MCP-серверов: идентичность, версия, свёртка, издатель, назначенная базовая модель и целевые среды исполнения (см. §16.5).
Инструкции и знания (`runbooks / knowledge`)	Диагностика, планы устранения, проверка результата, каталог сервисов и зависимости.
Решения и доказательства (`decisions / evidence`)	Ссылки на решения и инциденты, а также неизменяемые указатели на артефакты.

16.1. Иерархия доверия к источникам (trust hierarchy)

Работающая боевая среда и версионируемые в репозитории источники проекта имеют явную авторитетность. Текст обращения об инциденте, запросы к модели, внешние документы и ответы языковой модели недоверенны по умолчанию. Конфликт источников требует разбора человеком. Тело навыка агента и его метаданные относятся к тому же классу, что внешний документ: они недоверенны до проверки подписи и свёртки и остаются данными, а не инструкциями, после неё.

16.2. Машиночитаемые разрешения (machine-readable permissions)

Для роли машиночитаемо задаётся, что она может делать, что ей запрещено и какие условия обязательны (`can`, `cannot`, `requires`). Планировщик может читать доказательства и готовить предложение, но не вправе изменять инфраструктуру. Исполнитель получает доступ только после утверждения точного пакета изменения и требует ограниченных учётных данных, плана проверки результата и механизма аварийной остановки.

Каждый субъект, порождающий запись жизненного цикла, включая экземпляр агента, СЛЕДУЕТ снабжать аттестуемой идентичностью, привязанной к предоставившему полномочия субъекту (grantor) и к версии обвязки; разрешения субъекта не превышают разрешений предоставившего их субъекта. Идентичность МОЖЕТ реализовываться стандартами идентичности рабочих нагрузок (workload identity), например SPIFFE. Нормативная схема предоставления полномочий остаётся целевым состоянием (§17).

16.3. Проверка обвязки (Harness validation)

Валидатор проверяет схему и версию, обязательное наличие владельцев, явный перечень запретов, ссылки относительно корня пакета, битые ссылки, соответствие целей качества и оповещений, контракты инструментов и наличие авторитетности источника. Входные данные считаются потенциально враждебными: выход за пределы каталога через `../` (traversal), выход через символические ссылки (symlink escape), чрезмерно большие файлы YAML, якоря и алиасы YAML, позволяющие «раздуть» файл при разборе, дублирующиеся ключи и обращения к внешним ресурсам при разборе документа (network dereference) блокируются.

Соответствующий методологии валидатор проектной обвязки, работающий только на чтение, принимает явно заданный корень проекта и выдаёт один результат по схеме. Необязательные представления для инспекции и объяснения по селектору являются проекциями того же результата. Валидация не вызывает команд целевой системы, операций Git или сетевого доступа, а результат всегда содержит `execution_authority: none`.

16.4. Точки расширения (extension points)

Проектная обвязка резервирует стабильные идентификаторы проекта, среды, сервиса, владельца, политики, правила утверждения, инструмента, эксплуатационной инструкции и артефакта доказательства. Будущие сущности инцидентов и анализа корневых причин не добавляются пустыми; связь строится через ссылки с явно заданным типом и версионируемые контракты схем данных.

16.5. Инвентарь агентных артефактов (agent artifact inventory)

Навыки, инструменты, MCP-серверы, библиотеки запросов, модели, комплекты конфигурации среды исполнения или обработчиков и агенты, вызываемые как инструменты, явно перечисляются в проектной обвязке. Артефакт, отсутствующий в инвентаре, не является разрешённым к загрузке.

Поле	Назначение
`artifact_id` / `version`	Стабильная идентичность и точная версия; диапазоны версий и `latest` не допускаются.
`owner`	Ответственный владелец допуска, проверки и исключения из инвентаря.
`digest`	Свёртка полного содержимого комплекта, включая ресурсные файлы, а не только манифеста.

Поле	Назначение
`publisher` / `signer`	Отзываемая идентичность издателя и ключ проверки. Подпись подтверждает авторство, но не безопасность: проверенный издатель способен выпустить вредоносное содержимое.
`source`	Реестр или репозиторий происхождения.
`dependencies` / `external_refs`	Вложенные зависимости и разрешаемые во время исполнения ссылки, зафиксированные по неизменяемым свёрткам; неразрешимые ссылки и плавающие версии запрещены.
`installed_at` / `installed_by`	Когда и кем артефакт введён в контур.
`lifecycle_state` / `expires_at` / `revocation_ref` / `retention_rule`	Текущее состояние жизненного цикла, срок действия, доказательство отзыва и правило хранения артефакта и доказательств его проверки.
`declared_permissions`	Заявленные файловые, сетевые и инструментальные разрешения; заявленное состояние в смысле §15.
`last_check`	Ссылка на последнюю запись покрытия проверки и её исход (см. ниже).
`backbone_model`	Модель, фактически исполняющая артефакт на данном узле.
`target_runtimes`	Среды исполнения, в которых артефакт разрешён к использованию.

Проверенное происхождение или действительная подпись подтверждает источник и целостность, но не разрешает исполнять содержимое или считать его инструкциями. Для допуска инструкций применимая политика должна разрешать точную свёртку содержимого инструкций, заявленную роль и допустимую область действий. Изменение содержимого или роли требует нового допуска. Недоверенное содержимое не может само наделять себя полномочиями.

Смена `backbone_model` на узле, принимающем действия, является управляемым изменением и требует повторной аттестации, а не считается заменой равнозначного компонента. Стойкость к инъекции является свойством исполняющей модели, а не байтов артефакта, и не переходит вместе с подписью.

Инвентарь определяет только обязательную семантику записей. Он не предоставляет полномочий на исполнение, а соответствие доказывается только связанным с точной целью доказательством проверки, удовлетворяющим применимому контракту.

Для ограниченного профиля C4 инвентарь также фиксирует точные версии и идентичности конфигураций каждого компонента доверенной вычислительной базы (trusted computing base, TCB), названного в Приложении G: канонизатора, центра идентичности и политик, подписанта и хранилища ключей, авторитетного журнала, хранилища резервирований, шлюза или взаимодействующего адаптера backend (серверной стороны исполнения), наблюдателя эффектов, оракула Outcome (результата), а также источников времени и версий. Общие учётные данные, модели, источники доказательств, канонизаторы, часы, сервисы control plane (плоскости управления) и пути наблюдения объявляются зависимостями общей причины отказа, а не считаются независимыми контролями.

Проверка внесённого в инвентарь артефакта обязана публиковать машиночитаемую запись покрытия. В ней перечисляются каждый обнаруженный файл и его свёртка, исход анализатора для каждого файла, неподдерживаемые или пропущенные форматы, обработка архивов и внешних ссылок, а также каждый обрыв: таймаут, усечение, отказ разборщика или исчерпание лимита ресурсов. Допустимы исходы `pass`, `fail` и `incomplete`: нулевое число находок означает `pass` только при успешном завершении всего покрытия, требуемого выбранным профилем, а любой обрыв даёт `incomplete`. Повторная проверка проводится после изменения версии, модели, среды исполнения, зависимости или разрешений, до истечения срока действия и с периодичностью, заданной политикой. Сокращение охвата ради стоимости записывается как явное исключение, а не молча считается успешной проверкой.

16.6. Контрольные ворота допуска интеграции

Интеграция допускается для точного коннектора, его версии, среды и класса операций, а не для названия продукта в целом. При допуске проверяются аутентифицированная область полномочий, доказательство режима только для чтения, если он заявлен, семантика ошибок, полнота и свежесть доказательств, контрактные тесты, покрытие наблюдаемостью и оценками поведения, ограничения частоты вызовов, срок и условия отзыва. Другая версия, среда, класс операций или расширенный охват требуют нового решения.

Карточка типа	Контракт
Тип	`IntegrationAdmissionRecord`
Вход	Точные артефакт и свёртка коннектора, целевая среда, класс операций, запрашиваемая область полномочий, доказательства контрактных тестов и оценок, а также применимая политика.
Выход	`allow`, `deny`, `modify` или `escalate` с установленными границами допуска, сроком, условиями отзыва и ссылками на доказательства.
Ключевые поля	`integration_ref`, `connector_ref`, `version`, `digest`, `environment_ref`, `operation_class`, `authorization_scope`, `read_only_proof_ref`, `failure_semantics_ref`, `completeness_ref`, `freshness_ref`, `contract_test_refs`, `observability_refs`, `eval_refs`, `rate_limits`, `expires_at`, `revocation_ref`, `outcome`.
Схема	Исходы политики задаются схемами управления из §15; отдельной нормативной схемы `IntegrationAdmissionRecord` пока нет.
Статус контракта	Обязательная семантика задана здесь; до появления отдельной схемы машиночитаемый допуск является точкой расширения.

Допуск предоставляет только записанную техническую возможность интеграции. Он не предоставляет полномочий жизненного цикла, не заменяет решения человека и не выдаёт учётных данных исполнителя.

17. Полномочия и решения человека (Human Authority and Approvals)

Карточка типа	Контракт
Тип	`DecisionMap` / `HumanDecision`
Вход	Операционное намерение, идентичность перехода, субъект решения, роль субъекта, границы полномочий, блокируемое действие, обязательные доказательства и оценки, а также срок действия решения.
Выход	Явное решение `approve` либо `deny`, связанное с точным переходом, охватом, аттестацией издателя, временем выдачи, сроком действия и необязательным отзывом.
Ключевые поля	`intent_ref`, `points`, `transition_id`, `principal_ref`, `issuer_attestation_ref`, `actor_role`, `authority_scope`, `blocked_action`, `decision`, `issued_at`, `not_before`, `expires_at`, `revocation_ref`.
Схема	[`foundation_decision_map.schema.json`](../../schemas/foundation_decision_map.schema.json), [`foundation_human_decision.schema.json`](../../schemas/foundation_human_decision.schema.json).
Статус контракта	Нормативные схемы карты решений и решения человека существуют. Полная цепочка происхождения полномочий, доказывающая, что субъект обладает заявленными полномочиями, представлена лишь частично; отдельной нормативной схемы предоставления полномочий пока нет (см. §16.2).

Плоскость управления определяет не форму записи утверждения, а источник полномочия, обязательные точки участия человека и пределы решения. `HumanDecision` из §10 является доказательством такого решения, но не создаёт полномочие своему издателю.

17.1. Карта решений с участием человека (HITL Decision Map)

Участие человека в контуре (human in the loop, HITL) — схема, при которой определённые переходы невозможны без явного решения человека. Для каждого перехода указываются решения человека, допустимые действия агента, основания передачи решения на более высокий уровень и запрет самоутверждения — одобрения агентом собственного результата. Неформальный комментарий, заявка в системе учёта или запрос к модели не считаются решением об утверждении.

Роль или средство контроля	Что создаёт или предоставляет	Чего не может делать самостоятельно
Планировщик (Planner)	Предложение плана изменений `ChangePlan`	Утверждать или исполнять своё предложение
Проверка политики (Policy Hook) / сквозной слой управляющих правил (Governance Mesh)	Решение о переходе на основе правил	Обеспечивать персональную ответственность человека или исполнять изменение
Уполномоченный человек	Ограниченное решение `HumanDecision`	Расширять неизменённый план через запись утверждения
Независимый контролёр (Guardian) / детерминированные контрольные ворота	Доказательства независимой проверки	Утверждать собственную работу или предоставлять полномочия на исполнение
Исполнитель через контрольные ворота (Gated Executor)	Запись исполнения `ExecutionRecord` для утверждённого пакета	Выводить полномочия из ответа агента или изменять план

17.2. Границы человеческого решения

Для каждой точки указываются ответственный субъект решения, основание его полномочий, охват, запрещённое без решения действие, обязательные доказательства и оценки, срок действия, порядок отзыва и владелец эскалации. Изменение плана, цели, объекта воздействия, политики, уровня воздействия или доказательств после утверждения требует повторного решения.

Усталость от утверждений (approval fatigue) является риском управления: интерфейс должен показывать цель, воздействие, альтернативы, неизвестные факты, откат или компенсацию и способ проверки результата, а массовое утверждение не должно скрывать различия между действиями.

18. Сквозной слой управляющих правил (Governance Mesh), автономность и операционное воздействие



Плоскость управления и политик: правила проекта, полномочия, ограничения, проверка перехода, решение человека и запись допуска и исхода.

Текстовая альтернатива: правила и проектная обвязка задают границы; полномочия назначают ответственных и точки решения человека; ограничения разделяют автономность, воздействие и технические способности; потенциально опасный переход проверяется до действия; человек утверждает или отклоняет конкретный план; подписанные записи связывают допуск и исход без перезаписи трассы.

Карточка типа	Контракт
Тип	`GovernancePolicy` / `GovernanceResult` / `AdmissionReceipt` / `OutcomeReceipt`
Вход	Попытка перехода жизненного цикла, слои политики и идентичность реестра, точное действие и его охват, актуальные доказательства, контекст проектной обвязки и обязательное решение человека, если оно требуется.
Выход	Закрытый по умолчанию исход применения политики с классификацией автономии R0-R5 и воздействия I0-I5, ограничениями и эскалацией; для попытки исполнения — связанные подписанные квитанции допуска и исхода.
Ключевые поля	`layers`, `input_sha256`, `policy_sha256`, `applied_rules`, `state`, `autonomy`, `impact`, `authority`, `outcome`, `modifications`, `attempt_id`, `action_ref`, `parent_action_ref`, `terminal_state`, подпись и ссылка на ключ.
Схема	[`foundation_governance_policy.schema.json`](../../schemas/foundation_governance_policy.schema.json), [`foundation_governance_result.schema.json`](../../schemas/foundation_governance_result.schema.json); техническая возможность C0-C5 представлена в [`harness_project.schema.json`](../../schemas/harness_project.schema.json) и [`harness_agents.schema.json`](../../schemas/harness_agents.schema.json).
Статус контракта	Нормативные схемы политики и результата её применения существуют, но сейчас результат использует один общий классификатор для `autonomy` и `impact` и не требует наличия уровня, поэтому схема не полностью обеспечивает разделение значений R и I. Схемы проектной обвязки обеспечивают C0-C5. Самостоятельные схемы JSON для подписанных `AdmissionReceipt` и `OutcomeReceipt` пока отсутствуют.

Сквозной слой управляющих правил (Governance Mesh) проверяет переходы «Намерение -> Доказательства -> Диагностика -> План -> Утверждение -> Контролируемое изменение -> Проверка ->

Извлечение уроков» (Intent -> Evidence -> Diagnosis -> Plan -> Approval -> Controlled Change -> Verification -> Learning). Он действует до потенциально опасного шага и не зависит от доброй воли агента. Обязательная трасса процесса из §14.1 фиксирует каждую проверку и попытку перехода.

В Agent-Ops R0-R5 обозначает риск-адаптивную лестницу разрешений и автономии. Определения всех уровней приведены в [§5.5](#55-автономность-r0-r5). Шкала R0-R5 заимствована из таблицы §4.3 без изменения семантики, чтобы не плодить несовместимые нотации; первоисточник — AI-Disrupt PDLC 2.0. Операционное воздействие использует отдельную ось I0-I5, а техническая возможность — C0-C5. Ни одна из осей сама по себе не предоставляет полномочий на исполнение.

Автономия	Что разрешено	Типичная граница
R0	Только чтение.	Боевая среда без подтверждённых оценок поведения.
R1	Только предложения; решение принимает человек.	Критичные для боевой среды маршруты.
R2	Действия в отдельной ветке кода для конкретной задачи (feature branch) с разбором запроса на слияние ветки (merge request review).	Стандартная разработка.
R3	Автоматическое слияние ветки (auto-merge) только при успешных оценках поведения.	Стандартизированные шаблоны и библиотека проверенных шаблонов решения задач (Pattern Library).
R4	Многосессионная работа с контрольными точками для возобновления.	Долгие зрелые и проверенные сценарии.
R5	Полная автономия только в изолированной песочнице (sandbox).	Изолированные экспериментальные среды.

- Воздействие I0-I5 учитывает критичность, тип действия, радиус поражения (blast radius) — широту распространения последствий, обратимость и класс чувствительности данных.
- Точка применения политики возвращает нормализованное перечисление Agent-Ops с фиксированными значениями: разрешить (`allow`), запретить (`deny`), разрешить с изменением условий (`modify`) или передать решение выше (`escalate`). Она содержит владельца, версию, свёртку и объяснимые факторы; `deny` сильнее `allow`, конкурирующие изменения условий разрешаются только по явно заданному приоритету.
- Эффективный уровень R зависит от среды, ветки кода, воздействия, покрытия автоматическими оценками, пакета доказательств и горизонта задачи (task horizon) — её длительности и числа шагов.
- Неизвестное воздействие, конфликт политик или устаревшие доказательства приводят к `deny` либо эскалации.
- Агенту запрещено повышать автономию через запрос к модели, текст внутри обвязки или самоутверждение.

18.1. Квитанции допуска и исхода (admission and outcome receipts)

Подписанная квитанция позволяет независимо проверить целостность зафиксированного содержимого; полнота и истинность истории оцениваются отдельно. Поэтому решение точки применения политики (Policy Hook) и, если исполнение произошло, его исход фиксируются двумя связанными подписанными записями.

Квитанция допуска (admission receipt) формируется до потенциально опасного шага и содержит `attempt_id` (общий идентификатор попытки), `actor_id`, `action_type`, `scope` (границу затрагиваемых ресурсов), `policy_version`, `decision` (`allow`, `deny`, `modify` или `escalate`), `timestamp_ms` и подпись по каноническому набору полей с указанием алгоритма и ссылки на ключ.

Квитанция исхода (outcome receipt) формируется после исполнения и содержит тот же `attempt_id`, `action_ref` — ключ связывания с явно указанным набором исходных полей, из которых он вычисляется,

— `terminal_state` (`committed` или `failed`) и подпись. Рецензент должен знать точные исходные значения, использованные для вычисления ключа.

- `policy_version` связывается в момент принятия решения. Изменение политики между допуском и исполнением создаёт разрыв аудита, если версия не зафиксирована вместе с решением.
- `deny` без исполнения порождает подписанную квитанцию допуска наравне с `allow`. Отказ, не оставивший следа, неотличим от потери телеметрии.
- Отсутствие квитанции исхода означает, что действие было заблокировано, только при исправном, полном и защищённом от подделки конвейере квитанций. В остальных случаях оно означает сбой, обрыв очереди или вмешательство.
- `attempt_id` обязателен в обеих записях: без него нельзя подтвердить, что допущенное действие и исполненное действие совпадают.
- `parent_action_ref` связывает вызовы в цепочку: корневое действие несёт пустое значение, дочернее — `action_ref` предшественника, запись отказа наследует того же родителя. Реализация, не поддерживающая соединение нескольких предшественников, обязана объявлять это ограничение явно, а не создавать видимость полной реконструкции причинно-следственной цепи.

Квитанции определяют формы записей для исполнителя за контрольными воротами и журнала решений. Квитанция не предоставляет полномочий на исполнение и доказывает только решение или исход, связанный с её подписанным содержимым.

18.2. Технические возможности C0-C5 и граница исполнения

Техническая возможность (capability) описывает, что компонент способен подготовить или выполнить, но сама по себе не предоставляет полномочий. Полные определения уровней приведены в [§5.3](#53-техническая-возможность-c0-c5).

Техническая возможность C0-C5	**Разрешённая техническая возможность**
C0 Наблюдать (Observe)	Только факты и доказательства.
C1 Объяснять (Explain)	Причины, вопросы и пробелы в доказательствах.
C2 Планировать (Plan)	Оценка воздействия и план без изменения.
C3 Готовить изменение (Prepare Change)	Запрос на слияние, план пробного прогона, план проверки и пакет плана отката либо компенсирующего действия; никаких действий с целевой системой.
C4 Применять после одобрения человеком (Human-approved Apply)	Отдельный исполнитель после явного решения об утверждении.
C5 Ограниченное самовосстановление (Bounded Self-Healing)	Будущая техническая возможность для заданных политикой сценариев малого воздействия, у которых заранее одобрены точные класс действий, охват, предусловия, правила остановки и условия отзыва; в этой редакции выключена по умолчанию.

Текущая граница этой редакции требует, чтобы каждое реальное изменение целевой системы запускал человек через детерминированного исполнителя с контрольными воротами. C5 описывает будущий режим, а не действующие постоянные полномочия. В таком будущем режиме предварительное одобрение распространяется только на точную границу сценария; изменение цели, плана, цели воздействия, уровня воздействия, политики или состояния доказательств по-прежнему требует нового решения человека.

19. Плоскость независимой проверки и доверия: независимый контролёр (Guardian)



Плоскость независимой проверки и доверия: доказательства, правила и полномочия, намерение и результат, безопасность, воспроизводимость и работоспособность контроля.

Текстовая альтернатива: независимая проверяющая сторона сверяет происхождение, полноту и свежесть доказательств; версии политик, охват и полномочия; связь плана и результата с критериями намерения; соблюдение границ безопасности; воспроизводимость; а также работоспособность самого контроля. Недостаток доказательств блокирует продвижение или вызывает эскалацию.

Третья плоскость не создаёт данные жизненного цикла и не принимает управленческие решения за владельца. Она независимо подтверждает соответствие артефактов, переходов и результата их контрактам и блокирует продвижение при недостаточном доказательстве.

19.1. Независимый контролёр/оценщик (Independent Guardian/evaluator)

Независимый контролёр (Guardian) не имеет полномочий на исполнение. Он проверяет соответствие намерению, проектной обвязке и политике; полноту пакета доказательств; высказывания без опоры на доказательства; попытку вывести одобрение изменения из контекста, где его не было; а также готовность к проверке результата и откату либо компенсации. Критичный результат не может одобрять тот же субъект, который его сформировал.

Для ограниченного профиля C4 независимость контролёра обеспечивается разделением полномочий, учётных данных, источников доказательств и путей наблюдения. Вторая LLM (большая языковая модель), другой prompt (запрос к модели) или второй процесс с теми же полномочиями и путём доказательств не дают достаточной независимости. Согласие нескольких агентов или оценка мета-агентом остаются производным доказательством: они не образуют независимой проверки и не создают полномочий на исполнение. Контролёр не может создавать утверждение, хранить учётные данные записи в цель или исправлять оцениваемую запись незарегистрированным действием.

19.2. Программная проверка (library gate) или ИИ-агент

Детерминированные инварианты реализуются обычными программными проверками с однозначным результатом (validation gates). Отдельный ИИ-агент используется только для проверки смысловой непротиворечивости (semantic consistency) и не заменяет проверки схем и политик.

Независимый контролёр проверяет обе другие плоскости: в плоскости данных — схему, происхождение, полноту, свежесть, трассу и соответствие намерению; в плоскости управления — версию политики, полномочия, охват, разделение обязанностей и отсутствие самоутверждения. Независимый контролёр

не получает учётные данные исполнителя и не может исправить проверяемый результат незарегистрированным действием.

Детерминированная проверка и смысловой разбор с участием модели МОГУТ использовать общие входные данные, но каждые общие модель, источник доказательств, канонизатор, часы, учётные данные, сервис политик или зависимость control plane (плоскости управления) образуют одну границу общей причины отказа. Заявление о независимом обнаружении ограничивается компонентами и путями, сохранившими независимость после перечисления этих зависимостей.

19.3. Наблюдаемость среды исполнения агента и внешний контроллер состояния

Среда исполнения агента публикует задержку расследования, возраст очереди, ошибки инструментов и коннекторов, свежесть коннекторов, доступность точки применения политики (Policy Hook) и независимого контролёра (Guardian), доли пересмотра решений и неподтверждённых высказываний, стоимость на один проверенный результат (Outcome), расхождения теневой оценки и число остановок сессий агента по классу причины. Эти сигналы разделяются по среде и классу действий; запросы, секреты и уникальные идентификаторы случаев не включаются в метки показателей.

Пересмотр решения (override) — зафиксированное действие человека, которое изменяет, заменяет или отменяет первоначальное решение. Эти три формы учитываются отдельно там, где различие существенно. Такой пересмотр не разрешает обходить обязательный запрет политики и сам по себе не создаёт полномочий на исполнение.

Внешний контроллер состояния в отдельном домене отказа оценивает работоспособность среды исполнения агента и управляющих компонентов. У него нет полномочий на исполнение или учётных данных исполнителя. Неработоспособное или `unknown` состояние точки применения политики, независимого контролёра, конвейера квитанций, коннектора или самого контроллера состояния приводит к безопасному отказу либо эскалации согласно политике; отсутствие сигнала не считается признаком исправности. Наблюдения и решения о состоянии среды добавляются в трассу процесса и связываются с затронутым случаем.

20. Безопасность, доверие и жизненный цикл контекста агента

Раздел определяет, как защищается пакет контекста агента, какие утверждения о доверии к нему допустимы и как пакет проходит полный жизненный цикл. Он применяется к эксплуатационной работе по Agent-Ops; тестирование программного обеспечения, реализующего методологию, не является его предметом.

20.1. Безопасность

Пакет контекста сам по себе никогда не предоставляет полномочий. Контроли безопасности применяются к точным роли, цели, среде, версии пакета и предпринимаемому действию, а незавершённый контроль не считается доказательством безопасности.

Угроза	Как применяется контроль	Исход нарушения
Внедрение вредоносных инструкций в запрос или контекст	Статус инструкции получают только допускаемые политикой для точных свёртки и роли артефакты, предназначенные для передачи инструкций. Текст обращения, внешние документы, результат поиска и вывод другого агента поступают как данные; граница между инструкциями и данными восстанавливается при каждой передаче. До попадания материала в модель детерминированный сборщик применяет актуальный `ContextAssemblyProfile` (профиль сборки контекста): выполняет заданную политикой санитарную обработку, нормализует или отклоняет запрещённые управляющие и двунаправленные символы, имитацию ограждений и элементы сверх лимита, а также помещает каждый недоверенный элемент в типизированный структурный раздел с метками происхождения и класса доверия. Один только текстовый разделитель границу не устанавливает. Инструкция в системном запросе может усиливать этот контроль, но не заменяет его.	Если границу или происхождение установить нельзя, материал остаётся недоверенным и не может разрешить действие или считаться утверждением.

Угроза	Как применяется контроль	Исход нарушения
Утечка секретов или персональных данных	Профиль сборки классифицирует чувствительные поля, исключает или скрывает значения, при необходимом доступе передаёт не раскрывающие значение ссылки и применяет те же правила к фрагментам, запросам, журналам и результатам. Учётные данные никогда не включаются в пакет контекста.	Необходимые, но недоступные данные дают `incomplete`; раскрытие нескрытого значения или доступ вне охвата дают `fail` и запускают обработку инцидента.
Подмена цели, пути или входных данных	Проект, среда, сервис, ресурс и источник связываются с каноническими идентификаторами и свёртками. Механизмы разрешения ссылок отклоняют выход за допустимый корень, уход через символическую ссылку, неоднозначные псевдонимы и сетевые ссылки без зафиксированной версии.	Несовпадение идентичности или неразрешимая ссылка блокируют использование пакета и требуют исправленного либо заново собранного контекста.
Устаревшие, повторно предъявленные или взятые из другого проекта доказательства	Каждый элемент содержит окно наблюдения, правило свежести, привязку к точной цели и неизменяемую ссылку на источник. Истёкшее, отозванное, повторно предъявленное или иначе привязанное доказательство собирается заново, а не используется молча.	Зависящие выводы получают `unknown`; переход запрещается либо эскалируется до появления актуального доказательства.
Неразрешённое изменение или использование учётных данных	Субъекты, ограниченные уровнями C0-C3, работают без учётных данных, позволяющих изменять цель. Отдельный наблюдатель фиксирует фактические воздействия вызовов инструментов, файловых и сетевых операций, использования учётных данных и операций с ресурсами целевой системы. Только исполнитель через контрольные ворота (Executor) может получить ограниченные проектом, действием и временем учётные данные после требуемых решения политики и решения человека; каждое использование связывается с квитанциями допуска и исхода.	Любое неутверждённое воздействие, использование учётных данных вне охвата или неполное наблюдение дают `fail`, отзыв доступа и эскалацию; отсутствие телеметрии не доказывает отсутствие изменений.
Отравление цепочки поставки контекста и среды исполнения	Навыки, инструменты, MCP-серверы, библиотеки запросов, модели, конфигурация среды исполнения и способные влиять на исполнение обработчики должны присутствовать в инвентаре §16.5 с точными версией, свёрткой всего комплекта, идентичностью издателя, заявленными разрешениями, сроком действия и состоянием отзыва. Вложенные зависимости фиксируются по свёрткам; внешняя ссылка, разрешаемая только во время исполнения, запрещается. Перенос в другую среду исполнения требует новой проверки разрешений и риска.	Отсутствующая, изменённая, просроченная, отозванная или непроверяемая зависимость не загружается; невозможность выразить исходное ограничение в целевой среде даёт `deny`.

Угроза	Как применяется контроль	Исход нарушения
Дрейф цели или поведения агента (misalignment)	При старте сессии свёртки намерения, проектной обвязки, разрешений роли и `ContextAssemblyProfile` фиксируются в трассе процесса как опорное состояние сессии (session baseline). Любой вывод, не являющийся типизированной записью с опорой на доказательства, записывается как `unknown`. Субъекты, ограниченные уровнями C0-C3, не имеют пути к эффекту независимо от содержания вывода; поведенческий мониторинг (§19.3) дополняет эти границы, но не заменяет их. Остановка или изоляция сессии агента — управляемое событие: оно дописывается в трассу, сохраняет контрольную точку и контекст для разбора и блокирует использование уже выданных записей `HumanDecision` до нового решения, не отзывая их.	Расхождение опорного состояния с фактическим контекстом сессии блокирует продвижение; сессия останавливается с сохранением артефактов; отсутствие опорного состояния даёт `incomplete`.
Подмена манифеста, политики или контрольной точки	Прогон связывает версионируемые манифесты, политики, реестры и происхождение контрольных точек свёртками и записывает изменения в дополняемую трассу процесса. До проверки идентичности и применимых актуальных версий запуск безопасно блокируется.	Разрыв происхождения, несовпадение свёртки или непроверяемая актуальная политика блокируют возобновление и исполнение.

Таким образом, запрет изменений является инвариантом исполнения для точного случая Agent-Ops, а не только тестом исходного кода. Он доказывается отсутствием у наблюдающей роли полномочий на запись, независимо записанной трассой воздействий и сопоставлением связанного состояния цели до и после прогона. Неполная трасса даёт `incomplete`, но никогда `pass`.

20.2. Модель доверия

Доверие — не единая оценка, присваиваемая агенту или тексту. Это набор отдельно доказанных свойств применительно к конкретным источнику, утверждению, охвату, версии и моменту времени. «Авторитетный» (authoritative) означает, что источник признан для конкретного утверждения в этих границах, но не означает его безошибочности.

Свойство доверия	Как устанавливается	Чего не доказывает
Полномочия источника	Проектная обвязка и политика определяют источник, утверждения, для которых он авторитетен, охват и срок действия.	Корректность вне указанного охвата или временного окна.
Происхождение и целостность	Неизменяемая ссылка на источник, свёртка всего содержимого, идентичность подписанта, когда она применима, и происхождение преобразований связывают проверенные байты с их источником.	Истинность, безопасность или полномочия содержимого.
Свежесть	Время получения, окно наблюдения, срок актуальности, идентичность текущей цели и состояние отзыва проверяются на переходе, использующем элемент.	Полноту собранных доказательств.
Полнота	Обязательные классы источников, границы сбора, включённые и исключённые элементы, усечение и состояние обрывов записываются и сверяются с профилем.	Корректность каждого включённого утверждения.
Производный результат	Краткая выжимка, результат поиска, диагноз или вывод агента записывает источники и преобразование и остаётся в разделе `derived` (производные данные).	Полномочия источника, статус инструкции, утверждение или полномочия на исполнение.

Свойство доверия	Как устанавливается	Чего не доказывает
Независимая проверка	Независимый контролёр либо детерминированные контрольные ворота повторно проверяют обязательные свойства по точным входным данным, не наследуя полномочия автора.	Полномочия на исполнение или на незарегистрированное исправление результата автора.

Каждый элемент контекста остаётся в явно указанном разделе `authoritative` (авторитетные), `derived` (производные) либо `untrusted` (недоверенные данные). Уплотнение, копирование, поиск и передача не могут повысить этот уровень. Если обязательное свойство доверия отсутствует или противоречиво, вывод, использующий этот элемент, остаётся `unknown` либо `incomplete`, а управляемый переход безопасно блокируется или эскалируется.

Граница доверия ограниченного профиля C4 приведена в Приложении G. Безопасность допуска, знание об эффекте, успешность Outcome (результата) и последующее восстановление являются разными заявлениями. Безопасная остановка не устанавливает остальные три свойства, а атомарная удалённая авторизация не заявляется без взаимодействующего шлюза или backend (серверной стороны исполнения), применяющего решение в объявленной точке допуска.

20.3. Жизненный цикл контекста агента

Пакет контекста является версионированным и ограниченным представлением намерения, политики, ссылок на полномочия, доказательств и оставшейся работы для одной роли. Он содержит `execution_authority: none` и не может переносить скрытые полномочия между агентами.

Контракт	Требование
Назначение и охват	Зачем существует пакет; точные случай, роль, цель, среда и включённые объекты.
Разделы по уровню доверия	Авторитетные, производные и недоверенные элементы не смешиваются.
Граница полномочий	Ссылки на актуальные решения, разрешённые инструменты, запрещённые операции, обязательные утверждения и `execution_authority: none`; пакет записывает полномочия, но не предоставляет их.
Свежесть и идентичность	Окна наблюдения, происхождение, версии, неизменяемые ссылки на источники и свёртки содержимого.
Ограничения	Пределы по байтам, элементам, токенам, длине фрагментов, времени, попыткам и обращению с секретами.
История жизненного цикла	Кто и когда собрал, проверил, уплотнил, передал, обновил, отозвал, закрыл и утилизировал пакет.

Источники контекста допускаются профилем сборки, иерархией доверия §16.1 и применимой политикой. Зависимости среды исполнения поступают только через инвентарь §16.5, а интеграции — также через контрольные ворота допуска §16.6. Манифест записывает точные идентичности и версии источников и зависимостей; знакомого названия продукта или агента недостаточно.

Карточка типа	Контракт
Тип	`ContextAssemblyProfile` / `ContextManifest`
Вход	Намерение и роль, границы полномочий и политики, кандидатные источники, временное окно, ссылки на эксплуатационные инструкции, правила ранжирования и бюджеты в байтах, элементах и токенах.

Карточка типа	Контракт
Выход	Точные включённые элементы и фрагменты; исключённые элементы и причины; происхождение, свежесть, раздел по уровню доверия, свёртка содержимого, состояние жизненного цикла и `execution_authority: none`.
Ключевые поля	`profile_ref`, `intent_ref`, `role_ref`, `target_ref`, `environment_ref`, `time_window`, `source_refs`, `runbook_refs`, `ranking_rules`, `byte_budget`, `item_budget`, `token_budget`, `included_items`, `excluded_items`, `exclusion_reasons`, `provenance`, `freshness`, `trust_partition`, `manifest_digest`, `state`, `parent_manifest_ref`, `expires_at`, `revocation_ref`, `closed_at`, `disposition`, `execution_authority`.
Схема	[`ai_agent_context.schema.json`](../../schemas/ai_agent_context.schema.json) описывает один контракт результата Gitinsky Audit, но не определяет универсальные `ContextAssemblyProfile` и `ContextManifest`. Отдельных нормативных схем пока нет.
Статус контракта	Приведённая ниже семантика жизненного цикла является нормативным целевым состоянием; покрытие машиночитаемыми схемами частично.

Этап жизненного цикла	Как применяется	Требуемый результат
Сборка	Машиночитаемый профиль отбирает источники в границах роли, политики, полномочий, временного окна и бюджетов. Он формирует манифест с каждым включённым фрагментом, каждым исключением и его причиной, происхождением, свежестью, разделом по уровню доверия и свёрткой.	Рецензируемый пакет с полной историей отбора и без полномочий на исполнение.
Проверка и использование	Перед использованием получатель проверяет свёртку манифеста, идентичность пакета, свежесть, обязательные источники, актуальную политику, роль и оставшиеся ограничения. Элементы предоставляются согласно их разделу по уровню доверия; внешний материал остаётся данными.	Текущий шаг может опираться только на точный проверенный пакет; неуспешная или неполная проверка блокирует его использование.
Уплотнение	Уплотнение создаёт версию-преемник и сохраняет ссылки на источники и их версии, правила отбора, категории исключений, ограничения полномочий, бюджеты, оставшуюся работу и ссылку на предшественника. Краткая выжимка остаётся производной.	Уменьшенный пакет с явными пропусками и происхождением, полномочия которого не выросли.
Контрольная точка, передача и возобновление	Контрольная точка сохраняет монотонно возрастающие, не переиспользуемые идентификаторы сессии и контрольной точки, инструкции по восстановлению окружения, статусы «сделано», «в работе» и «заблокировано», машиночитаемый остаток работы, пределы токенов, времени и попыток, историю и ссылку на текущий пакет доказательств. Перед воспроизводимым восстановлением получатель заново проверяет целостность, свежесть, актуальную политику, ссылки на полномочия, бюджеты, а также идемпотентность вызовов инструментов либо состояние компенсации.	Работа продолжается из аутентифицированного состояния-преемника; передача не расширяет полномочия и не сбрасывает ограничения.

Контрольная точка является производной сводкой, а не источником полномочий C4. Она может только уточнять авторитетные долговечные состояния журнала и резервирований: она не может удалять израсходованную попытку, допущенный или неразрешённый эффект, ожидающую сверку или компенсацию, текущую эпоху утверждения или политики, резервирование, fence (маркер отсечения) либо ограничение полноты наблюдения. Отношение уточнения контрольной точкой (`C4-R12`) фиксирует это правило и сохраняет перечисленные в Приложении G контрпримеры со снятым допущением и совместный контрпример пропуска.

Обновление и отзыв	**Изменение источника, политики, утверждения, модели, инструмента, коннектора или истечение срока действия делает затронутый пакет недействительным. Новый манифест собирается и связывается как преемник; предыдущая версия остаётся неизменяемой и не может молча снова стать актуальной.**	**Получатель различает актуальный, просроченный, заменённый и отозванный контекст и отказывается от недействительных версий.**
Закрытие, хранение и утилизация	При закрытии записываются конечное состояние, ссылки на результат и пакет доказательств, сохраняемые манифест и история передач, правило хранения и принятое решение об утилизации. Временные расшифрованные значения, выданные учётные данные и чувствительные кэши отзываются или уничтожаются без удаления аудиторской трассы.	Случай можно проверить впоследствии, но после него не остаются повторно используемые полномочия или ненужный чувствительный контекст.

Среде исполнения рассуждающего компонента СЛЕДУЕТ быть сессионной: без учётных данных записи, без сохраняемого между случаями состояния вне контрольных точек этого раздела и без сетевого доступа сверх объявленного профилем. Отсутствие пути к эффекту не отменяет этого требования: сохраняемое состояние может переносить отравленный контекст между случаями.

20.4. Продолжение работы человеком при потере ответа (информативный пример)

Информативный пример — потерянный ответ. Допущенная операция завершается тайм-аутом до получения результата инженером. При передаче сохраняются точные случай, пакет, попытка и ключ backend, ссылки на текущие полномочия, израсходованный бюджет, наблюдения и неразрешённые обязательства. Отсутствие ответа не доказывает отсутствие эффекта. До рассмотрения повтора или отдельно разрешённой сверки получатель устанавливает, что известно, по авторитетному журналу и независимым актуальным наблюдениям.

Продолжение под руководством человека сохраняет те же требования к полномочиям, записи действий и проверке результата. Оно может включать координацию, разрешённое наблюдение, подготовку и запуск человеком разрешённой детерминированной процедуры при доступности всех необходимых контролей. Если требуемые журнал, служба полномочий, контрольные ворота или наблюдатель недоступны, сохранённая сводка не восстанавливает эти средства и не доказывает восстановление. Прямая ручная мутация не вводится как альтернативный путь исполнения. При остановке агентной сессии также сохраняются ограничения на использование решений из раздела 20.1.

21. Проверка качества, воспроизводимость и экономика применения ИИ (AI economics)

Приведённые ниже процедуры задают способы оценки качества, воспроизводимости и стоимости. Их включение не доказывает, что реализация Agent-Ops прошла эти проверки; данная редакция не содержит контролируемых сравнительных результатов исполнения.

Для `agent-ops.c4-human-approved-apply@1.0.0` профиль публикует допущения и границы заявлений, а настоящая исходная ревизия включает ограниченную исполняемую спецификацию, шесть схем записей C4, публичный каталог валидатора отношений и негативные трассы отношений. Принятие этих автономных артефактов не является соответствием работающей среды. Успешная ограниченная проверка формальной модели состояний и переходов или межзаписных фикстур не подменяет безопасность полномочий, измерение фактического эффекта, успешность Outcome (результата), полноту наблюдения или доказательства боевой среды.

Для `agent-ops.evidence-acquisition@1.0.0` качество доказательств не является одной оценкой. Полномочия источника, идентичность и связывание, целостность и происхождение, действительность сбора, охват и полнота, актуальность, точность преобразований, независимость, поддержка утверждения и воспроизводимость оцениваются отдельно, как задано в Приложении H. Отсутствующая, противоречивая или непроверенная обязательная ось остаётся `unknown` или неполной; агрегированная

оценка не компенсирует сбой полномочий источника, связывания цели, целостности или блокирующего охвата.

В этом разделе оценивается эксплуатационная работа, выполняемая по Agent-Ops. Раздел не предписывает, как разрабатывать или тестировать программное обеспечение, реализующее методологию.

21.1. Проверка качества

Качество оценивается для каждого ограниченного эксплуатационного случая на всём пути от намерения до проверенного результата. Связный ответ, завершившийся вызов инструмента или отсутствие зарегистрированных инцидентов сами по себе не доказывают качество.

Оценка поведения (eval) здесь — повторяемая проверка на заданном случае с ожидаемыми доказательствами, решениями и запрещёнными исходами, а не субъективная оценка стиля ответа. На одном случае с диском оценка способности проверяет, правильно ли агент разобрал ситуацию; регрессионная оценка — сохранилось ли это поведение после изменения модели, инструмента или политики; оценка длинной сессии — не потерялись ли охват и полномочия после сжатия и передачи контекста; оценка эскалации — распознал ли агент необходимость передать решение человеку. Это четыре независимых вида оценки, а не четыре обязательных последовательных этапа одного теста.

Проверка	Как применяется	Обработка неуспешного результата
Обоснованность доказательствами	Для каждого существенного факта, диагноза, рекомендации и решения проверяющий компонент проходит по указанной ссылке на доказательство и проверяет происхождение, свежесть, полноту, идентичность цели и отделение наблюдения от вывода.	Если основание отсутствует, устарело, противоречиво или не подтверждает утверждение, результат получает статус `unknown` либо `incomplete`, а зависящие от него переходы блокируются.
Соответствие намерения и результата	До планирования или изменения предлагаемый результат и ограничения сопоставляются с точной версией намерения. После действия проверка результата сопоставляет наблюдаемое состояние после изменения с каждым критерием результата и требованием к остаточному риску.	Невыполненный критерий даёт `failed`, непроверенный — `unknown`. Ни один из этих исходов не принимается как успех.
Политика и полномочия	На каждом управляемом переходе точка применения политики и независимый контролёр сопоставляют исполнителя, действие, цель, воздействие, охват, версию политики, утверждение, срок его действия и требования к разделению обязанностей.	Несовпадение охвата, идентичности, полномочий, срока или политики даёт `deny` либо `escalate`; действие не предпринимается.
Оценка способности (capability eval)	Версионируемый эксплуатационный случай задаёт ограниченные входные данные, ожидаемые факты и класс решения, разрешённые выходы, запрещённые действия и эталонное правило оценки (oracle) для точной комбинации агента, модели, инструментов, политики, среды и класса действий.	Отсутствие обязательного результата или любое запрещённое поведение приводит к провалу случая.
Регрессионная оценка (regression eval)	Те же случаи повторяются после изменения агента, модели, профиля контекста, инструмента, коннектора или политики. Сопоставляются класс решения, зависимости от доказательств, ограничения, эскалация и смысл результата, а не только формулировки.	Существенное необъяснённое изменение поведения проваливает затронутый охват и запрещает повышение полномочий.
Оценка длинной сессии (session-length eval)	Случай проходит через длинную сессию, сжатие контекста, контрольную точку, передачу работы и возобновление. Проверяется сохранение идентичности случая, намерения, полномочий, политики, происхождения доказательств, бюджета попыток и оставшейся работы.	Потеря ограничений, подмена доказательств, повторное использование одобрения или нетрассируемый переход проваливают случай.

Проверка	Как применяется	Обработка неуспешного результата
Оценка эскалации (escalation eval)	Случаи намеренно включают отсутствующие, устаревшие, противоречивые, выходящие за охват и высокорисковые данные, а также обычные ситуации, которые не должны эскалироваться. Ожидаемый `deny`, `escalate` или ограниченное продолжение сопоставляется с фактическим решением.	И пропущенная обязательная эскалация, и систематическая избыточная эскалация фиксируются как дефекты качества.
Проверка границ и защиты секретов	Роли с правами только на чтение запускаются в изолированной среде; вызовы инструментов, запись файлов, сетевой доступ, использование учётных данных и состав выходных данных наблюдаются и сопоставляются с объявленными ограничениями.	Несанкционированное изменение или обращение, использование учётных данных вне охвата либо раскрытие чувствительных данных проваливают случай; для критических нарушений допустимый порог равен нулю.

Оценка на сконструированном состоянии (constructed-state evaluation) начинается с точного проверенного предшествующего состояния, добавляет единственный проверяемый стимул и оценивает получившиеся артефакты и решение. Случаи намеренно включают противоречивую историю, устаревшие или усечённые доказательства, заменённые утверждения, а также пакет контекста после сжатия или возобновления либо состояние непосредственно после вызова инструмента. Точное сравнение ограничивается каноническими полями и текстом, которыми управляет политика и которые предоставляет детерминированная обвязка; прочий вывод модели оценивается семантически. Такой снимок проверяет поведение из предоставленного состояния, но не доказывает подлинность или корректность переходов, создавших это состояние. Оценка длинной сессии ДОЛЖНА фактически выполнять переходы сжатия, создания контрольной точки, передачи и возобновления. Полное воспроизведение сессии разрешено, но не обязательно для оценки состояния на позднем этапе.

Для каждого случая оценки указываются источник и версия, класс действий, среда, диапазон воздействия, точные версии компонентов, входные артефакты, эталонное правило оценки, ожидаемое решение, обязательные доказательства и запрещённые исходы. Результат имеет статус `pass`, `fail` или `incomplete`; неполное покрытие никогда не считается успешным. Версионируемая политика может задавать статистические пороги для зависящих от среды показателей качества, но нарушения полномочий, самоутверждение, разрушительные действия роли только для чтения и раскрытие секретов нельзя скрыть усреднением.

Теневая оценка и повышение полномочий применяют ту же дисциплину к реальным случаям. До предоставления классу действий полномочий на исполнение агент проводит расследование и формирует план с `execution_authority: none`; фактическое решение инженера и действие проходят по обычному уполномоченному пути, а проверенный результат сопоставляется с теневым предложением.

Карточка типа	Контракт
Тип	`ShadowEvaluationRecord`
Вход	Точные агент, модель, набор инструментов, политика, класс действий, среда, воздействие, теневой план, решение инженера, фактическое действие и проверенный результат.
Выход	Классифицированное согласие или расхождение, выводы о безопасности и доказательства для будущего решения о повышении полномочий; никогда не полномочия на исполнение.
Ключевые поля	`shadow_ref`, `case_ref`, `agent_ref`, `model_ref`, `toolchain_versions`, `policy_ref`, `action_class`, `environment_ref`, `impact`, `agent_plan_ref`, `engineer_decision_ref`, `actual_action_ref`, `outcome_ref`, `disagreement_class`, `safety_findings`, `execution_authority`.
Схема	Отдельной нормативной схемы пока нет.

Карточка типа	Контракт
Статус контракта	Обязательная семантика записи и повышения полномочий задана здесь; покрытие схемой ожидается.

Повышение полномочий ограничивается точными классом действий, средой, диапазоном воздействия и версиями агента, модели и набора инструментов; оно никогда не распространяется на агента в целом. Метрики и доли совпадений не могут автоматически повысить техническую возможность. Требуются явное решение человека (`HumanDecision`) или решение политики, привязанные к точной цели доказательства оценки и сохранение независимости контролёра. Любое изменение охвата или версии требует повторной оценки.

21.2. Воспроизводимость

Воспроизводимость означает, что независимый рецензент может восстановить, какие сведения были известны, какие правила и полномочия действовали, как был получен результат и приводят ли те же связанные входные данные к тем же управляемым выводам. Она не означает, что живая инфраструктура перестаёт меняться или что вероятностная модель обязана побайтно повторить формулировку.

Проверка	Как применяется	Требуемый результат
Привязка входных данных	Прогон фиксирует точную идентичность цели и окно наблюдения, а также свёртки или неизменяемые ссылки на намерение, пакет доказательств, политику, схемы, манифест контекста, агента, модель, инструменты и коннекторы.	Рецензент может восстановить полный набор входных данных без догадок и скрытого разрешения ссылки на более новую версию.
Детерминированное повторение	Разборщики, проверка схем, нормализация, применение политики, контрольные ворота и канонические модули записи повторно выполняются на сохранённых входных данных.	Нормализованные факты, классификации, решения политики и канонические свёртки совпадают.
Повторение этапа с моделью	Повторно используются сохранённый манифест контекста и точные версии агента, модели, инструментов и политики. Проверяющий сопоставляет указанные факты, классификацию неопределённости, класс решения, ограничения, требуемое утверждение, эскалацию и запрещённые действия, а не внешнюю формулировку текста.	Повтор остаётся в тех же границах доказательств и полномочий; существенное смысловое расхождение фиксируется явно.
Обновление живого состояния	Если цель или авторитетный источник изменились, новые доказательства собираются как новый прогон с собственным окном наблюдения и связываются с предыдущим прогоном, а не подменяют его.	Исходная запись остаётся воспроизводимой по своему снимку, а текущее состояние представлено записью-преемником.
Обработка расхождения	Несовпадение классифицируется по источнику, свежести, контексту, модели, инструменту, политике или недетерминированности и связывается с обоими сравниваемыми прогонами.	Пока существенное расхождение не объяснено и не принято, затронутый вывод имеет статус `unknown` и не может разрешать действие.

Доказательства воспроизведения входят в пакет доказательств соответствующего случая. Сокрытие чувствительных значений (redaction) может скрывать сами значения, но сохраняет событие, наличие и тип поля, основание политики, защищённую ссылку на исходную запись и подтверждение применения правила сокрытия; иначе независимый рецензент не сможет отличить защищённые данные от отсутствующих.

21.3. Экономика применения ИИ (AI economics)

Экономика применения ИИ измеряется для каждого ограниченного эксплуатационного случая и каждого принятого владельцем проверенного результата. Объём сгенерированного текста, число вызовов инструментов или доля автоматизации сами по себе не являются полезным результатом.

Показатель	Как рассчитывается и применяется
Прямые затраты на ИИ и инструменты	Суммируется денежная стоимость использования модели, рассчитанная по учтённым токенам или вычислительным ресурсам, стоимость вызовов коннекторов и платных инструментов, а также относимые к случаю затраты на хранение и обработку данных. Используются тарифы, валюта и версия тарифов, действовавшие во время прогона.
Труд человека	Отдельно фиксируется время инженеров и рецензентов на уточнение, утверждение, расследование, исправление и эскалацию. Если оно переводится в деньги, объявляются ставка и способ расчёта; иначе человеко-часы остаются отдельным показателем, а не считаются нулевой стоимостью.
CTOR	Отношение затрат к результату (Cost-to-Outcome Ratio) — сумма объявленных прямых затрат и явно переведённого в деньги труда человека, делённая на число принятых владельцем итоговых проверок со статусом `succeeded`. Затраты на случаи `failed` и `unknown` остаются в числителе и дополнительно показываются отдельно. При нуле принятых успешных результатов CTOR не определён и помечается как «не вычисляется», а не как ноль.
Время до проверенного результата	Измеряется от времени регистрации заявки в системе учёта до конечного результата. Активная обработка, время в очереди, внешнее ожидание и ожидание решения человека показываются отдельно, чтобы задержка не приписывалась модели без основания.
Стоимость повторов и переделок	Показываются затраты и труд человека на повторный сбор, повторы модели, отклонённые рекомендации, повторно открытые случаи, откат и циклы исправления. Повтор, создающий только дополнительный текст, не создаёт ценности результата.
Нагрузка эскалаций и утверждений	Число эскалаций и пересмотров решений показывается вместе со временем рецензента, классом действий, воздействием и результатом. Изменённые, заменённые и отменённые решения учитываются отдельно там, где это существенно. Пакетное утверждение (batch approval) не устраняет стоимость или риск отдельных включённых в него случаев.

Например, 120 000 денежных единиц учитываемых затрат, делённые на 80 принятых владельцем успешных результатов, дают CTOR 1 500 денежных единиц на успешный результат. Затраты на неуспешные попытки входят в 120 000. Если труд человека не переведён в деньги, его часы показываются рядом с CTOR, а не молча приравниваются к нулю.

Метрики остаются с небольшим числом сочетаний меток (low-cardinality) и не содержат запросов к модели, персональных данных, секретов или уникальных идентификаторов случаев в метках. Экономические сравнения выполняются для сопоставимых границ сервисов, сегментов степени серьёзности и воздействия, требований к доказательствам, рабочей нагрузки и мощности команды по правилам сравнения раздела 24.

Снижение стоимости или времени первичной оценки не является улучшением, если ухудшаются полнота доказательств, безопасность решений или качество проверенного результата. И наоборот, для случая с высоким воздействием более безопасный результат может оправдывать более высокие затраты. Поэтому набор показателей совместно отражает качество, риск, время и стоимость, а одна метрика никогда не предоставляет полномочия и не приводит к их повышению.

21.4. Полные затраты на внедрение (информативное пояснение)

Оценка внедрения может дополнить состав затрат раздела 21.3 внедрением, интеграциями, обучением, сопровождением, обновлениями, сменой поставщика и трудом по сбору измерений. Регулярная работа по случаям и общие накладные расходы учитываются однократно; разовые затраты показываются целиком и, если распределяются, с явным периодом распределения. Чистое инженерное время — сопоставимый исходный труд за вычетом работы по случаям, общих накладных расходов и указанной доли затрат внедрения. Неподтверждённые оценки остаются оценками. Это дополнительное представление не меняет отношение затрат к результату (Cost-to-Outcome Ratio, CTOR), не заменяет показатели результата и качества из раздела 24 и не отменяет свидетельства завершения из раздела 14.3.

22. Как принципы реализуются в трёх плоскостях

Принципы из §4 становятся проверяемыми только тогда, когда для каждого указаны тип данных, правило перехода и независимое доказательство.

Принцип	Плоскость данных	Плоскость управления	Независимая проверка и доверие
Намерение прежде действия (Intent before action)	`OperationalIntent` предшествует сбору и плану.	Охват и полномочия ограничивают переходы.	Контрольные ворота проверяют идентичность и полноту намерения.
Доказательство прежде вывода (Evidence before inference)	Факты отделены от диагностики.	Политика определяет допустимые источники.	Независимый контролёр выявляет высказывания без доказательств.
Детерминированное ядро и исполнитель (Deterministic core and Executor)	Сбор и исполнение оставляют типизированные записи.	Исполнитель допускается только по политике и решению об утверждении.	Проверки отсутствия неразрешённых изменений, точной цели и квитанций доказывают границы.
Неизвестно ≠ в порядке (Unknown is not OK)	Непроверенный факт остаётся `unknown`.	Неизвестное воздействие или полномочие дают `deny` либо `escalate`.	Неполнота блокирует переход.
Заявленное состояние — доказательство (Declared state is evidence)	Заявленные и наблюдаемые данные не смешиваются.	Действующая политика разрешает конфликты явно.	Расхождения и признанный статус источников проверяются независимо.
Управление внутри процесса (Governance in process)	Каждый шаг оставляет `LifecycleEvent`.	Сквозной слой управляющих правил проверяет каждый переход.	Пропущенная проверка видна как разрыв трассы.
Сбор в границах (Bounded collection)	Доказательство содержит пределы и сведения об усечении.	Политика разрешает источник, сеть и профиль.	Запись покрытия доказывает фактический охват.
Обвязка конкретного проекта (Project-specific Harness)	Артефакты ссылаются на стабильные идентификаторы проекта.	Обвязка задаёт владельцев, правила и разрешения.	Валидатор обвязки проверяет целостность, не получая полномочий.
Ответственность человека (Human accountability)	Одобрение изменения хранится отдельным типом.	Только уполномоченный человек принимает требуемое решение.	Проверяются разделение обязанностей и запрет самоутверждения.
Независимая оценка (Independent evaluation)	Результат проверки адресуем и версионируем.	Автор не может отменить обязательные контрольные ворота.	Независимый контролёр и детерминированные проверки не зависят от производителя результата.

Принцип	Плоскость данных	Плоскость управления	Независимая проверка и доверие
Проверь и извлеки урок (Verify and learn)	Результат предшествует `LearningRecord`.	Изменение политики проходит новый управляемый цикл.	Итоговая проверка результата доказывает его до извлечения уроков.
Полная трасса процесса (Complete process trace)	Все попытки перехода дописываются в трассу.	Решения и основания полномочий сохраняются.	Разрыв, перестановка или подмена блокируют продвижение.

Сквозной слой управляющих правил и независимый контролёр дополняют друг друга: первый решает, допустим ли переход по действующим правилам, а второй независимо проверяет, что входы, решение и доказательство перехода корректны. Ни один из них не исполняет изменение.

23. Практический сценарий: быстрое заполнение диска

Это иллюстративный сценарий, а не отчёт об эксперименте или измерении в боевой среде. Его проценты и окно наблюдения поясняют жизненный цикл и не устанавливают измеренных улучшений.



Учебный сценарий быстро заполняющегося диска по восьми шагам Agent-Ops: от цели и 8 процентов свободного места до утверждённого изменения и проверки 24 процентов через 24 часа.

Текстовая альтернатива: намерение требует более 20 процентов свободного места без удаления данных и простоя. Доказательства показывают 8 процентов и рост логов. Диагностика возвращается за дополнительными фактами, поскольку владелец срока хранения неизвестен. План `R1/C2/I3` сначала расширяет том, затем исправляет срок хранения. Человек утверждает ровно эти два действия; детерминированный исполнитель применяет пакет. Через 24 часа проверка показывает 24 процента свободного места, исправную ротацию, отсутствие удалений и ухудшения SLO, после чего обновляются правило, проверка и журнал решения.

Сценарий показывает один и тот же случай сразу в трёх плоскостях. До повышения полномочий то же предложение можно оценить в теневом режиме по фактическим решению инженера, действию и проверенному результату, не выдавая агенту полномочий на исполнение. Проверка результата сопоставляет состояние после изменения с критериями намерения и создаёт результат с явным исходом `succeeded`, `failed` или `unknown`. Извлечение уроков использует уже проверенный результат и не

подменяет его.

Контрольная точка	Свободное место	Другие условия успеха
Цель	Более 20%	Без удаления рабочих данных и без перерыва в работе сервиса
Исходные доказательства	8%	Владелец правила срока хранения ещё требует подтверждения
Проверенное состояние после 24 часов наблюдения	24%	Ротация работает; данные не удалены; SLO не ухудшен

Это учебный пример, а не универсальный порог. Автономность R1 допускает только предложение, способность C2 — планирование, а воздействие I3 описывает изменение внутри одного боевого сервиса. Три обозначения отвечают на разные вопросы и сами по себе не предоставляют полномочий.

Шаг жизненного цикла	Плоскость данных	Плоскость управления	Независимая проверка и доверие
Намерение (Intent)	Цель — восстановить запас места выше 20% без удаления рабочих данных и без простоя сервиса.	Владелец задаёт охват, ограничения и критерии остановки.	Схема и контрольные ворота подтверждают полноту намерения и полномочия владельца.
Доказательства (Evidence)	Проверка фиксирует 8% свободного места, расход индексных дескрипторов, самые объёмные каталоги, разросшиеся логи и расход места контейнерами.	Политика разрешает только ограниченный сбор из объявленных источников.	Проверяются актуальность, полнота, усечение и идентичность сервиса.
Диагностика (Diagnosis)	Гипотеза связывает рост с правилами `logrotate` и недавней выкаткой; доказательства о владельце правила срока хранения ещё нет. Ограниченный дополнительный запрос доказательств подтверждает этого владельца до продолжения планирования.	Отсутствие подтверждённого полномочия вызывает эскалацию, а не выдуманное назначение владельца.	Независимый контролёр проверяет подтверждающие и опровергающие доказательства, проверки на опровержение и новое доказательство о владельце.
План (Plan)	Планировщик предлагает сначала временно расширить том хранения, затем исправить правило срока хранения; оценивает радиус воздействия и готовит планы проверки и отката либо компенсации.	Автономность R1, способность C2 и воздействие I3 допускают только предложение для проверки человеком; удаляющая очистка запрещена.	Контрольные ворота проверяют предусловия, обратимость, воздействие и соответствие намерению.
Решение об утверждении (Approval)	`HumanDecision` связывает одобрение обоих действий в указанном порядке с охватом, сроком и точной версией плана.	Подтверждённый уполномоченный владелец утверждает или отклоняет изменение; самоутверждение невозможно.	Проверяются подпись, срок, полномочия, разделение обязанностей и неизменность входов.

Шаг жизненного цикла	Плоскость данных	Плоскость управления	Независимая проверка и доверие
Контролируемое изменение (Controlled Change)	Отдельный исполнитель применяет только утверждённые расширение тома и исправление правила срока хранения, именно в таком порядке, с ограниченными учётными данными и записывает `ExecutionRecord` с единым для случая бюджетом и номером попытки.	Сквозной слой управляющих правил повторно проверяет политику и формирует квитанцию допуска; автоматический размыкатель блокирует исполнение при нарушении бюджета или правила периода ожидания.	Проверяются точная цель, свёртка пакета, основание идемпотентности либо компенсации и фактически выполненные действия; квитанция исхода связывается с той же попыткой.
Проверка результата (Verification)	После 24 часов наблюдения доказательства показывают 24% свободного места, исправную ротацию логов, отсутствие удаления рабочих данных и отсутствие ухудшения SLO. Итоговая проверка записывает `succeeded` и связывает факты с точным изменением и свёртками доказательств.	Политика требует выполнения всех критериев и запрещает закрытие при `unknown`.	Независимая проверка подтверждает состояние после изменения, период наблюдения, полноту критериев и остаточные риски.
Извлечение уроков (Learning)	Результат, решение и свёртки доказательств связываются; предлагаются обновления эксплуатационной инструкции и политики.	Каждое изменение политики или инструкции проходит новый управляемый цикл.	Трасса подтверждает, что извлечение уроков основано на проверенном результате и не переписывает историю.

ИИ объясняет и планирует. Управление ограничивает. Человек утверждает. Исполнитель применяет. Итоговая проверка доказывает результат.

23.1. Небольшая команда и владелец сервиса (информативный пример)

Информативный пример — небольшая команда и владелец сервиса. Инженер совмещает разработку и дежурство; другой компетентный участник с необходимыми полномочиями обеспечивает требуемую независимую проверку. Этот участник может быть внешним. Владелец сервиса задаёт приоритет и допустимые границы, назначает ответственного за коммуникацию с пользователями. Эти функции не требуют отдельных подразделений, но смена названия роли у одного человека не создаёт независимости.

Первое намерение может требовать только проверенного отчёта о росте занятого места. Тогда подтверждённый отчёт может выполнить намерение без изменения целевой системы. Если же намерение требует восстановления сервиса, а предложенное изменение отклонено, трасса фиксирует отказ и результат относительно исходных критериев; она не выдумывает запись исполнения и не отмечает восстановление как успешное. При недоступности обязательной независимой проверки команда продолжает только разрешённые сбор доказательств, анализ, подготовку и эскалацию.

После случая владелец получает проверенный результат, остаточные риски и предложения по надёжности или разработке. Правка инструкции остаётся предложением до завершения применимых проверки и допуска. Когда разработка впоследствии изменит продукт, критерии эксплуатационной приёмки можно проверить заново на новых доказательствах, связанных с прежним случаем.

Это учебный пример, а не доказательство реализации среды исполнения. Ограниченный профиль C4 в Приложении G охватывает изолированную, восстанавливаемую тестовую среду вне production; ни этот пример, ни прохождение офлайн-валидаторов не расширяют её границы.

24. Критерии успеха применения методологии

Успех оценивается сравнением сопоставимых периодов до внедрения Agent-Ops и после его внедрения и стабилизации. Основной показатель — количество исполненных единиц работы: заявок и инцидентов, доведённых до явного конечного результата, принятого ответственным владельцем и подтверждённого предусмотренными для этого случая доказательствами. Принятые владельцем конечные результаты `succeeded` и `failed` могут учитываться как завершённая работа, но их классы показываются отдельно; случаи `unknown` или с неполными доказательствами завершёнными не считаются. Публикуются как абсолютное число, так и число единиц работы за единицу времени. Рядом обязательно показываются входящий поток, изменение очереди и трудоёмкость команды, чтобы рост закрытий за счёт накопленного перечня незавершённых работ или дополнительных ресурсов не выглядел улучшением методологии.

До начала сравнения фиксируются одинаковая длительность окон, границы сервисов, источники данных, правила включения и исключения, сегменты по тяжести и воздействию, а также доступная мощность команды в человеко-часах. Исходный период берётся до внедрения, последующий — после согласованного периода стабилизации. Если состав сервисов, команда или поток работ существенно изменились, результаты показываются отдельно, а не объединяются в одну величину.

Показатель	Сравнение до и после внедрения
Исполненные единицы работы — основной показатель	Количество заявок и инцидентов с принятым конечным результатом за окно и на единицу времени. Одновременно показываются поступившие единицы, изменение очереди, человеко-часы и распределение по тяжести и сервису.
Время первичной оценки	Время от регистрации заявки в системе учёта до первого принятого диагностического заключения либо управляемого решения о следующем шаге. Сравниваются медиана и 90-й перцентиль по тяжести и сервису; ожидание ответа заказчика учитывается отдельно.
Полнота доказательств	Число требуемых для данного случая элементов, по которым имеется авторитетное, полное и актуальное доказательство, делённое на общее число требуемых для данного случая элементов. Доказанно неприменимые элементы исключаются; `unknown`, устаревшие, конфликтующие и отсутствующие элементы остаются в знаменателе, показываются отдельно и не входят в числитель.
Переделки	Число повторно открытых или отклонённых диагнозов и планов, а также циклов исправления, делённое на число принятых случаев; дополнительно сравнивается среднее число повторных подач на случай.
Безопасные решения	Решения, прошедшие применимые проверки политик и полномочий без неразрешённого действия, делённые на все решения в охвате. Отдельно показываются отказы, эскалации, нарушения политик, инциденты и откаты; отсутствие зарегистрированного инцидента не доказывает безопасность.
Подтверждённые результаты	Итоговые проверки результата (Outcome Checks) с доказательством состояния точной цели и явным исходом `succeeded`, `failed` или `unknown`, делённые на все утверждённые намерения, для которых срок проверки приходится на окно сравнения. Утверждённое намерение, не дошедшее до проверки, остаётся в знаменателе и показывается как отсутствующий или `unknown` результат; достигнутыми считаются только `succeeded`.
Готовность к повышению полномочий	Для каждой точной комбинации класса действий, среды, диапазона воздействия и версий агента, модели и набора инструментов показывается число теневых случаев с полными решениями инженера, фактическими действиями и проверенными результатами, делённое на все подходящие теневые случаи. Отдельно с абсолютными числителями и знаменателями публикуются безопасные и небезопасные совпадения, расхождения по классам, нарушения политик, пересмотры решений и исходы `unknown`. Ни один порог не повышает техническую возможность автоматически.

Рост числа исполненных единиц работы считается успехом применения методологии только при сохранении или улучшении полноты доказательств и безопасности решений. Значения до и после публикуются вместе с абсолютными числителями и знаменателями, а не только в процентах.

25. Внедрение по этапам

Эта таблица — план внедрения, а не порядок переходов отдельного случая. В каждом случае намерение, охват, полномочия и необходимые доказательства по-прежнему определяются до соответствующей работы, как установлено разделами 6–7. Более позднее положение в плане внедрения не откладывает эти предусловия.

Этап	Результат
0. Границы и управление (Boundaries + governance)	Области боевой среды, классификация данных, полномочия, список запретов и базовые точки применения политики.
1. Фундамент доказательств (Evidence foundation)	Профили `audit-fast`/`audit-regular`, схемы, сокрытие чувствительных данных и проверка точного коммита.
2. Заявленное / наблюдаемое (Declared / observed)	Источники описания инфраструктуры кодом, действующая политика и классифицированное расхождение.
3. Проектная обвязка (Project Harness)	Владельцы, цели качества сервиса, политики, эксплуатационные инструкции, разрешения, устойчивые идентификаторы и реестр политик.
4. Готовность наблюдаемости (Observability readiness)	Покрытие телеметрией, свежесть, оповещения и диагностические ссылки.
5. Намерение и доопределение задачи (Intent + Discovery)	Гипотеза о результате, карта решений с участием человека и план проверки.
6. Пакет доказательств и оценки (Evidence Bundle + evals)	Версионируемая трасса решений, четыре класса оценок и проверка результата.
7. Инцидент и анализ корневых причин (Incident/RCA)	Хронология, гипотезы, противоречия и цикл доопределения задачи.
8. Планировщик и независимый контролёр (Planner + Guardian)	Предложение, проверенное человеком, независимая оценка и безопасная передача работы.
9. Теневая эксплуатация	Расследование и планирование на реальных случаях с `execution_authority: none`; сравнение предложения агента с решением инженера, фактическим действием и проверенным результатом по классам действий.
10. Применение через контрольные ворота	Отдельный исполнитель только для явно повышенных в правах и утверждённых ограниченных действий.
11. Ограниченное самовосстановление	Только доказанно безопасные повторяемые сценарии; по умолчанию выключено.

> *Пилот -> стандартизация -> масштабирование. Каждый этап закрывается доказательством и не наследует полномочия следующего.*

Команда может устойчиво получать пользу от аудита, диагностики и подготовки изменений без перехода к исполнению. Прогресс оценивается по проверенным результатам, сохранению контроля и полным трудозатратам команды, а не только по автономности. Каждая дополнительная способность сохраняет применимые требования к оценке и допуску. C5 остаётся будущим режимом, выключенным по умолчанию; каждое реальное изменение целевой системы в текущих эксплуатационных границах сохраняет явное утверждение человеком и отдельный запуск детерминированного исполнителя человеком.

26. Ожидаемый эффект и позиционирование

Agent-Ops следует позиционировать как методологию эксплуатации серверов, изначально построенную вокруг ИИ (AI-native Server Operations Methodology), и каркас, в котором эксплуатационные правила хранятся и версионируются как код (Operations-as-Code framework), предназначенный для поддержки безопасной эксплуатации и технической поддержки серверной, облачной и Kubernetes-инфраструктуры.

Следующие преимущества ожидаются, но не являются доказанными сравнительными результатами:

- снижение зависимости от неформального знания отдельных инженеров;
- повторяемая первичная оценка и единый стандарт между проектами;
- быстрое выявление отсутствующих доказательств и пробелов в наблюдаемости;
- более безопасное использование больших языковых моделей в эксплуатации боевой среды;
- проверяемые планы изменений вместо свободных рекомендаций;
- связность артефактов аудита, инцидента, изменения и разбора после инцидента;
- масштабирование управляемых услуг без выдачи агентам универсальных прав.

Процентные улучшения без измерений не заявляются. Будущие научные заявления зависят от сравнения с сильной композицией существующих механизмов при согласованных предпосылках. Если преимущество не установлено, вклад описывается как операционный профиль, эталонная реализация или методика измерения — в соответствии с фактически созданными артефактами. Любое наблюдаемое преимущество ограничивается исследованными средами и классами отказов.

> *Agent-Ops: намерение задаёт результат; доказательства подтверждают реальность; управление ограничивает; человек утверждает; исполнитель применяет; итоговая проверка подтверждает результат.*

27. Двуязычная нормативная публикация

Английский нормативный документ имеет смысловой приоритет. У каждого включённого в каталог английского документа есть ровно один русский документ той же исходной ревизии. Структурный паритет проверяется машиной; смысловая эквивалентность требует назначенного владельцем внешнего двуязычного рецензента.

Перевод МОЖЕТ уточнять термин по-русски, но он ДОЛЖЕН сохранять публичные идентификаторы, значения схем и статусов, нормативную силу и смысл контракта. В обычном русском изложении сначала приводится понятное русское название, а английский термин МОЖЕТ следовать в скобках при первом введении или в начале самостоятельного раздела. Буквальные имена типов и полей, значения статусов, зарегистрированные названия продуктов и названия источников сохраняются точно. Утверждённое исключение указывает точную пару, ревизию, неизменяемую цель, различающуюся структуру и рецензента.

Замкнутый набор публикации состоит из этого аналитического документа, карты стандартов и управляемого глоссария. Манифесты пар и публикации определяют точных участников и различают артефакты-кандидаты, проверенные, сгенерированные и ожидающие обработки.

28. Статус этой редакции

Редакция v0.5.2, исходная ревизия `aom-04-r9`, заменяет редакцию v0.5.1 и, как и она, публикует два независимо версионируемых профиля: ограниченный C4 Human-approved Apply (применение после одобрения человеком) и Evidence Acquisition (сбор доказательств). Здесь нормативным кандидатом являются их стабильные идентичности, охват, явные нецели, границы допущений и заявлений, доверенные вычислительные базы, границы общих причин отказа, отдельные ограниченные исполняемые формальные модели состояний и переходов, независимо версионируемые каталоги отношений и небольшой семантический интерфейс. Эта редакция также публикует ограниченные срезы реализации: шесть схем записей C4 и `agent-ops.relation-validation@1.0.0`, пять схем Evidence Acquisition и `agent-ops.evidence-relation-validation@1.0.0`, а также их композиционную проверку `agent-ops.integrated-conformance@1.0.0` с зафиксированным протоколом оценки до запуска среды исполнения. Положительные, схемно-негативные, межзаписно-негативные и парные межпрофильные фикстуры остаются синтетическими и офлайн. Редакция не реализует среду исполнения, не предоставляет полный валидатор жизненного цикла, не сообщает результат оценки работающей среды и не устанавливает безопасность боевой среды. Добавленные в v0.5.1 требования к идентичности агента, опорному состоянию и остановке сессии, совокупным ограничениям и сессионной среде исполнения рассуждающего компонента (§3.3, §11.2, §16.2, §19.1, §19.3, §20.1, §20.3) имеют статус `target` и не добавляют срезов реализации; Приложение G и формальные модели не изменяются. Эта редакция уточняет границы случая, организационную ответственность, завершение без изменения и внедрение без обязательного роста автономности; добавляет информативные пояснения о небольшой команде, продолжении работы при потере ответа и полных затратах на внедрение. Эти уточнения и примеры не меняют профили, схемы, каталоги отношений, семантический интерфейс, формальные модели, полномочия на исполнение или требования к свидетельствам завершения. Метки редакции на иллюстрациях согласованы с v0.5.2. Парные PDF v0.5.2 отображают эту исходную ревизию и имеют доказательства детерминированного Build/Test, но ни эти артефакты, ни предыдущие ревью не одобряют преемник. Для выпуска по-прежнему нужны новые привязанные к точной цели ревью владельца и двуязычное ревью, полные доказательства выпуска и воспроизводимости, а также отдельное разрешение для защищённой среды выпуска. Само наличие в репозитории не делает редакцию выпущенной.

Приложение A. 47 базовых проверок

Идентификаторы `check_id` не переводятся: это буквальные значения в артефактах и коде. Домены проверок: **инвентаризация** (Inventory) — что есть на хосте; **здоровье и нагрузка** (Health); **службы и процессы** (Services); **сеть** (Network) — сеть, порты и сертификаты; **логи и ротация** (Logs / Rotation); **дисковое пространство и очистка** (Disk / Cleanup); **резервные копии и восстановление** (Backups / Restore); **готовность мониторинга** (Monitoring readiness).

Домен	check_id
Инвентаризация	inventory.host.identity
Инвентаризация	inventory.os.version
Инвентаризация	inventory.kernel.version
Инвентаризация	inventory.cpu.summary
Инвентаризация	inventory.memory.total
Инвентаризация	inventory.disk.layout
Инвентаризация	inventory.time.sync
Здоровье и нагрузка	health.cpu.load_average
Здоровье и нагрузка	health.cpu.iowait
Здоровье и нагрузка	health.memory.pressure

Домен	check_id
Здоровье и нагрузка	health.memory.swap_usage
Здоровье и нагрузка	health.memory.oom_events
Здоровье и нагрузка	health.systemd.failed_units
Службы и процессы	services.systemd.active_inventory
Службы и процессы	services.systemd.enabled_inventory
Службы и процессы	services.systemd.restart_count
Службы и процессы	services.processes.orphan_long_running
Службы и процессы	services.containers.inventory
Сеть	network.ports.listening
Сеть	network.ports.bind_address
Сеть	network.firewall.status
Сеть	network.dns.resolution
Сеть	network.tls.certificate_expiry
Логи и ротация	logs.journald.disk_usage
Логи и ротация	logs.journald.retention_policy
Логи и ротация	logs.logrotate.configuration
Логи и ротация	logs.files.oversized
Логи и ротация	logs.docker.rotation
Логи и ротация	logs.system.high_severity_errors
Дисковое пространство и очистка	disk.filesystem.free_percent
Дисковое пространство и очистка	disk.filesystem.inode_usage
Дисковое пространство и очистка	disk.usage.largest_directories
Дисковое пространство и очистка	disk.usage.large_files
Дисковое пространство и очистка	disk.cleanup.package_cache
Дисковое пространство и очистка	disk.cleanup.docker_usage
Резервные копии и восстановление	backups.jobs.presence
Резервные копии и восстановление	backups.last_success.age
Резервные копии и восстановление	backups.coverage.critical_data
Резервные копии и восстановление	backups.retention.policy
Резервные копии и восстановление	backups.integrity.last_check
Резервные копии и восстановление	backups.restore.last_test
Готовность мониторинга	monitoring.node_exporter.presence
Готовность мониторинга	monitoring.textfile_collector.configured
Готовность мониторинга	monitoring.audit_metrics.freshness
Готовность мониторинга	monitoring.prometheus.scrape_visibility
Готовность мониторинга	monitoring.alerts.baseline_rules
Готовность мониторинга	monitoring.logs.collection_readiness

Приложение B намеренно поддерживается как самостоятельный [глоссарий Agent-Ops](../normative/glossary.ru.md), чтобы управляемый словарь можно было версионировать и рецензировать как отдельную двуязычную пару. Поэтому в этом документе после Приложения A следует Приложение C.

Приложение C. Материалы, положенные в основу

- AI-Disrupt PDLC, полное практическое руководство, версия 2.0, Сбер, июнь 2026 года; автор методологии — Кирилл Меньшов. Таблица §4.3 — источник лестницы R0-R5; §4.5 — источник обозначений и последовательности T1-T5, адаптированных к границе независимого контроля Agent-Ops в §5. T5 источника и отдельный исполнитель Agent-Ops не эквивалентны. §2.11 содержит подход к пакету доказательств завершения, адаптированный в §14.3; §2.13 — предшествующий подход к сквозному слою управляющих правил для §18. Подходы источника к управлению контекстом и цепочке поставки контекста также используются в §20. Это операционные адаптации, а не заявления об изобретении или принятии жизненного цикла разработки продукта из источника. Основание атрибуции — английская редакция: <https://aipdlc.ru/documents/en/whitepaper_full_en.pdf>; русский перевод: <https://aipdlc.ru/documents/ru/whitepaper_full_ru.pdf>.
- OWASP Agentic Skills Top 10 (AST01-AST10), версия 1.0, 2026 — источник требований редакции 0.3.2 к цепочке поставки навыков, квитанциям аудита и записи покрытия проверки: <https://owasp.org/www-project-agentic-skills-top-10/>.
- Ganesh Gurudu, «Building an AI Agent That Runs Your SRE Operations — What I Learned, What Works, and How You Can Do It Too» («Создание ИИ-агента, который ведёт SRE-эксплуатацию: извлечённые уроки, рабочие подходы и способ повторить»), 8 апреля 2026 года — информативный источник опыта реализации для теневой оценки, бюджета попыток, наблюдаемости среды исполнения агента, сборки контекста и допуска интеграций. Он не создаёт зависимости от архитектуры, шкалы риска, топологии размещения или стека технологий из статьи: <https://blog.stackademic.com/building-an-ai-agent-that-runs-your-sre-operations-what-i-learned-what-works-and-how-you-can-do-8a3801124bdc>.
- Ali Shazal и Matthew Koen, Anthropic, «A guide to the anatomy of effective commerce agents» («Руководство по устройству эффективных торговых агентов»), 2 сентября 2026 года — информативный источник за пределами эксплуатационного домена для границы между предложением и утверждением, авторитетных идентификаторов, сериализованной записи и ограничений результирующего состояния, детерминированного отделения контекста и оценки на сконструированном состоянии. Он не создаёт зависимости от торговой архитектуры, выбора модели, приёмов снижения задержки или стека развёртывания из статьи: <https://claude.com/blog/the-anatomy-of-effective-commerce-agents>.
- NIST SP 800-207 «Zero Trust Architecture» («Архитектура нулевого доверия»), август 2020 года — первоисточник принципов явной проверки, минимальных привилегий и допущения компрометации, на которые ссылается §3.3: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf>.
- Alexander Klizhentas и Diana Jovin, Teleport, «From Zero Trust to Agent Trust» («От нулевого доверия к доверию агентам»), 2 июля 2026 года — информативный источник за пределами методологии для идентичности агента, аттестуемой относительно предоставившего полномочия субъекта, делегирования, эфемерной среды исполнения и ограничения совокупной автономии. Он не создаёт зависимости от продукта, платформы идентичности или топологии размещения из документа: <https://goteleport.com/resources/white-papers/zero-trust-to-agent-trust/>.
- Внешние стандарты, нормативные акты и карта соответствий поддерживаются в самостоятельной [карте стандартов Agent-Ops](../normative/standards-map.ru.md).

Приложение D. Нормативный контракт результата

Каждая проверка формирует самостоятельный результат, проверяемый по схеме. Сводка, выдача результатов в формате метрик Prometheus и производные представления данных, предназначенные для передачи модели (AI-facing projections), не могут менять его семантику. Если такое представление усечено, факт и причина усечения фиксируются отдельно.

Поле	Обязательная семантика
`check_id`	Стабильный идентификатор логической проверки.
`role` / `technology`	Применимость к роли, платформе и набору правил под конкретную платформу (dialect pack).
`status`	Ровно одно значение: `ok`, `info`, `warning`, `critical`, `unknown`, `not_applicable` или `error`.
`severity`	Нормативная шкала степени серьёзности S0-S5; не является автономией, воздействием или зрелостью.
`applicable` (применимость)	Явная применимость; отсутствие поставщика данных не означает `not_applicable` автоматически.
`evidence_refs`	Проверяемые ссылки с источником, временной меткой, свежестью, полномочиями источника и свёрткой. Ссылка не доказывает действительность сбора или достаточность для используемого заявления без оценки применимого требования к доказательству.
`policy_source` (источник правила)	Применённое правило, версия, владелец и свёртка.
`recommendation`	Предложение только в виде плана, без права его исполнить; никогда не утверждение и не полномочия на исполнение.
`completeness` / `truncation`	Полнота, физические пределы, отброшенные элементы и причина сокращения.
`schema` / `tool identity`	Версии схемы, инструмента, профиля и канонического модуля записи.

Приложение E. Семантика статусов результата проверки

Ось	Значение	Интерпретация
status	ok	Проверка применима, выполнена, доказательств достаточно, политика соблюдена.
status	info	Нейтральный факт, не являющийся нарушением.
status	warning	Подтверждённое отклонение, требующее внимания.
status	critical	Подтверждённое критическое отклонение.
status	unknown	Доказательств недостаточно либо они устарели, неполны или противоречивы.
status	not_applicable	Проверка доказанно неприменима к нормализованной роли.
status	error	Технический сбой сбора, разбора, оценки или модуля записи.

«Неизвестно» не равно «в порядке»; техническая ошибка сбора не равна неизвестности; «неприменимо» нужно доказать, а не предположить.

(Unknown is not OK; error is not unknown; not_applicable requires proven inapplicability.)

Приложение F. Реестр шкал и пространств имён

Пространство имён (namespace) шкалы — замкнутый набор значений с собственным смыслом. Полные определения каждого уровня и порядок применения приведены в §5.1–5.7; этот реестр перечисляет все значения и ведёт к соответствующему определению.

Пространство имён	Все значения	Определение и применение
M0-M6	M0 — неуправляемый процесс / неизвестное состояние; M1 — разовый аудит; M2 — регулярная диагностика; M3 — выстроенная наблюдаемость; M4 — управляемая эксплуатация; M5 — полуавтоматическое восстановление под утверждением человека; M6 — ограниченное самовосстановление и поддержка надёжности с участием ИИ, будущий режим.	[Зрелость процесса, §5.1](#51-зрелость-процесса-m0-m6). Не предоставляет полномочий.
S0-S5	S0 — информационная; S1 — низкая; S2 — умеренная; S3 — значимая; S4 — высокая; S5 — критическая.	[Серьёзность результата проверки, §5.2](#52-серьёзность-результата-проверки-s0-s5). Учитывает риск неопределённости; не заменяет статус и не доказывает отклонение.
C0-C5	C0 — наблюдать; C1 — объяснять; C2 — планировать; C3 — готовить изменение; C4 — применять после одобрения человеком; C5 — ограниченное самовосстановление, будущая возможность, выключенная по умолчанию.	[Техническая возможность, §5.3](#53-техническая-возможность-c0-c5); граница исполнения — §18.2. Возможность не создаёт полномочий.
I0-I5	I0 — без воздействия на боевую среду, только чтение / работа без подключения к сети; I1 — изолированное обратимое изменение вне боевой среды; I2 — локальное обратимое изменение низкой критичности; I3 — изменение внутри одного боевого сервиса; I4 — межсервисное, критичное или чувствительное изменение; I5 — системное или необратимое воздействие, потеря данных или регуляторные последствия.	[Операционное воздействие, §5.4](#54-операционное-воздействие-i0-i5): границы и меры контроля для каждого уровня; допуск — §18.
R0-R5	R0 — только чтение; R1 — только предложения; R2 — отдельная ветка с разбором запроса на слияние; R3 — автоматическое слияние после успешных оценок; R4 — многосессионная работа с контрольными точками; R5 — полная автономия в изолированной песочнице.	[Автономность, §5.5](#55-автономность-r0-r5); применение политик — §18. Источник — таблица §4.3 AI-Disrupt PDLC 2.0, указанная в §5.
T1-T5	T1 — разбор; T2 — проверка; T3 — перенаправление; T4 — блокировка; T5 — организация исправления через отдельно уполномоченный путь.	[Возможности независимого контролёра, §5.6](#56-возможности-независимого-контролёра-t1-t5). Адаптация §4.5 AI-Disrupt PDLC 2.0; контролёр не получает полномочий на изменение цели.

[Пример совместного применения всех шести шкал, §5.7](#57-применение-шкал-на-одном-случае), поясняет разные объекты оценки. Одинаковые номера разных шкал не взаимозаменяемы.

Внешние источники и сопоставления, ранее находившиеся в Приложении J, поддерживаются в самостоятельной [карте стандартов Agent-Ops](../normative/standards-map.ru.md).

Приложение G. Ограниченный профиль C4 применения после одобрения человеком 1.0.0

G.1. Статус, идентичность и охват

Стабильный публичный идентификатор этого профиля — `agent-ops.c4-human-approved-apply@1.0.0`. Настоящее приложение является нормативным контрактом его границы, допущений, словаря заявлений и доверенной вычислительной базы и вместе с версионируемыми артефактами в `formal/` задаёт ограниченный исполняемый кандидат. Шесть схем C4 и публичный валидатор отношений реализуют только именованный автономный межзаписный срез. Реализация НЕ ДОЛЖНА заявлять runtime-conformance (соответствие работающего исполнения) этому профилю только на основании настоящего приложения, ограниченной формальной модели состояний и переходов, соответствующих схемам записей или успешно пройденных фикстур.

Профиль охватывает одно применение одного канонического `ChangePackage` (пакета изменения), одобренного человеком, отдельно запущенным человеком детерминированным исполнителем через контрольные ворота. Пакет связывает одно намерение и каноническую идентичность Outcome (результата), точный UID цели, типизированную операцию, канонические аргументы, критичные для решения зависимости, интервал действия, эпохи политики и утверждения, а также бюджет попыток. Планировщик и контролёр не имеют учётных данных записи в цель. Доверенный шлюз или взаимодействующий адаптер серверной стороны исполнения проверяет полномочия в момент использования и допускает либо отклоняет операцию в объявленной точке линеаризации. Авторитетный долговечный журнал сохраняет события жизненного цикла; наблюдатель эффектов только для чтения классифицирует фактические изменения цели и полноту наблюдения независимо от исполнителя.

Исходный эталонный домен содержит только заранее объявленные типизированные операции над изолированной, сбрасываемой, небоевой тестовой целью. Каждая операция объявляет семантику идентичности цели, версий, допуска, идемпотентности или сверки, домен конфликтов, поддержку отсечения устаревшего писателя, охват наблюдения и оракул Outcome (результата). Внедрение МОЖЕТ реализовать более узкое подмножество, но НЕ ДОЛЖНО молча его расширять.

В профиль 1.0.0 не входят:

- постоянные полномочия C5 или автоматически созданное от имени человека утверждение;
- произвольные shell-команды, неограниченные скрипты или необъявленные сторонние API;
- боевые либо необратимые эффекты и общая изоляция от невзаимодействующих писателей;
- заявления о семантике «ровно один раз» для произвольного внешнего эффекта;
- заявления о восстановлении, пока требуемые плоскость управления, наблюдатель или оракул Outcome (результата) недоступны;
- заявления, что одна проверка схемы устанавливает межзаписные полномочия, темпоральную безопасность, безопасность реализации, научную новизну или готовность к боевой эксплуатации.

G.2. Словарь гарантий и правила заявлений

Следующие понятия не взаимозаменяемы:

Термин	Управляемое значение
безопасность допуска (admission safety)	Операция допускается в объявленной точке принудительного исполнения только тогда, когда предикат полномочий профиля выполнен для точных пакета, цели, попытки, эпох политики и утверждения и времени использования.
знание об эффекте (effect knowledge)	Классифицированное знание `applied`, `not_applied`, `partial` или `unknown` о связанных с попыткой изменениях цели вместе с охватом наблюдения. Это не состояние попытки и не Outcome (результат).

Термин	Управляемое значение
успешность Outcome (результата)	Канонический оракул Outcome (результата) выдаёт `succeeded` относительно исходного Intent (намерения) и связанных доказательств состояния после изменения. Допущенная или применённая операция может не достичь успеха.
восстановление (recovery)	Каждое допущенное неразрешённое обязательство в итоге получает авторизованное терминальное решение, а требуемое состояние сервиса или теста восстанавливается при заявленных допущениях доступности и обратимости. Безопасная остановка не является восстановлением.
безопасность повтора (retry safety)	Повторная доставка не создаёт эффекта за пределами заявленной семантики операции, потому что действует тот же ключ идемпотентности серверной стороны исполнения, полное наблюдение доказывает отсутствие прежнего эффекта либо авторизованная сверка определяет следующее действие. Одна компенсация не доказывает безопасность повтора.
безопасность конфликтов (conflict safety)	Взаимодействующие писатели с пересекающимися записями цели или критичными для решения чтениями сериализуются либо отклоняются в объявленном домене принудительного исполнения, а устаревшие писатели отсекаются.
непрерывность (continuity)	Попытки, резервирования, метки отсечения, эпохи утверждения и политики и неразрешённые эффекты переживают контрольную точку, передачу, перезапуск и истечение аренды без сброса или пропуска.
полнота доказательств (evidence completeness)	Наблюдатель явно фиксирует, что он охватил, исключил или не смог определить. Отсутствие наблюдаемого изменения при неполном охвате не означает `not_applied`.

Заявления безопасности и живучести ДОЛЖНЫ формулироваться раздельно. Каждое заявление ДОЛЖНО называть версию профиля, допущения, доверенные компоненты, домен принудительного исполнения, точное семейство операций и класс целей, охват наблюдения и доказательства проверки. Снятие или невыполнение допущения сужает либо отменяет заявление и НЕ ДОЛЖНО скрываться агрегированной долей успеха. `unknown`, частичные эффекты, тайм-ауты и неполное наблюдение остаются видимыми.

Настоящее приложение устанавливает этот словарь и приведённые ниже допущения. G.7 предоставляет ограниченные исполняемые определения; они становятся гарантиями работающей среды, которые можно заявлять, лишь после принятия точного кандидата и его обязательств соответствия и предоставления реализацией актуальных доказательств для точной цели.

G.3. Каталог допущений

Каждое допущение имеет стабильный идентификатор. Реализации ДОЛЖНЫ объявлять для каждого применимого допущения `satisfied`, `not_satisfied` или `unknown` вместе с доказательством; зависимое заявление поддерживает только `satisfied`.

ID	Допущение и требуемое доказательство	Последствие снятия, невыполнения или неизвестности для заявления
C4-A01	Цель изолирована, допускает сброс, не является боевой и принимает только зарегистрированный набор типизированных операций. Доказательство называет класс цели и процедуру сброса.	Применимость профиля и оценка восстановления не установлены.
C4-A02	Каждый писатель, способный пересечься с защищённой записью или критичным для решения чтением, взаимодействует с объявленными доменами шлюза, резервирования и отсечения. Писатели вне домена перечислены и наблюдаются.	Безопасность допуска и конфликтов действует только внутри взаимодействующего домена; внешний писатель может её нарушить.

ID	Допущение и требуемое доказательство	Последствие снятия, невыполнения или неизвестности для заявления
C4-A03	Шлюз или адаптер серверной стороны исполнения имеет документированную атомарную точку допуска, проверяющую связанный предикат полномочий до принятия изменения.	Проверка перед вызовом даёт только локальное доказательство политики; атомарную удалённую авторизацию заявлять нельзя.
C4-A04	Серверная сторона исполнения предоставляет неизменяемые UID целей и монотонные либо иным образом устойчивые к ABA критичные для решения версии ресурсов.	Повторное использование имени или ABA может перенаправить утверждение либо представить устаревшее доказательство актуальным.
C4-A05	Каждая операция объявляет семантику идемпотентности серверной стороны исполнения, повтора с тем же ключом, частичного эффекта, отмены, сверки и отсечения.	Нельзя выводить безопасность повтора, отмены или устаревшего писателя.
C4-A06	Авторитетный журнал жизненного цикла дополняется без перезаписи, долговечно упорядочен и атомарен для каждого принятого события; повреждение и недоступность обнаружимы.	Нельзя доверять отсутствию и порядку событий, расходованию попыток и неразрешённым обязательствам.
C4-A07	Хранилище резервирований обеспечивает долговечные атомарные получение и обновление и монотонные метки отсечения в объявленном домене конфликтов.	Конкурирующие либо просроченные писатели могут оба действовать; безопасность конфликтов и непрерывность нарушаются.
C4-A08	Идентичности субъекта, подписанта и политики и жизненный цикл ключей аутентифицированы; ключи подписи защищены, а отзыв видим в момент использования.	Происхождение пакета или утверждения и действительность полномочий не установлены.
C4-A09	Планировщик, контролёр, наблюдатель и обычные пользователи не имеют учётных данных записи в цель; учётные данные исполнителя ограничены операцией, целью, временем и попыткой.	Обходной путь может создать неатрибутированный или неавторизованный эффект.
C4-A10	Источники времени, толкование границ действия и максимальная неопределённость часов объявлены. Допуск консервативно использует наиболее раннее возможное истечение и наиболее позднее возможное `not_before`.	Кажущееся действительным доказательство может быть просрочено или ещё не вступить в силу в точке допуска.
C4-A11	Наблюдатель эффектов только для чтения объявляет охват ресурсов, полей, времени и внешних эффектов и сообщает `complete`, `partial` или `unknown`. Он операционно отделён от пути ответа исполнителя.	Отсутствие наблюдаемого изменения не доказывает `not_applied`; знание об эффекте остаётся `unknown`.
C4-A12	Канонический оракул Outcome (результата) связан с исходным Intent (намерением), UID цели, доказательствами состояния после изменения и правилами завершения; объявлена его независимость от пути операции.	Знание об эффекте нельзя повысить до успешности Outcome (результата), а восстановление сервиса нельзя заявить.
C4-A13	Внешние контроллеры, асинхронные механизмы сверки, контрольные обработчики допуска и побочные эффекты серверной стороны исполнения перечислены вместе с их доменами записи и наблюдения.	Заявления об атрибуции, классификации конфликтов и результирующем состоянии исключают необъявленные контроллеры.
C4-A14	Допущения доступности явно заданы для сервиса идентичности и политик, журнала, хранилища резервирований, шлюза или серверной стороны исполнения, наблюдателя и оракула Outcome (результата). Потеря требуемого сервиса там, где это обеспечивается принудительно, приводит к безопасному запрету допуска.	Безопасность может выродиться в остановку; итоговое завершение, сверка и восстановление во время недоступности не гарантируются.

ID	Допущение и требуемое доказательство	Последствие снятия, невыполнения или неизвестности для заявления
C4-A15	Необратимые эффекты исключены. Каждая допущенная операция имеет объявленную процедуру сброса либо отдельно авторизованный путь компенсации или сверки, остаточные эффекты которого наблюдаемы.	Заявления о восстановлении и ограниченном остаточном риске недействительны.
C4-A16	Компрометация учётных данных, ключа подписи, шлюза или серверной стороны исполнения, журнала, хранилища резервирований, наблюдателя либо оракула находится вне гарантии соответствующего компонента, отдельно обнаруживается и обрабатывается.	Скомпрометированный компонент доверенной базы может подделать или скрыть факт, принудительное исполнение либо наблюдение которого ему доверено.
C4-A17	Контрольные точки и сводки производны от авторитетных долговечных состояний журнала и резервирований и могут только уточнять их. При возобновлении сначала читается авторитетное состояние.	Перезапуск или передача могут сбросить попытки, эпохи, резервирования, метки отсечения либо неразрешённые обязательства.
C4-A18	Алгоритмы канонизации и свёртки версионируются, детерминированы, устойчивы к коллизиям для профиля и одинаково толкуются компонентами утверждения и допуска.	Разные пакеты могут получить или выглядеть имеющими одну идентичность утверждения.
C4-A19	Одобряющий и запускающий люди аутентифицированы, видят каноническую идентичность пакета и заявленные последствия и действуют по применимому правилу организационных полномочий.	Действительная подпись или событие интерфейса не устанавливает ответственное человеческое полномочие на операцию.

G.4. Доверенная вычислительная база

Планировщик и вывод модели являются недоверенными входами предложения и не входят в доверенную базу допуска. Контролёру не доверяется создание полномочий; ему доверяются только точные независимые проверки, входы и пути которых удовлетворяют таблице. Каждая реализация ДОЛЖНА публиковать эквивалентную таблицу для конкретного внедрения.

Компонент	Граница принудительного исполнения или наблюдения	Допущение об отказе или компрометации	Требуемая независимость	Ограничение заявлений
Центр идентичности и политик	Аутентифицирует субъектов и предоставляет актуальные организационные полномочия, эпоху политики, отзыв и охват.	Доступность объявляется; поддельная идентичность, устаревшая эпоха или компрометация отменяют заявления о полномочиях.	Отделён от планировщика и пути записи в цель; отзыв проверяется в момент использования.	Не доказывает целостность пакета, допуск серверной стороной, эффект или Outcome (результат).
Подписант и хранилище ключей	Подписывает точное каноническое решение человека или аттестацию и защищает ключевой материал.	Извлечение ключа, неавторизованная подпись или скрытый отзыв отменяют происхождение.	Ключ утверждения недоступен планировщику, контролёру и исполнителю.	Подпись доказывает источник и целостность, но не безопасность или использование серверной стороной.

Компонент	Граница принудительного исполнения или наблюдения	Допущение об отказе или компрометации	Требуемая независимость	Ограничение заявлений
Канонизатор и реализация свёртки	Формирует байтовую идентичность, которую человек одобряет и которая позднее проверяется при допуске.	Расхождение версий, неоднозначность или коллизия отменяют связывание пакета.	Компоненты утверждения и допуска МОГУТ быть разными экземплярами, но ДОЛЖНЫ использовать один принятый алгоритм и версию и общие тестовые векторы.	Не устанавливает полномочия, свежесть или эффект.
Авторитетный долговечный журнал	Упорядочивает неизменяемые факты жизненного цикла, попытки, допуски, классификации эффектов, квитанции, резервирования, сверку, компенсацию и происхождение контрольных точек.	Предполагаются атомарность и долговечность принятого события; обнаружимая недоступность там, где требуется, ведёт к безопасному отказу.	Идентичности писателей аутенти фицированы; наблюдатель и исполнитель не могут переписать историю.	Записанное утверждение не доказывает внешний эффект без объявленного наблюдателя.
Хранилище резервирований и отсечения	Сериализует объявленные домены конфликтов и выдаёт монотонные метки отсечения.	Предполагаются атомарное долговечное обновление и принудительное применение метки серверной стороной.	Исполнитель не может сбросить эпоху или создать принятую метку вне политики.	Защищает только взаимоде йствующих писателей и объявленные конфликты чтения и записи.
Шлюз или взаимодействующий адаптер серверной стороны	Проверяет полномочия в момент использования и допускает либо отклоняет типизированную операцию в объявленной точке линеаризации; применяет предусловия версии, ключ и метку отсечения.	Его семантика допуска доверенна; компрометация или обходной писатель отменяют безопасность допуска.	Нет учётных данных планировщика или контролёра; идентичности политики и пакета независимо проверяемы.	Отдельная проверка перед вызовом не подменяет эту границу.
Наблюдатель эффектов	Читает цель и объявленные внешние поверхности, атрибутирует изменения и сообщает охват и `applied`, `not_applied`, `partial` или `unknown`.	Пробелы охвата и недоступность явны; компрометация может подделать знание об эффекте.	Учётные данные только для чтения и путь наблюдения, независимый от ответа исполнителя.	Не может предоставить полномочия или самостоятельно решить Outcome (результат).
Рабочая нагрузка и оракул Outcome (результата)	Оценивает исходное Intent (намерение) по каноническому состоянию после изменения и доказательствам сервиса или теста.	Предполагаются корректность оракула и заявленный охват; недоступные или противоречивые доказательства дают `unknown`.	Отделён от планировщика и ответа на изменение; общие источники данных раскрываются.	Успех Outcome (результата) не доказывает безопасность допуска или отсутствие остаточных эффектов.

Компонент	Граница принудительного исполнения или наблюдения	Допущение об отказе или компрометации	Требуемая независимость	Ограничение заявлений
Одобряющий и запускающий люди	Предоставляют ответственное утверждение и отдельное событие запуска точного пакета.	Предполагаются аутентифицированная идентичность, понятное каноническое представление и организационные полномочия.	Не могут быть заменены агентом, метрикой или результатом контролёра; применимые правила самоутверждения сохраняются.	Действие человека не исправляет устаревшее доказательство, подмену пакета, обход серверной стороны или отсутствие наблюдения.
Источники времени и версий	Предоставляют ограниченную неопределённость времени, UID цели, эпохи политики и утверждения и критичные для решения версии, используемые при допуске.	Объявляются границы, монотонность или устойчивость к ABA и доступность.	Источники авторитетны для названных фактов; общие источники являются зависимостями общей причины отказа.	Один TTL никогда не доказывает актуальность для конкретного решения.

G.5. Общие причины отказа и независимость контролёра

Запись внедрения ДОЛЖНА перечислять общие языковые модели, запросы к ним, производителей доказательств, учётные данные, ключи, канонизаторы, часы, сервисы версий, политик и идентичности, журналы, хранилища, сетевые пути, API цели, наблюдателей и источники данных Outcome (результата). Две проверки, зависящие от одного ошибочного факта, не являются независимыми доказательствами этого факта. Различие имени языковой модели, формулировки запроса, процесса или узла МОЖЕТ улучшить обнаружение, но не устанавливает независимость, пока полномочия, учётные данные, источники и пути наблюдения также не разделены для заявленного класса отказа.

Контролёр МОЖЕТ блокировать, перенаправлять или запрашивать отдельно авторизованное исправление. Он НЕ ДОЛЖЕН одобрять от имени человека, создавать или расширять учётные данные исполнителя, изменять цель, переписывать авторитетную запись, скрывать `unknown` или исправлять оцениваемый артефакт вне журналируемого жизненного цикла. Общее ложное доказательство, переданное и планировщику, и контролёру, сохранено как исполняемый контрпример и остаётся обязательной негативной трассой проверки соответствия.

G.6. Матрица заявлений

Заявление	Вид	Требуемые допущения и доверенные компоненты	Домен принудительного исполнения и текущий статус
C4-G01 Безопасность допуска точного пакета	безопасность	C4-A02-A04, C4-A08-A10, C4-A18-A19; центр идентичности и политик, подписант, канонизатор, шлюз или серверная сторона, источники времени и версий	Только взаимодействующие шлюз или серверная сторона и точная типизированная операция; ограничена связыванием точного пакета, актуальным утверждением, актуальным запуском человеком, удерживаемым резервированием, актуальной меткой отсечения и авторизованным допуском (`C4-R01`-`C4-R06`), а также P1/P2/P6; пока не является заявлением работающей среды.

Заявление	Вид	Требуемые допущения и доверенные компоненты	Домен принудительного исполнения и текущий статус
C4-G02 Безопасность конфликтов и устаревших писателей	безопасность	C4-A02, C4-A04-A07, C4-A13; хранилище резервирований и отсечения и взаимодействующая серверная сторона	Только объявленный домен конфликтов и взаимодействующие писатели; ограничена удерживаемым резервированием (`C4-R04`), актуальной меткой отсечения (`C4-R05`) и P4.
C4-G03 Безопасность повтора	безопасность	C4-A05-A07, C4-A11, C4-A15; серверная сторона, журнал, наблюдатель и авторизованный механизм сверки	Только зарегистрированное семейство операций; безопасный повтор (`C4-R08`) требует ключ backend, полное отрицательное Evidence (доказательство) или авторизованную сверку; одной компенсации недостаточно.
C4-G04 Непрерывность полномочий и бюджета	безопасность	C4-A06-A10, C4-A17-A18; журнал, хранилище резервирований, центр идентичности и политик, канонизатор, источники времени и версий	Переходы контрольной точки, передачи, перезапуска и истечения аренды; ограничена монотонным бюджетом попыток (`C4-R07`), уточнением контрольной точкой (`C4-R12`) и P3.
C4-G05 Честное знание об эффекте	эпистемическая безопасность	C4-A06, C4-A11, C4-A13; журнал и наблюдатель эффектов	Только объявленная поверхность наблюдения; обоснованное знание об эффекте (`C4-R09`) и интерфейсные отношения применимости, актуальности, охвата, диспозиции, достаточности, происхождения и общих причин (`CEI-R01-CEI-R07`) требуют сохранять `unknown` при неполном охвате; ограничена P5.
C4-G06 Целостность классификации Outcome (результата)	эпистемическая безопасность	C4-A06, C4-A12-A13; журнал и оракул Outcome (результата)	Только исходное Intent (намерение) и заявленный охват оракула; ограничена канонической классификацией Outcome (`C4-R10`) и P7.
C4-G07 Итоговое разрешение обязательств	живучесть	C4-A01, C4-A05-A07, C4-A11-A15, C4-A17; все требуемые сервисы плоскости управления и путь эскалации человеку	Разрешённое обязательство (`C4-R11`), допустимое терминальное освобождение (`C4-R13`) и L1 проверяют наличие ограниченного авторизованного пути разрешения при доступности и обратимости; неограниченная живучесть не доказана.

G.7. Исполняемые формальные модели состояний и переходов и каталог отношений

Исполняемая спецификация задаёт две композиционные версионированные формальные модели состояний и переходов: модель полномочий и эффекта C4 в `formal/c4.go` и модель утверждений и достаточности Evidence Acquisition (сбора доказательств) в `formal/evidence.go`. Это детерминированные спецификации, которые обходит проверяющая программа, а не языковые модели. Их соединяет `agent-ops.c4-evidence-interface@1.0.0` без слияния доменов полномочий. Независимо версионируемый каталог C4 — `agent-ops.c4-relations@1.0.0`; каждое отношение называет источник полномочий или авторитетного факта, момент проверки и исход отказа. Состояние C4 включает канонические свёртку и версию пакета, UID цели и критичные для решения версии, эпохи утверждения и политики, ограниченный временной интервал, события утверждения и запуска человеком, бюджет и число израсходованных попыток, домен конфликтов, резервирование и метку отсечения, состояние допуска, знание об эффекте, состояние Outcome (результата), обязательства сверки или компенсации и происхождение контрольной точки.

Идентификаторы отношений являются точными нормативными указателями, а не объяснениями для читателя. При первом упоминании текст даёт понятное название и смысл перед идентификатором; последующие компактные ссылки могут использовать только идентификатор. Следующий указатель охватывает все три каталога отношений.

ID	Понятное и каталожное название	Смысл
`C4-R01`	Связывание точного пакета (`package_bound`)	Связывает утверждение и допуск с одним каноническим пакетом, точной целью и критичными для решения версиями.
`C4-R02`	Актуальное утверждение (`approval_current`)	Требует актуальности подписанного решения человека, идентичности, политики и состояния отзыва в момент допуска.
`C4-R03`	Актуальный запуск человеком (`human_start_current`)	Требует аутентифицированного события запуска человеком до допуска первой попытки.
`C4-R04`	Удерживаемое резервирование (`reservation_held`)	Требует авторитетного резервирования объявленного домена конфликтов при резервировании и допуске.
`C4-R05`	Актуальная метка отсечения (`fence_current`)	Требует, чтобы взаимодействующая серверная сторона отклонила устаревшего писателя в момент изменения.
`C4-R06`	Авторизованный допуск (`admission_authorized`)	Требует авторизации операции взаимодействующим шлюзом или адаптером серверной стороны в объявленной атомарной точке допуска.
`C4-R07`	Монотонный бюджет попыток (`attempt_budget_monotonic`)	Не позволяет контрольной точке или перезапуску обнулить израсходованные попытки в долговечном журнале жизненного цикла.
`C4-R08`	Безопасный повтор (`retry_safe`)	Разрешает повтор только по объявленной семантике того же ключа, полному применимому отрицательному доказательству или авторизованной сверке.
`C4-R09`	Обоснованное знание об эффекте (`effect_knowledge_supported`)	Требует обосновать классификацию эффекта Evidence (доказательствами) объявленного наблюдателя; иначе эффект остаётся `unknown`.
`C4-R10`	Классифицированный Outcome (`outcome_classified`)	Требует канонического оракула Outcome (результата) и применимых доказательств; иначе Outcome остаётся `unknown`.
`C4-R11`	Разрешённое обязательство (`obligation_resolved`)	Требует, чтобы допущенный неразрешённый эффект получил авторизованное решение через сверку или компенсацию.
`C4-R12`	Уточнение контрольной точкой (`checkpoint_refines`)	Разрешает контрольной точке только уточнять, но не сбрасывать и не пропускать авторитетные состояния журнала и резервирований.

ID	Понятное и каталожное название	Смысл
C4-R13	Допустимое терминальное освобождение (terminal_release_allowed)	Освобождает случай только тогда, когда это разрешает его авторитетное состояние жизненного цикла.

ID	Понятное и каталожное название	Смысл
EVD-R01	Применимое требование (requirement_applicable)	Устанавливает применимость именованного требования получателя к сбору и использованию.
EVD-R02	Авторизованный профиль сбора (collection_profile_authorized)	Требует авторизованного версионированного профиля сбора перед каждой попыткой.
EVD-R03	Добавляемые без перезаписи попытки (attempt_append_only)	Сохраняет каждую попытку и отказ при повторе, контрольной точке и перезапуске.
EVD-R04	Связанная квитанция (receipt_bound)	Связывает квитанцию сбора с точной целью, источником и событием получения.
EVD-R05	Целостность артефакта (artifact_integrity)	Проверяет канонические байты и свёртку до допуска артефакта или использования получателем.
EVD-R06	Полное происхождение вывода (derivation_lineage_complete)	Сохраняет происхождение во всех преобразованиях, создающих артефакт или утверждение.
EVD-R07	Диспозиция утверждения (assertion_disposition)	Записывает, поддерживает или опровергает ограниченный оценщик утверждение, не проверяет его либо не может определить результат.
EVD-R08	Действительное отрицательное наблюдение (negative_observation_valid)	Принимает отрицательное утверждение только при полном применимом охвате и способности обнаружения.
EVD-R09	Актуальность при использовании (fresh_at_use)	Оценивает точные идентичности цели и источника, время и критичные версии в момент решения получателя.
EVD-R10	Обоснованная независимость (independence_supported)	Считает доказательства независимыми только тогда, когда это подтверждают объявленные зависимости от общих причин отказа.
EVD-R11	Вычисленная полнота (completion_computed)	Вычисляет полноту пакета из требования получателя и полного происхождения сбора.
EVD-R12	Запечатанный пакет (bundle_sealed)	Разрешает запечатывание только над вычисленным состоянием полноты.
EVD-R13	Достаточность для получателя (consumer_sufficient)	Оценивает достаточность по именованной версии правила получателя в момент его решения.
EVD-R14	Распространённая инвалидация (invalidation_propagated)	Переводит зависимые факты в unknown после изменения относящейся к ним версии цели, источника, профиля или политики.

ID	Понятное и каталожное название	Смысл
CEI-R01	Применимое Evidence (evidence_applicable)	Передаёт применимость требования точному решению C4 о допуске, эффекте или Outcome (результате).
CEI-R02	Актуальность Evidence при использовании (evidence_fresh_at_use)	Передаёт результат проверки актуальности Evidence точному решению получателя C4.
CEI-R03	Охват наблюдения (observation_coverage)	Передаёт охваченные поверхности и пределы обнаружения в классификацию эффекта.
CEI-R04	Диспозиция утверждения (assertion_disposition)	Передаёт ограниченную диспозицию оценщика в классификацию эффекта или Outcome (результата).

ID	Понятное и каталожное название	Смысл
`CEI-R05`	Достаточное Evidence (`evidence_sufficient`)	Передаёт достаточность для конкретного получателя; отрицательный или неизвестный результат блокирует решение.
`CEI-R06`	Связанное происхождение (`provenance_bound`)	Связывает каждый интерфейсный факт с происхождением его сбора и вывода.
`CEI-R07`	Раскрытые общие причины (`common_causes_disclosed`)	Передаёт общие зависимости, ограничивающие заявления о независимости и достаточности.

Действия формальной модели включают создание пакета, утверждение человеком, отзыв, запуск человеком, резервирование, допуск либо запрет в точке линеаризации шлюза или серверной стороны, наблюдение эффекта, потерю ответа, запрос повтора, истечение аренды, отклонение по метке отсечения, сверку, компенсацию, оценку Outcome (результата), контрольную точку, перезапуск и окончательное освобождение. Каждая модель состояний и переходов разделяет безопасность и ограниченный анализ наличия пути разрешения, явно представляет недоступные или снятые допущения, использует именованные события вместо неограниченного ожидания по настенным часам и сохраняет `unknown` как самостоятельное состояние.

Для каждого предлагаемого свойства нужен как минимум один контрпример, полученный снятием именованного допущения:

Планируемое свойство	Снятое именованное допущение	Обязательная форма контрпримера
P1 Нет допуска без точных актуальных полномочий	снять C4-A02 или C4-A03	Обходной писатель изменяет цель либо удалённая серверная сторона принимает действие после неатомарной предварительной проверки, несмотря на отзыв.
P2 Утверждение связывает один канонический пакет и точное поколение цели	снять C4-A04 или C4-A18	Удаление и пересоздание цели или расхождение канонизаторов позволяет другой UID либо набору аргументов повторно использовать утверждение.
P3 Бюджет попыток и неразрешённые обязательства монотонны	снять C4-A06 или C4-A17	Контрольная точка или перезапуск пропускает израсходованную попытку либо допущенный неразрешённый эффект и разрешает ещё одну попытку.
P4 Два конфликтующих писателя, включая устаревшего, не могут оба создать эффект	снять C4-A05 или C4-A07	Аренда истекает без принудительного отсечения серверной стороной, и старый писатель действует после допуска нового.
P5 `not_applied` выдаётся только при полном применимом наблюдении	снять C4-A11 или C4-A13	Существует ненаблюдаемый внешний контроллер либо побочный эффект, а наблюдатель сообщает отсутствие изменения.
P6 Недействительные по времени полномочия не допускаются	снять C4-A10	Расхождение часов превышает объявленную границу, и принимается истёкшее либо ещё не вступившее в силу решение.
P7 Успешность Outcome (результата) не выводится из допуска или эффекта	снять C4-A12	Запрошенное изменение произошло, но критерий исходного Intent (намерения) не выполнен, а попытка помечена успешной.
L1 Каждое допущенное обязательство в итоге получает авторизованное терминальное решение	снять C4-A14 или C4-A15	Наблюдатель или плоскость управления остаются недоступны либо необратимый остаточный эффект не имеет авторизованного пути сверки.

Исполняемая проверка использует закреплённый Go 1.26.10, детерминированный полный обход до глубины 8, не использует случайное начальное число и применяет конечные наборы событий из `formal/check.go`. На этом кандидате она посещает 1233 состояния C4 по 6014 принятым переходам и 332 состояния Evidence по 2325 принятым переходам. `formal/counterexamples.json` сохраняет

минимизированные трассы, связанные с именованными снятыми допущениями, включая три обязательные совместные трассы. Ограниченная формальная модель состояний и переходов может опровергнуть универсальное заявление; успешная ограниченная проверка не доказывает безопасность боевой среды. Точное принятие остаётся решением владельца по проверенному кандидату.

G.8. Публичный срез схем и валидации отношений

Настоящая исходная ревизия публикует шесть версионируемых схем: для `ChangePackage` (пакета изменения), `ExecutionRecord` (записи исполнения), квитанции полномочий и допуска, `Outcome` (результата), неизменяемого события жизненного цикла C4 и контрольной точки. Схемы повторно используют существующие контракты `HumanDecision` и `OutcomeCheck` через ссылки на точные версии и канонические свёртки. `agent-ops.relation-validation@1.0.0` проверяет пять упорядоченных уровней ограничений: локальное соответствие схемам; целостность ссылок и свёрток; полномочия и допуск; инварианты времени, бюджета, конфликтов и контрольной точки; независимую семантику эффекта, результата и полноты.

Каждый валидатор отношений имеет негативную трассу, отдельные записи которой всё равно проходят свои схемы JSON. Корпус отклоняет подмену пакета или цели, повторное использование UID или поколения цели, устаревшие эпохи полномочий, недействительные в момент использования доказательства, небезопасный повтор, потерю резервирования или отсечения, сброс бюджета попыток, разрыв цепочки жизненного цикла и пропуск обязательств в контрольной точке. В каталогах каждому служебному идентификатору отношения сопоставлено понятное читателю название, а результаты валидации содержат стабильные коды ошибок и идентификаторы отношений. Канонизация зафиксирована как `agent-ops.c4-json-canonicalization@1.0.0`; `HumanDecision.blocked_action` связывает канонический предмет утверждения пакета без двух полей связи с решением, чтобы избежать циклической свёртки, а связь события жизненного цикла аналогично хеширует каноническое событие без поля ссылки на предшественника.

Валидатор является автономной проверкой только этого точного среза записей. Он не аутентифицирует подписи или происхождение, не управляет шлюзом, журналом, наблюдателем, сборщиком или backend (серверной стороной исполнения), не устанавливает полное соответствие жизненного цикла, не оценивает развёрнутую систему и не доказывает безопасность боевой среды. Рабочая реализация, оценка исполнения, боевая эксплуатация, платная оценка, исследование с участием людей и выпуск требуют отдельных применимых решений и доказательств.

Приложение H. Профиль сбора доказательств Evidence Acquisition 1.0.0

H.1. Статус, идентичность, охват и нецели

Стабильный публичный идентификатор этого профиля — `agent-ops.evidence-acquisition@1.0.0`. Настоящее приложение является его нормативным контрактом границ сбора, осей качества, семантики отказов, доверенной вычислительной базы и обработки общих причин отказа и вместе с `formal/` задаёт исполняемые отношения и переходы. Описанные в H.8 ограниченное публичное семейство записей и валидатор реализуют только этот именованный автономный срез; они не являются сборщиком рабочей среды или заявлением о соответствии боевой среды.

Профиль управляет доказательствами, собираемыми для одного явного получателя и способа использования: что требуется знать, о какой точной цели и эпохе, из каких классов источников, в каком интервале наблюдения, с каким охватом, полномочиями источника, независимостью, актуальностью, семантикой отрицательного наблюдения и блокирующим поведением. Профиль не даёт полномочий на исполнение и не превращает успешный транспорт, свёртку, подпись, запечатанный пакет или связную интерпретацию в доказательство истинности или достаточности.

H.2. Управляемая цепочка сбора

Управляемая цепочка имеет вид:

```
EvidenceRequirement -> CollectionProfile -> CollectionAttempt/Receipt -> EvidenceArtifact
-> EvidenceAssertion -> EvidenceBundle
```

Объект	Смысл в этом профиле
`EvidenceRequirement`	Задаёт получателя, предполагаемое заявление или решение, точную цель и эпоху, требуемые источники, границы времени и охвата, полномочия источника, независимость, актуальность, семантику отрицательного наблюдения и результат отсутствующих или противоречивых доказательств.
`CollectionProfile`	Версионируемый контракт сборщика, связанный со свёрткой: инструмент и конфигурация, разрешения и граница только чтения, связь источника и цели, совокупность либо выборка и начальное число, лимиты, правила повторов и ошибок, разбор, нормализация, удаление чувствительных данных, ссылки на исходные данные, охват, зависимости общей причины отказа и входы воспроизводимости.
`CollectionAttempt/Receipt`	Неизменяемая запись каждой попытки, включая неуспешные: свёртка запроса, состояния транспорта и разбора, повторы и прерывание, фактический охват, включённые и исключённые элементы, количества, усечение, отброшенные данные и преобразования. Повтор создаёт новую квитанцию.
`EvidenceArtifact`	Точные сохранённые байты или защищённая ссылка, связанные с одной квитанцией, идентичностью содержания и историей преобразований.
`EvidenceAssertion`	Ограниченное утверждение, что артефакт `supports` (поддерживает), `refutes` (опровергает), `does_not_test` (не проверяет), `conflicts` (противоречит) либо оставляет `unknown` (неизвестным) именованное заявление. Оно отличает наблюдаемое отсутствие от отсутствующих данных.
`EvidenceBundle`	Адресуемый по содержимому пакет ссылок и происхождения. Упаковка или запечатывание не создают полномочия или качество доказательств. Существующая семантика Bundle v1 (пакета версии 1) не меняется; несовместимый контракт сбора использует отдельную идентичность схемы Bundle v2.

Каждое звено ДОЛЖНО сохранять идентичности и ограничения предыдущего звена. Производное представление не может молча расширять охват, повышать класс доверия, стирать исключения, подменять точную цель или превращать `unknown` в успех.

H.3. Независимые оси качества и правило заявления

Ось качества	Обязательный вопрос
Полномочия источника	Признан ли источник для этого точного факта, области и времени?
Идентичность и связывание	Однозначно ли связаны получатель, цель, источник, эпоха и артефакт?
Целостность и происхождение	Можно ли проследить исследованные байты и каждое преобразование без необъяснимого разрыва?
Действительность сбора	Действительно ли объявленные сборщик, запрос, разрешения, границы, разбор и правила ошибок управляли попыткой?
Охват и полнота	Какие совокупность, интервал, поверхности, включения, исключения, усечение и отброшенные данные фактически покрыты?
Актуальность	Пригодно ли доказательство в момент использования для цели и каждой версии, критичной для решения?
Точность преобразований	Сохранили ли разбор, нормализация, агрегирование, удаление чувствительных данных или проекция относящиеся к заявлению смысл и ограничения?

Ось качества	Обязательный вопрос
Независимость	Какие источники, учётные данные, сборщики, анализаторы, хранилища, часы, пути, языковые модели или запросы к ним являются общими?
Поддержка заявления	Поддерживает или опровергает утверждение точное заявление, только не проверяет его, противоречит ему или остаётся неизвестным?
Воспроизводимость	Может ли рецензент восстановить входы, профиль, квитанции, артефакты, преобразования и ограниченный результат?

Получатель оценивает достаточность относительно собственного `EvidenceRequirement`; доказательство не бывает глобально достаточным. Каждая требуемая ось ДОЛЖНА быть явной. Отсутствующие, противоречивые, неуспешные или непроверенные блокирующие оси дают `unknown` или неполный результат. Агрегированная оценка НЕ ДОЛЖНА компенсировать сбой полномочий источника, связывания идентичности, целостности или блокирующего охвата. Доказательство может обосновать фактическое заявление только в оценённой границе; оно НЕ ДОЛЖНО давать утверждение, полномочия на исполнение или полномочия источника.

H.4. Доверенная вычислительная база

Компонент	Обязательная ответственность и предел заявления
Полномочия источника	Устанавливают, какой источник может сообщать какой факт; не доказывают правильность источника.
Сборщик или инструмент	Исполняет связанный контракт чтения и показывает сбои и лимиты; успешный транспорт не доказывает полноту или истинность.
Сервис идентичности цели и источника	Разрешает точные идентичности и эпохи; одни псевдонимы не доказывают связывание.
Источник времени и версий	Предоставляет ограниченное время и версии, критичные для решения; общие или неопределённые источники ограничивают заявления об актуальности.
Хранилище исходных доказательств	Сохраняет точные артефакты или защищённые ссылки; хранение не создаёт полномочия.
Конвейер преобразований	Разбирает, нормализует, агрегирует и удаляет чувствительные данные с записанным происхождением; не может повысить качество источника.
Канонизатор и функция свёртки	Связывают канонические байты по одному принятому алгоритму и версии; не доказывают семантику.
Модуль записи пакета	Упаковывает ссылки и ограничения без их переписывания; запечатывание не устанавливает достаточность.
Валидатор отношений	Опубликованный валидатор C4 проверяет только принятый межзаписный срез; он не может вывести отсутствующие факты или полномочия, и универсальный валидатор Evidence Acquisition не заявляется.
Оценщик или оракул	Применяет требование получателя к утверждениям; его правильность и охват остаются именованными допущениями.

H.5. Семантика отказов и отрицательных доказательств

Сбой транспорта, отказ в разрешении, тайм-аут, сбой разбора, усечение, недостаточность выборки, потеря при удалении чувствительных данных, конфликт источников и несовпадение цели или версии являются разными исходами и ДОЛЖНЫ оставаться записанными. Успешный запрос или непустой артефакт их не стирает. Повтор является другой попыткой с другой квитанцией и не переписывает предыдущую попытку.

`not_observed` (не наблюдалось) является доказательством отсутствия только тогда, когда требование задаёт совокупность и окно, а квитанция показывает полный применимый охват и способность обнаружения. Иначе утверждение имеет состояние `does_not_test` или `unknown`, а не является отрицательным фактом. Противоречащие авторитетные источники остаются противоречивыми до применения объявленного правила разрешения; удобный выбор источника не является разрешением.

H.6. Интерфейс с ограниченным профилем C4

C4 ДОЛЖЕН использовать этот профиль, когда заявляет знание о фактическом эффекте, включая `not_applied`; классифицирует Outcome (результат) относительно исходного Intent (намерения); либо заявляет достаточность доказательств в момент допуска или использования. Наблюдатель эффектов и оракул Outcome (результата) C4 действуют как производители и получатели доказательств по явным требованиям. Их квитанции, охват, способность обнаружения, актуальность, диспозиция утверждения и ссылки на общие причины отказа ограничивают C4-G05 и C4-G06.

Профили остаются независимыми. Доказательство никогда не даёт утверждение или полномочия на исполнение, а полномочия допуска C4 никогда не доказывают истинность, полноту, актуальность или достаточность доказательства. Правомерно допущенная операция может иметь `unknown` эффект или Outcome (результат); хорошо обоснованное наблюдение может описывать неавторизованную операцию, не разрешая её. `agent-ops.c4-evidence-interface@1.0.0` переносит применимость требования и получателя, точную цель и эпоху, актуальность в момент использования с версиями, критичными для решения, охват наблюдения и способность обнаружения, диспозицию утверждения, достаточность для конкретного получателя, связывание происхождения и раскрытие общих причин отказа. Эти интерфейсные отношения (`CEI-R01`-`CEI-R07`) по отдельности сопоставлены со смыслом в указателе отношений G.7. Ограниченная запись `ExecutionRecord` C4 сериализует факты, используемые этим интерфейсом; она не является самостоятельным семейством записей или валидатором Evidence Acquisition.

H.7. Границы общих причин отказа и контрпримеры

Запись реализации ДОЛЖНА перечислять общие источники, учётные данные, сборщики, анализаторы, хранилища, часы, сетевые пути, языковые модели, запросы к ним, канонизаторы, сервисы идентичности и API цели. Два утверждения, полученные через одну дефектную зависимость, не являются независимыми доказательствами этой зависимости. Различающиеся имена процесса, языковой модели или запроса не устанавливают независимость, если общими остаются полномочия, данные, учётные данные или путь наблюдения.

Исполняемая спецификация сохраняет минимизированные контрпримеры как минимум для следующих случаев: артефакт с действующим TTL после изменения версии, критичной для решения; вывод `not_applied` из неполного охвата; успешный транспорт с усечённым или неразбираемым содержимым; запечатанный пакет с неавторитетным источником; два якобы независимых утверждения с одним ложным источником; правильное наблюдение эффекта, ошибочно принятое за успешность Outcome (результата); и действительные полномочия C4, ошибочно принятые за истинность доказательства. Они являются исполняемыми записями `formal/counterexamples.json`.

Проверка Evidence (доказательств) называет следующие свойства:

Свойство	Требуемый инвариант
E1	Актуальность требует точного связывания цели, эпохи и критичных для решения версий в момент использования.
E2	Поддерживаемое отрицательное утверждение требует полного применимого охвата и способности обнаружения.
E3	Сбой транспорта или разбора, усечение, недостаточность выборки либо потеря преобразования остаются видимыми и блокируют достаточность.
E4	Неавторитетный источник не может дать достаточный или запечатанный результат.

Свойство	Требуемый инвариант
E5	Общие либо нераскрытые причины отказа нельзя учитывать как независимые доказательства.
E6	Попытки, квитанции, сбои, конфликты и ожидающие обязательства добавляются без перезаписи при контрольной точке и перезапуске.
E7	Полномочия утверждения или допуска C4 не могут устанавливать истинность или достаточность доказательства.

H.8. Граница кандидата

Настоящий кандидат задаёт исполняемую формальную модель состояний и переходов Evidence Acquisition (сбора доказательств), связывает её через `agent-ops.c4-evidence-interface@1.0.0` с отдельной формальной моделью состояний и переходов C4 и публикует `agent-ops.evidence-relations@1.0.0`. Запись детерминированной проверки и сохранённые контрпримеры заданы в G.7 и `formal/README.md`.

Ограниченная публичная реализация добавляет схемы `EvidenceRequirement` (требования к доказательствам), `CollectionProfile` (профиля сбора), `CollectionReceipt` (квитанции сбора) с записью артефакта и `EvidenceAssertion` (утверждения о доказательстве) версии `1.0.0`, отдельную идентичность `evidence_bundle_v2.schema.json` версии `2.0.0` и `agent-ops.evidence-relation-validation@1.0.0`. Результат отдельно сообщает локальное соответствие схемам, действительность межзаписных отношений и достаточность для конкретного получателя. Корпус фикстур сопоставляет каждый негативный случай с действительной исходной парой и охватывает пустую или неверно связанную полноту, скрытую потерю при сборе, неверные цели и эпохи, устаревшие критичные версии, разорванное происхождение, недействительные отрицательные наблюдения, ложную независимость, скрытые конфликты, несоответствие свёртки и потерянную историю.

Evidence Bundle v1 остаётся действительным только по своему исходному локальному контракту. Миграция создаёт новые записи требования, профиля, квитанции и артефакта, утверждения и пакета v2; она никогда не переписывает байты v1 и не повышает поля v1 `present`, digest (свёртка) или `sealed` (запечатан) до полноты либо достаточности v2. Отсутствующая история попытки, исключения, `partial`, `unknown` или конфликта делает миграцию недостаточной. Валидатор не аутентифицирует источники, не собирает доказательства, не предоставляет полномочия исполнения, не устанавливает общую истинность и не доказывает безопасность боевой эксплуатации.

H.9. Интегрированное соответствие и протокол оценки до запуска среды исполнения

Композиционный офлайн-профиль `agent-ops.c4-evidence-integrated@1.0.0` закрепляет точные версии валидаторов C4 и Evidence и интерфейса C4-Evidence. Его результат раздельно сообщает локальное соответствие схемам, действительность отношений C4, действительность отношений Evidence, достаточность Evidence для конкретного получателя, действительность межпрофильной границы и решение получателя. `allowed` допускается только после успеха каждого компонентного результата и проекции интерфейса; некорректно сформированный вход остаётся `unknown`, а полный, но недействительный вход получает `blocked`. Композиционная проверка не может изменить отношение или код причины компонентного каталога, а агрегированная оценка не может перекрыть блокирующую находку.

Межпрофильный корпус сопоставляет каждую негативную трассу с действительной исходной парой. Он охватывает ответ Executor (исполнителя), использованный как доказательство эффекта; `not_applied` при частичном охвате; успешный Outcome (результат), выведенный из устаревшего или недостаточного Evidence; повтор на основании неполного отрицательного наблюдения; а также контрольную точку и замену пакета, скрывающие сбой сбора и неразрешённый эффект. Тесты свойств проверяют независимость результата от порядка, монотонное распространение `unknown`, безопасность некорректного входа и точные закрепления каталогов. Поля C4 `consumer_rule` используют тот же явный

формат версионированного идентификатора, что и Evidence Acquisition; это исправляет прежнее несоставимое ограничение кандидата и меняет закреплённые байты двух затронутых схем C4 без изменения их ресурсных идентичностей.

Протокол оценки и воспроизводимости до запуска среды исполнения `agent-ops.c4-evidence-evaluation@1.0.0` закрепляет все назначенные случаи, детерминированный seed (начальное значение генератора) `30404`, именованные барьеры, точные версии инструментов, моделей, запросов, профилей, схем и каталогов, снимки целей и источников, карты полномочий и ожидаемого охвата, внесение отказов, сброс, adjudication (разбор спорного результата), а также обязательные исходные и производные артефакты. `succeeded`, `failed`, `timeout`, `partial` и `unknown` остаются в знаменателе. Основные метрики раздельно охватывают неправомерные или дублированные эффекты, успешность чистых и атакующих сценариев, нарушения контрольной точки, ложную полноту, неподдержанные утверждения, неверные цель или источник, устаревшее или скрывающее потери Evidence, разорванное происхождение, ложную независимость, недействительное отрицательное доказательство и воспроизводимость снимка и метода.

Протокол задаёт, как будущая отдельно разрешённая оценка ДОЛЖНА измеряться до просмотра результатов. Он не сообщает о запуске collector (сборщика) или Executor (исполнителя), пилоте, развёртывании, частоте отказов работающей среды, безопасности боевой среды, универсальной истинности Evidence или готовности выпуска. Конечный синтетический корпус может выявить дефект или опровергнуть ограниченное утверждение, но не может установить эти более широкие заявления.

> *Примечание: этот документ описывает методологию и не заменяет нормативные машиночитаемые схемы, спецификации, политики и документацию конкретной реализации.*